

Access-Controlled Inner Product Function-Revealing Encryption

Ojaswi Acharya¹ , Weiqi Feng¹ , Roman Langrehr^{a, 2}  and
Adam O’Neill¹ 

¹ University of Massachusetts Amherst, Manning College of Information and Computer Sciences,
USA

² University of Waterloo, Canada

Abstract. We extend the concept of access control for functional encryption, introduced by Abdalla *et al.* (ASIACRYPT 2020), to *function-revealing* encryption (Joy and Passelègue, SCN 2018). Here “access control” means that function evaluation is only possible when a specified access policy is met. Specifically, we introduce *access-controlled inner-product function-revealing encryption (AC-IPFRE)* and give two applications.

On the theoretical side, we use AC-IPFRE to show that function-hiding inner-product functional encryption (FH-IPFE), introduced by Bishop *et al.* (ASIACRYPT 2015), is equivalent to IPFRE. To show this, we in particular generically construct AC-IPFRE from IPFRE for the “non-zero inner-product” (NZIP) access policy. This result uses an effective version of Lagrange’s Four Square Theorem. One consequence of this result is that lower bounds by Ünal (EUROCRYPT 2020) suggest that, as for FH-IPFE, pairings will be needed to build IPFRE.

On the practical side, we build an outsourced approximate nearest-neighbor (ANN) search protocol and mitigate its leakage via AC-IPFRE. For this, we construct a practical AC-IPFRE scheme in the generic bilinear group model for a specific access policy for ANN search. To this end, we show that techniques of Wee (TCC 2020) implicitly give the most practical FH-IPFE scheme to date. We implement the resulting outsourced ANN search protocol and report on its performance.

Of independent interest, we show AC-IPFRE for NZIP implies *attribute-hiding* small-universe AC-IPFRE for arbitrary access policies. Previous work on access control for FE did not achieve attribute hiding. Overall, our results demonstrate that AC-IPFRE is of both theoretical and practical interest and set the stage for future work in the area.

1 Introduction

FE AND FRE. Traditional encryption is an all-or-nothing affair: either a receiver has the decryption key and can recover m from its ciphertext ct_m , or it does not, and therefore learns nothing about m from ct_m . To address this, in *functional encryption* [BSW11] different receivers have different “functional keys.” Namely, a functional key sk_f associated to a function f allows the receiver to recover $f(m)$ from ct_m . One can extend to the multi-arity functions f accordingly [GGG⁺14]. Moreover, a functional encryption scheme is called *function-hiding* if sk_f hides f . A related concept to FE is *function-revealing encryption* [JP18]. Here there are no functional keys; rather, one builds a dedicated scheme

E-mail: oacharya@umass.edu (Ojaswi Acharya), weiqifeng@umass.edu (Weiqi Feng), roman.langrehr@uwaterloo.ca (Roman Langrehr), adamoneill@umass.edu (Adam O’Neill)

^aMost of the work was done while the author was affiliated with ETH Zurich.



for an n -ary g such that from ciphertexts $\text{ct}_{m_1}, \dots, \text{ct}_{m_n}$, *anyone* can recover $f(m_1, \dots, m_n)$. Here, for $n = 1$ such a scheme is trivial, unless one constrains the length of a ciphertext.

ADDING ACCESS CONTROL. A problem with deploying FE in practice is that *too much* information about the messages can be leaked to the receivers. This is because *every* functional key sk_f is “compatible” with *every* ciphertext ct_m , *i.e.* the receiver can always recover $f(m)$. In many applications, it is desirable that the sender have fine-grained control over which ciphertexts are compatible with a given sk_f . Of course, this is trivially possible using FE for *all* efficient functions, which follows from recent breakthroughs in indistinguishability obfuscation [GGH⁺13, JLS21]. However, we are interested in more practical schemes for simple functionalities.

Access control for *inner-product* FE was previously addressed by Abdalla *et al.* [ACGU20] and numerous follow-up works *e.g.* [LLW21, AGT21a, AGT21b, DDM⁺23]. The idea is to associate ciphertexts and keys to tags, *i.e.* $\text{ct}_{\mathbf{x},s}$ and $\text{sk}_{\mathbf{y},t}$ for $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_p^d$, which reveals $\langle \mathbf{x}, \mathbf{y} \rangle$ iff $a(s, t)$ for an access policy $a(\cdot, \cdot)$ fixed by the scheme. Broadly, in this work we extend their study to inner-product *FRE* (IPFRE). Moreover, even without access control, IPFRE is new to this work.

1.1 Our New Notion: AC-IPFRE

Accordingly, we first introduce (private-key) *inner-product function-revealing encryption* (IPFRE), an instance of the more general notion of function-revealing encryption [JP18, HJL⁺17]. In IPFRE, there are no function keys, and a ciphertext $\text{ct}_{\mathbf{x}}$ encrypts a vector $\mathbf{x} \in \mathbb{Z}_m^d$. Given ciphertexts $\text{ct}_{\mathbf{x}}, \text{ct}_{\mathbf{y}}$ encrypting $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_m^d$ respectively, anyone can compute $\langle \mathbf{x}, \mathbf{y} \rangle$. Security, which we formulate using both indistinguishability and simulation-based notions, requires nothing else is leaked.

As for IPFE, many applications do not need the inner-product for all pairs of vectors. For example, we will see this is case below in an application to outsourcing approximate nearest-neighbor search. To reduce leakage, we introduce *access-controlled* IPFRE (AC-IPFRE), where there is an *access policy* $a: \mathcal{T} \times \mathcal{T} \rightarrow \{0, 1\}$ such that a ciphertext $\text{ct}_{\text{tag}, \mathbf{x}}$ is associated with a tag¹ $\text{tag} \in \mathcal{T}$ in addition to a plaintext vector $\mathbf{x} \in \mathbb{Z}_m^d$, and given two ciphertexts $\text{ct}_{\text{tag}, \mathbf{x}}, \text{ct}_{\text{tag}', \mathbf{x}'}$ anyone can learn $\langle \mathbf{x}, \mathbf{x}' \rangle$ iff $a(\text{tag}, \text{tag}') = 1$. We are interested in two types of access policies. The first one is the “non-zero inner product” access policy a_{nzip} where tags are vectors in $\mathcal{T} = \mathbb{Z}_m^r$ and a outputs 1 on tag, tag' iff $\langle \text{tag}, \text{tag}' \rangle \neq 0$. We call this non-zero inner-product AC-IPFRE (NZIP-AC-IPFRE). The second one is the “small tag universe” access policy, where tags come from a polynomial-size set $\mathcal{T}$ and the access policy a is an arbitrary predicate.

1.2 Equivalence of IPFRE and FH-IPFE

Our first application of AC-IPFRE is to show that IPFRE is in fact *equivalent* to function-hiding IPFE (FH-IPFE). This result has implications for assumptions that imply IPFRE, because FH-IPFE constructions are believed to require bilinear maps; in particular, lattice-based constructions will be difficult [Üna20].

FROM FH-IPFE TO IPFRE. We first show that IPFRE can be constructed generically from FH-IPFE, where an IPFRE ciphertext for the vector $\mathbf{x}$ consists of an FH-IPFE ciphertext and an FH-IPFE functional key for the same vector $\mathbf{x}$. This idea goes back to [PR12, AAB⁺15].

FROM IPFRE TO FH-IPFE. The other direction is more difficult. Here, we first show that we can generically equip an IPFRE with the non-zero inner product (NZIP) access

¹We use the term “tag” and not “attribute”, because here each ciphertext can only have one tag. This is in contrast to attribute-based encryption, where a ciphertext can have many attributes.

policy. From a technical perspective, the construction exploits the algebraic relation of tensor and inner product. If we further restrict the set of tags $\mathcal{T} \subseteq \mathbb{Z}_m^r$ such that it satisfies for all $\mathbf{t}, \mathbf{t}' \in \mathcal{T}$ $\langle \mathbf{t}, \mathbf{t}' \rangle \in \{0, 1\}$, our construction also hides the tags. This might seem restrictive at first, but we show that even with this restriction we can realize arbitrary small tag universe access policies, by constructing a suitable vector $\mathbf{v}(\text{tag})$ for each tag tag . Interestingly, a key ingredient of this construction is *Lagrange's four square theorem*.

In more detail, we can represent any access policy by an undirected graph where the nodes are the tags and there is an edge between nodes $\text{tag}_1, \text{tag}_2$ if $a(\text{tag}_1, \text{tag}_2) = 1$. For every non-loop edge $e = \{\text{tag}_1, \text{tag}_2\}$ with $\text{tag}_1 \neq \text{tag}_2$ we add one dimension with index i to the tag vectors and set $\mathbf{v}(\text{tag}_1)_i = \mathbf{v}(\text{tag}_2)_i = 1$ and 0 for all other tags. These vectors already satisfies $\langle \mathbf{v}(\text{tag}_1), \mathbf{v}(\text{tag}_2) \rangle \in \{0, 1\}$ and $\langle \mathbf{v}(\text{tag}_1), \mathbf{v}(\text{tag}_2) \rangle = 1 \iff a(\text{tag}_1, \text{tag}_2) = 1$, as desired. The challenging part is to enforce $\langle \mathbf{v}(\text{tag}_1), \mathbf{v}(\text{tag}_1) \rangle \in \{0, 1\}$, depending on whether a loop for tag_1 exists in the access graph or not. To do this, let $s := \langle \mathbf{v}(\text{tag}_1), \mathbf{v}(\text{tag}_1) \rangle$ and s' is $-s$ or $-s + 1$ (depending on whether $a(\text{tag}_1, \text{tag}_1) = 1$ or not). Our goal is now to find a vector $\mathbf{v}'(\text{tag}_1) \in \mathbb{Z}_q^\ell$ for some ℓ with $\langle \mathbf{v}'(\text{tag}_1), \mathbf{v}'(\text{tag}_1) \rangle = s'$. With such a vector we can add additionally ℓ dimensions to the tag vectors and append $\mathbf{v}'(\text{tag}_1)$ to $\mathbf{v}(\text{tag}_1)$ and fill it with zeros in $\mathbf{v}(\text{tag}')$ for all vectors $\text{tag}' \neq \text{tag}_1$. Performing this step for all tags, will give us vectors that encode the access policies as needed for the NZIP access policy.

The problem of finding $\mathbf{v}'(\text{tag}_1)$ with $\langle \mathbf{v}'(\text{tag}_1), \mathbf{v}'(\text{tag}_1) \rangle = s'$ is equivalent to writing s as a sum of squares $s' = a_1^2 + a_2^2 + \dots + a_\ell^2$. A classical result due to Lagrange guarantees us that a solutions with $\ell \leq 4$ exists and it can also be computed efficiently [RS86, PT18].

Finally, we can turn an AC-IPFRE (for small tag universe) into an FH-IPFE by using two different tags $\text{tag}_1, \text{tag}_2$. An FH-IPFE ciphertexts consists of an AC-IPFRE ciphertext with tag_1 and an FH-IPFE functional key consists of an AC-IPFRE ciphertext with tag_2 . The access policy is satisfied if exactly one of the two tags is tag_1 .

GENERALIZATIONS. All our results also apply to orthogonality revealing encryption. In particular, we also obtain equivalence of orthogonality-revealing encryption and function-hiding predicate-encryption for orthogonality. Moreover, our results apply to schemes that only recover the inner product in the exponent of a group element (as typical for pairing-based schemes). Formally, we show our results for any scheme that only depends on the inner-product of the two inputs. We call this an *inner-product-based* functionality. We then present our formal results for inner-product-based functional encryption (IPBFE) and inner-product-based function revealing encryption (IPBFRE). Finally, we can also achieve a variant of AC-IPBFRE that supports adding tags dynamically by applying our transformations to unbounded FH-IPBFE [TT18, DP19].

1.3 Using AC-IPFRE for Inner-Product Similarity

Many applications, especially in machine learning, information retrieval, and statistics rely on a notion of *inner-product similarity* between data vectors (which can be defined in several ways). Privacy of the data vectors are a natural concern. AC-IPFRE can be used to add privacy to such applications while protecting the data vectors in a fine-grained manner. We give two concrete examples.

OUTSOURCING APPROXIMATE NEAREST-NEIGHBOR SEARCH. A number of state-of-the-art retrieval models for search engines, *e.g.*, dense retrieval models [KZ20, ZZV22, XXL⁺21], use *nearest neighbor search* to retrieve documents in response to a search query [YNL21]. These models learn distributed representations (vectors) for queries and documents using deep neural networks and compute their inner-product similarity. As these applications deal with nearest neighbor search on massive data collections, *exact* nearest neighbor search, which simply uses brute-force search, is fundamentally impractical. They instead rely on *approximate* nearest-neighbor (ANN) search that indexes the vectors in a data structure.

Recent such protocols can run efficiently even on billion scale collections, see [JDJ19]. Outsourcing storage and ANN search on the vectors to a powerful server is also desirable. To this end, we adopt the notion of an outsourced database (ODB) system from [BKK⁺21] and adapt the formalism to cover ANN. We refer to the resulting outsourced ANN protocol as ANN-ODB.

For privacy-preserving outsourcing of *exact* nearest neighbor search, Kim *et al.* [KLM⁺18] propose to use FH-IPFE. However, FH-IPFE only supports *brute-force* search. Using IPFRE, ANN search on encrypted data can work in exactly the same way as state-of-the-art algorithms for plaintext data (except for how an inner-product between two encrypted vectors is computed). Namely, a user encrypts its vectors under IPFRE, sends the resulting ciphertexts to the server who, using IPFRE evaluation, indexes them into an ANN data structure. Later, the user can encrypt a query vector and send this ciphertext to the server, who using IPFRE evaluation can perform fast search.

Unfortunately, this protocol leaks all pairwise inner-product across data, query, and update vectors. To mitigate the leakage, we can use AC-IPFRE with tag-space $\{\text{data}, \text{query}, \text{update}\}$ and access policy a_{ann} defined as $a_{\text{ann}}(\text{tag}_1, \text{tag}_2) = 1$ unless $\text{tag}_1 = \text{tag}_2 = \text{data}$ or $\text{tag}_1 = \text{tag}_2 = \text{query}$. The idea is to split the database into vectors in the initial dataset and those later added dynamically. The user indexes its initial vectors in *plaintext*, then encrypts each one with tag **data**, sending the resulting data structure to the server. To query, the user sends an encryption of its query vector with tag **query**. To update the database, the user sends the encryption of the update vector with tag **update**. This access policy allows the server to learn only the inner-products necessary to maintain the data structure and answer the queries. Note that the data-structure can leak some information about the initial data vectors to the server. In the case of HSNW [MY20], the inner-product comparison bits of the initial vectors are leaked, *i.e.*, from three such encrypted vectors $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$, the server can tell if $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle \geq \langle \mathbf{x}_1, \mathbf{x}_3 \rangle$.

We formalize the leakage profile of our AC-IPFRE based ANN search protocol and show that any FH-IPFE can be upgraded to an AC-IPFRE for a_{ann} more efficiently than going through our general transform to small tag universe AC-IPFRE. We provide an implementation and evaluation of our ANN search protocol using the TinyFHIPFE scheme discussed below. We demonstrate that for typical workloads query processing takes about 24s with 20ms of active client time. In comparison, a prior protocol based on oblivious data structures [BT21] involves many rounds of interaction between the client and server, lasting over 8 minutes. On the other hand, we do not hide access pattern.

PEARSON'S CORRELATION COEFFICIENTS. In scenarios where we want to encrypt datasets, but still reveal some statistics about these datasets, IPFRE is also a useful tool. Consider for example a hospital that wants to protect personal medical data through encryption, but reveal certain statistics about this data, like the correlation of obesity with certain diseases, without computing each of these statistics. The standard measure for correlation is Pearson's correlation coefficient $r_{\mathbf{x}, \mathbf{y}}$ and, if the data vectors $\mathbf{x}, \mathbf{y}$ are normalized to have mean 0, it can be expressed as a form of inner-product (aka. cosine) similarity, namely $r_{\mathbf{x}, \mathbf{y}} = \langle \mathbf{x}, \mathbf{y} \rangle / \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle \langle \mathbf{y}, \mathbf{y} \rangle}$, and thus be computed from the inner products of every pair of vectors, which is exactly what is revealed by IPFRE.

Moreover, it can be desirable to *not* reveal certain correlations coefficients, *e.g.*, to reduce leakage or because it is unethical. For example, revealing the correlation between skin color and intelligence quotient can be seen as unethical. This can be realized with AC-IPFRE. We leave a more in-depth treatment of this application to future work.

1.4 TinyFHIPFE: An Efficient FH-IPFE Scheme

We show that part of an FE construction of Wee [Wee20] can be adapted to give an FH-IPFE that is asymptotically and concretely more efficient than existing FH-IPFE

schemes, which we call TinyFHIPFE. We note that Wee does not consider FH-IPFE, and his FE scheme differs substantially from ours as it is public-key, supports quadratic functions, and does not achieve function hiding. TinyFHIPFE scheme is our starting point for the AC-IPFRE scheme we implement as part of our ANN-ODB protocol above.

The intuition behind TinyFHIPFE is as follows. We work in prime-order groups G_1, G_2, G_T , generated by g_1, g_2, g_T respectively, and equipped with a asymmetric bilinear pairing. The master secret key will consist of matrices. Let us first describe a partial version of the scheme, and then discuss how to complete it. Here, the master secret key will be $\mathbf{A} \leftarrow \mathbb{Z}_p^{2 \times d}$. Moreover, a function key for a vector $\mathbf{x} \in \mathbb{Z}_p^d$ will be computed as $[\mathbf{A}^T \mathbf{s}_1 + \mathbf{x}]_{g_1}$ (using the notation $[x]_g := g^x$) for $\mathbf{s}_1 \leftarrow \mathbb{Z}_p^2$. Then, a ciphertext is formed analogously but in the other source group; namely, encryption of a vector $\mathbf{y} \in \mathbb{Z}_p^d$ will be computed as $[\mathbf{A}^T \mathbf{s}_2 + \mathbf{y}]_{g_2}$ for random $\mathbf{s}_2 \in \mathbb{Z}_p^2$. Now, to evaluate inner-product, suppose we pair a function key $\text{sk}_{\mathbf{x}}$ and ciphertext $\text{ct}_{\mathbf{y}}$. Then we end up with

$$[\langle \mathbf{x}, \mathbf{y} \rangle + \mathbf{s}_1^T \mathbf{A} \mathbf{y} + \mathbf{x}^T \mathbf{A}^T \mathbf{s}_2 + \mathbf{s}_1^T \mathbf{A} \mathbf{A}^T \mathbf{s}_2]_{g_T}.$$

Then, we observe that the sum of the last three terms can be computed as the inner product of two 4-dimensional vectors: $\mathbf{s}_1 \| (\mathbf{A} \mathbf{x} + \mathbf{A} \mathbf{A}^T \mathbf{s}_1)$ and $\mathbf{A} \mathbf{y} \| \mathbf{s}_2$. So, to allow the evaluator to compute this sum, we augment the ciphertexts and function keys with encryptions and function keys for these two vectors under a “tiny” instance of the (less efficient) FH-IPFE scheme by Kim *et al.* [KLM⁺18]. Namely, we include two 4×4 matrices $\mathbf{B} \leftarrow \mathbf{S}\mathbf{L}_4(\mathbb{Z}_p)$, $\mathbf{B}^* \leftarrow (\mathbf{B}^{-1})^T$ in the master secret key and add $\mathbf{B} \cdot (\mathbf{s}_1 \| \mathbf{A} \mathbf{x} + \mathbf{A} \mathbf{A}^T \mathbf{s}_1)$ to a function key $\text{sk}_{\mathbf{x}}$ and $\mathbf{B}^* \cdot (\mathbf{A} \mathbf{y} \| \mathbf{s}_2)$ to a ciphertext $\text{ct}_{\mathbf{y}}$. Inner product evaluation in our scheme now additionally pairs these components as in the inner product evaluation for the scheme of [KLM⁺18] and uses the result to obtain $[\langle \mathbf{x}, \mathbf{y} \rangle]_{g_T}$. Finally, assuming $\langle \mathbf{x}, \mathbf{y} \rangle$ is suitably bounded, it can be recovered by taking discrete-log to the base g_T . Details of our TinyFHIPFE scheme are given in Section 5 and the security proof is given in Appendix B.

We assess the efficiency of TinyFHIPFE and compare it with the most efficient FH-IPFE schemes in the table in Table 1. Specifically, we compare to the FH-IPFE schemes of [KLM⁺18] (denoted KLM⁺) and of [KKS19] (denoted KKS). In the figure, d denotes the input dimension, $\text{mult}_{\mathbb{Z}_p}$ denotes multiplications in $\mathbb{Z}_p$, mult_{G_1} denotes multiplications in G_1 , and exp_{G_1} denotes exponentiations in G_1 . Furthermore, ‘PP’ in the last column stands for *preprocessing-friendly*, that is, discrete-log is computed with respect to a *fixed* base and hence can be made fast using preprocessing. The size of a function key and the runtime of key-generation (not shown) are comparable to the ciphertext size and runtime of encryption respectively for all FH-IPFE schemes considered.

We can see that TinyFHIPFE has the shortest master secret key for $d \geq 8$ while the sizes of its ciphertext and function keys are comparable to the shortest ones. Similarly, the runtime of the encryption and key generation algorithms are linear in the number of multiplication in $\mathbb{Z}_p$ compared to quadratic number of multiplication in $\mathbb{Z}_p$ in KLM⁺ and a linear number of multiplications in the group G_1 and G_2 in KKS. Note that multiplications in $\mathbb{Z}_p$ are much faster than multiplications in G_1 and G_2 .

Table 1: Performance comparison for TinyFHIPFE.

Scheme	msk size	ct size	Encryption runtime	PP
KLM ⁺ [KLM ⁺ 18]	$(2d^2)_{\mathbb{Z}_p}$	$(d+1)_{G_1}$	$d^2 \cdot \text{mult}_{\mathbb{Z}_p} + (d+1) \cdot \text{exp}_{G_1}$	✗
KKS [KKS19]	$(6d+4)_{\mathbb{Z}_p}$	$(2d+8)_{G_1}$	$(2d+2) \cdot \text{mult}_{G_1} + (7d+12) \cdot \text{exp}_{G_1}$	✓
TinyFHIPFE	$(2d+32)_{\mathbb{Z}_p}$	$(d+4)_{G_1}$	$(4d+16) \cdot \text{mult}_{\mathbb{Z}_p} + (d+4) \cdot \text{exp}_{G_1}$	✓

1.5 Discussion and Open Problems

Our work lends insight into the relationship between FE and FRE, which historically developed along different paths. Joye and Passelègue [JP18] cast FRE as a weakening

of multi-input FE (MIFE) [GGG⁺14]. That is, while MIFE supports decryption keys associated to multi-input functions chosen on-the-fly, FRE supports a particular multi-input function fixed at setup.

We define AC-IPFRE with “ideal leakage,” meaning only the inner product values are leaked. In such a case, despite the fact that our schemes are private-key, even without function hiding public-key assumptions are necessary [HLOW24]. However, unlike FE, it is common in the literature for FRE constructions to have less-than-ideal leakage. An interesting direction is to construct AC-IPFRE with limited leakage that is more efficient or is based on weaker assumptions. A particularly fascinating question is how much leakage one must tolerate to rely on only symmetric-key primitives. Note that Fuchsbaauer *et al.* [FGHO22] build distance-comparison-preserving encryption for Euclidean distance based on symmetric-key primitives; however, their scheme is only shown to resist specific attacks.

To further reduce the leakage in our outsourced ANN search protocol, an open problem is to build FRE for inner-product *comparison*. Indeed, ANN search only requires such comparison and does not need to learn the raw inner-products. We also leave it for future work to analyze the quality of our protocol’s leakage profile under recent frameworks such as [BGW24]. Finally, access control may be useful for other forms of FRE.

1.6 Other Related Work

INNER PRODUCT FUNCTIONAL ENCRYPTION. IPFE (for inner-product recovered in the exponent) was first introduced by Abdalla *et al.* [ABDP15], with the function hiding case first considered by Bishop *et al.* [BJK15]. Bishop *et al.* give a weakly function-hiding scheme under SXDH. Other FH-IPFE schemes include a SIM-secure scheme by Kim *et al.* [KLM⁺18] proven in the generic bilinear group model, and IND-secure schemes under SXDH by Tomida *et al.* [TAO16] and Datta *et al.* [DDM16]. These schemes can be plugged into our generic transform to give AC-IPFRE. A scheme from class groups [CLT18] does not recover inner-product in the exponent and hence supports unrestricted inner-product, but is not function hiding.

FUNCTION-REVEALING ENCRYPTION. FRE grew out of work on deterministic encryption [BBO07], order-preserving encryption [BCLO09], and property-preserving encryption [PR12]. FRE was first defined (independently) by Joye and Passelègue [JP18] and Haagh *et al.* [HJL⁺17].² Joye and Passelègue study FRE for orthogonality and cardinality of set intersection, while Haagh *et al.* study FRE for partial ordering and its application to skyline queries. None of these works consider inner-product.

1.7 Organization of the Paper

We start with preliminaries in Section 2. Then, in Section 3, we present the definitions of our two new primitives: inner-product based function-revealing encryption (IPBFRE) in Section 3.1 and access-controlled IPBFRE in Section 3.2, as well as inner-product-based functional encryption in Section 3.3. In Section 4 we present generic constructions of AC-IPBFRE for the non-zero inner product functionality in Section 4.2 and for small tag universes in Section 4.3. We present TinyFHIPFE, our function-hiding inner-product functional encryption scheme in Section 5. In Section 6 we present an AC-IPFRE for a_{ann} access policy in Section 6.1 and our ANN-ODB protocol with its leakage profile in Section 6.4. Finally, in Section 7, we give details and results of our experimental evaluation.

²“Preserving” vs. “revealing” terminology reflects whether the evaluation operation on ciphertexts is required to be the same as the evaluation operation on plaintexts.

2 Preliminaries

NOTATION. If $\mathbf{v}$ is a vector then $|\mathbf{v}|$ is its length (the number of its coordinates) and $\mathbf{v}[i]$ is its i -th coordinate. For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_m^d$ we use $\langle \mathbf{x}, \mathbf{y} \rangle := \mathbf{x}^\top \mathbf{y}$ to denote the standard scalar product. For vectors $\mathbf{x} \in \mathbb{Z}_m^d, \mathbf{y} \in \mathbb{Z}_m^r$ we use $\mathbf{x} \otimes \mathbf{y} := \text{vec}(\mathbf{x}\mathbf{y}^\top)$ to denote the standard tensor product for vectors. Here, vec is the vectorization operator takes a matrix and returns a vector obtained by stacking all columns of the input matrix on top of each other. By $\mathbf{x} \parallel \mathbf{y}$ we denote the concatenation of vectors $\mathbf{x}, \mathbf{y}$ length-wise.

Strings are identified with vectors over $\{0, 1\}$. By ϕ we denote the empty string or vector. If S is a finite set, then $|S|$ denotes its size. If X is a finite set, we let $x \leftarrow^* X$ denote picking an element of X uniformly at random and assigning it to x . Algorithms may be randomized unless otherwise indicated. If A is an algorithm, we let $y \leftarrow A^{O_1, \dots}(x_1, \dots; \omega)$ denote running A on inputs $x_1, \dots$ and coins ω , with oracle access to $O_1, \dots$, and assigning the output to y . By $y \leftarrow^* A^{O_1, \dots}(x_1, \dots)$ we denote picking ω at random and letting $y \leftarrow A^{O_1, \dots}(x_1, \dots; \omega)$. We let $\text{Out}(A^{O_1, \dots}(x_1, \dots))$ denote the set of all possible outputs of A when run on inputs $x_1, \dots$ and with oracle access to $O_1, \dots$. Running time is worst case, which for an algorithm with access to oracles means across all possible replies from the oracles. We use $\perp$ (bot) as a special symbol to denote rejection, and it is assumed to not be in $\{0, 1\}^*$.

A function $\nu: \mathbb{N} \rightarrow \mathbb{N}$ is *negligible* if for every positive polynomial $p: \mathbb{N} \rightarrow \mathbb{R}$ there is a $\lambda_p \in \mathbb{N}$ such that $\nu(\lambda) \leq 1/p(\lambda)$ for all $\lambda \geq \lambda_p$. “PT” stands for “polynomial time,” whether randomized or deterministic. By 1^λ we denote the unary representation of the integer security parameter $\lambda \in \mathbb{N}$.

GAMES. We use the code-based game-playing framework of BR [BR06]. By $\Pr[G \Rightarrow y]$ we denote the probability that the execution of game G results in the output being y . In games, integer variables, set variables, boolean variables and string variables are assumed initialized, respectively, to 0, the empty set $\emptyset$, the boolean false and $\perp$.

3 Definitions

We focus in this work on inner-product based functionalities.

Definition 1 (Inner-product based function). A function $f: \mathbb{Z}_m^d \times \mathbb{Z}_m^d \rightarrow \mathcal{Y}$ (for an arbitrary codomain $\mathcal{Y}$) is *inner-product based* if it can be expressed as $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$ for a function $g: \mathbb{Z}_m \rightarrow \mathcal{Y}$.

Some examples of inner-product based functions of interest are the function computing the inner product, the inner product in the exponent (where $g(x) = g_T^x$ for a group generator g_T of the target group of a pairing) or the function testing for orthogonality.

3.1 Inner-Product Based Function Revealing Encryption

SYNTAX. A (private-key) *inner-product based function-revealing encryption (IPBFRE) scheme* for a functionality $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$ is a tuple of PT algorithms $\text{IPBFRE} = (\text{PPGen}, \text{Setup}, \text{Enc}, \text{Eval})$ that work as follows:

- $\text{PPGen}(1^\lambda)$: On input a unary encoding of the security parameter $\lambda \in \mathbb{N}$, the public parameters generation algorithm outputs public parameters pp (in particular including a modulus $m \in \mathbb{Z}_+$), which is implicitly input to all algorithms.
- $\text{Setup}(1^d)$: On input a unary encoding of the plaintext dimension $d \in \mathbb{N}$, the setup algorithm outputs a master secret key msk .
- $\text{Enc}(\text{msk}, \mathbf{x})$: On inputs a master secret key msk , and an input vector $\mathbf{x} \in \mathbb{Z}_m^d$, the encryption algorithm outputs a ciphertext ct .

Game $\mathbf{G}_{\text{IPBFRE},A}^{\text{ind-b}}(\lambda)$	Game $\mathbf{G}_{\text{IPBFRE},A}^{\text{sim-1}}(\lambda)$	Game $\mathbf{G}_{\text{IPBFRE},A,S}^{\text{sim-0}}(\lambda)$
MAIN: 1 $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$ 2 $(d, st) \leftarrow A_1(1^\lambda, \text{pp})$ 3 $\text{msk} \leftarrow \text{Setup}(1^d)$ 4 $b' \leftarrow A_2^{\text{ENCO}(\cdot, \cdot)}(st)$ 5 if $\exists j, k \leq i : g(\langle \mathbf{x}_0^j, \mathbf{x}_1^k \rangle) \neq g(\langle \mathbf{x}_1^j, \mathbf{x}_1^k \rangle) :$ 6 return $\perp$ 7 else return $(b = b')$ ENCO ($\mathbf{x}_0, \mathbf{x}_1$): 8 $i \leftarrow i + 1$ 9 $(\mathbf{x}_0^i, \mathbf{x}_1^i) \leftarrow (\mathbf{x}_0, \mathbf{x}_1)$ 10 $\text{ct} \leftarrow \text{Enc}(\text{msk}, \mathbf{x}_b)$ 11 return ct	MAIN: 1 $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$ 2 $(d, st) \leftarrow A_1(1^\lambda, \text{pp})$ 3 $\text{msk} \leftarrow \text{Setup}(1^d)$ 4 $b \leftarrow A_2^{\text{ENCO}(\cdot)}(st)$ 5 return b ENCO ($\mathbf{x}$): 6 $\text{ct} \leftarrow \text{Enc}(\text{msk}, \mathbf{x})$ 7 return ct	MAIN: 1 $C_{\text{ip}} \leftarrow \emptyset; i \leftarrow 0$ 2 $(\text{pp}, st_S) \leftarrow S_1(1^\lambda)$ 3 $(d, st_A) \leftarrow A_1(1^\lambda, \text{pp})$ 4 $st_S \leftarrow S_2(1^d, st_S)$ 5 $b \leftarrow A_2^{\text{ENCO}(\cdot)}(st_A)$ 6 return b ENCO ($\mathbf{x}$): 7 $i \leftarrow i + 1$ 8 $\mathbf{x}_i \leftarrow \mathbf{x}$ 9 for $j \leq i$ do 10 $c_{i,j} \leftarrow g(\langle \mathbf{x}, \mathbf{x}_j \rangle)$ 11 $C_{\text{ip}} \leftarrow C_{\text{ip}} \cup \{(i, j), c_{i,j}\}$ 12 $(\text{ct}, st_S) \leftarrow S_3(C_{\text{ip}}, st_S)$ 13 return ct

Figure 1: Games defining IND-security (left) and SIM-security (middle and right) for IPBFRE.

- $\text{Eval}(\text{ct}_1, \text{ct}_2)$: On input two ciphertexts ct_1, ct_2 , the evaluation algorithm outputs a string y .

Remark 1. When the function computed is inner-product i.e., $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle) = \langle \mathbf{x}, \mathbf{y} \rangle$, we call the scheme Inner-Product Function Revealing Encryption (IPFRE).

CORRECTNESS. We require *evaluation correctness* meaning that

$$\Pr[\text{Eval}(\text{Enc}(\text{msk}, \mathbf{x}_1), \text{Enc}(\text{msk}, \mathbf{x}_2)) = g(\langle \mathbf{x}_1, \mathbf{x}_2 \rangle)] = 1$$

for all $\lambda, d \in \mathbb{N}$, all $\text{pp} \in \text{Out}(\text{PPGen}(1^\lambda))$ and all $\text{msk} \in \text{Out}(\text{Setup}(\text{pp}, 1^d))$, where $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{Z}_m^d$ and probability is taken over the coins of the Enc algorithm.

INDISTINGUISHABILITY-BASED SECURITY. First, we define an indistinguishability-based security definition for IPBFRE in Figure 1. For an adversary $A = (A_1, A_2)$, define its IND-advantage for all $\lambda \in \mathbb{N}$ as:

$$\text{Adv}_{\text{IPBFRE},A}^{\text{ind}}(\lambda) = \Pr[\mathbf{G}_{\text{IPBFRE},A}^{\text{ind-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{IPBFRE},A}^{\text{ind-0}}(\lambda) \Rightarrow 1].$$

We say that an IPBFRE scheme is *IND-secure* if $\text{Adv}_{\text{IPBFRE},A}^{\text{ind}}(\cdot)$ is negligible for all PT adversaries A .

SIMULATION SECURITY. Next, we give a simulation-based security definition for IPBFRE in Figure 1. For an adversary $A = (A_1, A_2)$ and a simulator $S = (S_1, S_2, S_3)$, we define the SIM-advantage of A, S for all $\lambda \in \mathbb{N}$ as follows:

$$\text{Adv}_{\text{IPBFRE},A,S}^{\text{sim}}(\lambda) = \Pr[\mathbf{G}_{\text{IPBFRE},A}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{IPBFRE},A,S}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

We say that an IPBFRE scheme is *SIM-secure* if for every PT adversary A there exists a PT simulator S such that $\text{Adv}_{\text{IPBFRE},A,S}^{\text{sim}}(\cdot)$ is negligible.

3.2 Access-Controlled IPBFRE

We introduce IPBFRE with *access policies*, following Abdalla *et al.* in the FE setting [ACGU20]. An *access policy* is a PT function $a : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}$. Here, the arguments in the access policy are public parameters, auxiliary information³, the first tag, and the second tag respectively. We assume access policies are *symmetric* in the last two arguments meaning $a(\mathbf{pp}, \mathbf{aux}, x, y) = a(\mathbf{pp}, \mathbf{aux}, y, x)$ for all $\mathbf{pp}, \mathbf{aux}, x, y \in \{0, 1\}^*$. We introduce *access-controlled IPBFRE* (AC-IPBFRE), which is defined like IPBFRE except that ciphertexts are equipped with tags. The inner-product of the plaintexts is revealed by the evaluation algorithm if and only if the tags of the two ciphertexts combine to satisfy the given access policy.

Formally, an AC-IPBFRE for access policy a is a tuple of three algorithms $\text{AC-IPBFRE}[a] = (\text{Setup}, \text{Enc}, \text{Eval})$.

- $\text{PPGen}(1^\lambda)$: On input a unary encoding of the security parameter λ , the public parameters generation algorithm outputs the public parameters $\mathbf{pp}$ (that in particular includes the modulus $m \in \mathbb{Z}_+$). The public parameters are implicitly input to all other algorithms.
- $\text{Setup}(1^d, \mathbf{aux})$: On input a unary encoding of the plaintext dimension d and some auxiliary information about the access policy $\mathbf{aux}$, the setup algorithm outputs a master secret key $\mathbf{msk}$.
- $\text{Enc}(\mathbf{msk}, \mathbf{tag}, \mathbf{x})$: On inputs a master secret key $\mathbf{msk}$, a tag $\mathbf{tag}$, and a message vector $\mathbf{x}$, the encryption algorithm outputs a ciphertext $\mathbf{ct}$.
- $\text{Eval}(\mathbf{ct}_1, \mathbf{ct}_2)$: On input two ciphertexts $\mathbf{ct}_1, \mathbf{ct}_2$, the evaluation algorithm outputs a scalar y or $\perp$.

Remark 2. When the function computed is inner-product i.e., $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle) = \langle \mathbf{x}, \mathbf{y} \rangle$, we call the scheme Access-Controlled Inner-Product Function Revealing Encryption (AC-IPFRE).

CORRECTNESS. We require *evaluation correctness* meaning that

$$\Pr[\text{Eval}(\text{Enc}(\mathbf{msk}, \mathbf{tag}_1, \mathbf{x}_1), \text{Enc}(\mathbf{msk}, \mathbf{tag}_2, \mathbf{x}_2)) = g(\langle \mathbf{x}_1, \mathbf{x}_2 \rangle)] = 1$$

for all $\lambda, d \in \mathbb{N}$, all $\mathbf{aux} \in \{0, 1\}^*$, all $\mathbf{pp} \in \text{Out}(\text{PPGen}(1^\lambda))$, all $\mathbf{msk} \in \text{Out}(\text{Setup}(1^\lambda, \mathbf{aux}))$, all $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{Z}_m^d$ and all $\mathbf{tag}_1, \mathbf{tag}_2 \in \{0, 1\}^*$ such that $a(\mathbf{pp}, \mathbf{aux}, \mathbf{tag}_1, \mathbf{tag}_2) = 1$, where the probability is taken over the coins of the Enc algorithm.

3.2.1 Security Definitions

Our security definitions guarantee that nothing about the tags is leaked, except whether they satisfy the access policy, and nothing about the message vectors is leaked, except the inner product for pairs of vectors encrypted with tags satisfying the access policy.

We give an indistinguishability-based security definition for AC-IPBFRE in Figure 2. For an adversary $A = (A_1, A_2)$, define its IND-advantage for all $\lambda \in \mathbb{N}$ as:

$$\text{Adv}_{\text{AC-IPBFRE}, A}^{\text{ind}}(\lambda) = \Pr[\mathbf{G}_{\text{AC-IPBFRE}, A}^{\text{ind-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{AC-IPBFRE}, A}^{\text{ind-0}}(\lambda) \Rightarrow 1].$$

We say that an AC-IPBFRE scheme is *IND-secure* if $\text{Adv}_{\text{AC-IPBFRE}, A}^{\text{ind}}(\cdot)$ is negligible for all PT adversaries A .

³Looking ahead, the auxiliary information for non-zero inner product access policy will be the dimension of tag vectors.

Game $\mathbf{G}_{\text{AC-IPBFRE},A}^{\text{ind-b}}(\lambda)$
MAIN: 1 $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$ 2 $(d, \text{aux}, st) \leftarrow A_1(1^\lambda, \text{pp})$ 3 $\text{msk} \leftarrow \text{Setup}(1^d, \text{aux})$ 4 $b' \leftarrow A_2^{\text{ENCO}(\cdot, \cdot, \cdot)}(st)$ 5 if $\exists j, k \leq i : (a(\text{pp}, \text{tag}_0^j, \text{tag}_0^k) \neq a(\text{pp}, \text{tag}_1^j, \text{tag}_1^k) \vee (a(\text{pp}, \text{tag}_0^j, \text{tag}_0^k) = 1 \wedge g(\langle \mathbf{x}_0^j, \mathbf{x}_0^k \rangle)) \neq g(\langle \mathbf{x}_1^j, \mathbf{x}_1^k \rangle))$ then 6 return $\perp$ 7 else return $b = b'$ ENCO $(\mathbf{x}_0, \mathbf{x}_1, \text{tag}_0, \text{tag}_1)$: 8 $i \leftarrow i + 1$ 9 $(\mathbf{x}_0^i, \mathbf{x}_1^i, \text{tag}_0^i, \text{tag}_1^i) \leftarrow (\mathbf{x}_0, \mathbf{x}_1, \text{tag}_0, \text{tag}_1)$ 10 $\text{ct} \leftarrow \text{Enc}(\text{msk}, (\mathbf{x}_b, \text{tag}_b))$ 11 return ct

Figure 2: Game defining IND-security for AC-IPBFRE.

Game $\mathbf{G}_{\text{AC-IPBFRE},A}^{\text{sim-1}}(\lambda)$	Game $\mathbf{G}_{\text{AC-IPBFRE},A,S}^{\text{sim-0}}(\lambda)$
MAIN: 1 $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$ 2 $(d, \text{aux}, st) \leftarrow A_1(1^\lambda, \text{pp})$ 3 $\text{msk} \leftarrow \text{Setup}(1^d, \text{aux})$ 4 $b \leftarrow A_2^{\text{ENCO}(\cdot, \cdot)}(st)$ 5 return b ENCO $(\mathbf{x}, \text{tag})$: 6 $(\mathbf{x}_i, \text{tag}_i) \leftarrow (\mathbf{x}, \text{tag})$ 7 $\text{ct} \leftarrow \text{Enc}(\text{msk}, \mathbf{x}, \text{tag})$ 8 return ct	MAIN: 1 $\mathcal{C}_{\text{ip}} \leftarrow \emptyset; i \leftarrow 0$ 2 $(\text{pp}, st_S) \leftarrow S_1(1^\lambda)$ 3 $(d, \text{aux}, st_A) \leftarrow A_1(1^\lambda, \text{pp})$ 4 $st_S \leftarrow S_2(1^d, \text{aux}, st_S)$ 5 $b \leftarrow A_2^{\text{ENCO}(\cdot, \cdot)}(st_A)$ 6 return b ENCO $(\mathbf{x}, \text{tag})$: 7 $i \leftarrow i + 1$ 8 $(\mathbf{x}_i, \text{tag}_i) \leftarrow (\mathbf{x}, \text{tag})$ 9 for $j \leq i$ do 10 if $a(\text{pp}, \text{tag}_i^i, \text{tag}_i^j) = 1$ then 11 $c_{i,j} \leftarrow g(\langle \mathbf{x}, \mathbf{x}_j \rangle)$ 12 else 13 $c_{i,j} \leftarrow \perp$ 14 $\mathcal{C}_{\text{ip}} \leftarrow \mathcal{C}_{\text{ip}} \cup \{(i, j), c_{i,j}\}$ 15 $(\text{ct}, st_S) \leftarrow S_3(\mathcal{C}_{\text{ip}}, st_S)$ 16 return ct

Figure 3: Games defining SIM-security for AC-IPBFRE.

Next, we give a simulation-based security definition for AC-IPBFRE in Figure 3. For an adversary $A = (A_1, A_2)$ and a simulator $S = (S_1, S_2, S_3)$, we define the SIM-advantage of A, S for all $\lambda \in \mathbb{N}$ as:

$$\text{Adv}_{\text{AC-IPBFRE},A,S}^{\text{sim}}(\lambda) = \Pr[\mathbf{G}_{\text{AC-IPBFRE},A}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{AC-IPBFRE},A,S}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

We say that an AC-IPBFRE scheme is *SIM-secure* if for every PT adversary A there exists a PT simulator S such that $\text{Adv}_{\text{AC-IPBFRE},A,S}^{\text{sim}}(\cdot)$ is negligible.

3.3 Inner Product Based Functional Encryption

A *private-key inner-product based functional encryption* (IPBFE) scheme is a tuple of algorithms $\text{IPBFE} = (\text{PPGen}, \text{Setup}, \text{KeyGen}, \text{Enc}, \text{Eval})$ that work as follows:

- $\text{PPGen}(1^\lambda)$: On input a unary encoding of the security parameter λ , the public parameters generation algorithm outputs the public parameters pp (that in particular includes the modulus $m \in \mathbb{Z}_+$). The public parameters are implicitly input to all other algorithms.
- $\text{Setup}(1^d)$: On input a unary encoding of the plaintext dimension 1^d , the setup algorithm outputs the master secret key msk .
- $\text{KeyGen}(\text{msk}, \mathbf{x})$: On input the master secret key msk and a vector $\mathbf{x} \in \mathbb{Z}_m^d$, the key generation algorithm outputs a function key $\text{sk}_{\mathbf{x}}$.
- $\text{Enc}(\text{msk}, \mathbf{y})$: On input the master secret key msk and a vector $\mathbf{y} \in \mathbb{Z}_m^d$, the encryption algorithm outputs a ciphertext $\text{ct}_{\mathbf{y}}$.
- $\text{Eval}(\text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{y}})$: On input a secret key $\text{sk}_{\mathbf{x}}$ and a ciphertext $\text{ct}_{\mathbf{y}}$, the decryption algorithm outputs a string z .

CORRECTNESS. We require *decryption correctness* meaning that

$$\Pr[\text{Eval}(\text{pp}, \text{KeyGen}(\text{msk}, \mathbf{x}), \text{Enc}(\text{msk}, \mathbf{y})) = g(\langle \mathbf{x}, \mathbf{y} \rangle)] = 1$$

for all $\lambda, d \in \mathbb{N}$, all $\text{pp} \in \text{Out}(\text{PPGen}(1^\lambda))$ and all $\text{msk} \in \text{Out}(\text{Setup}(\text{pp}, 1^d))$, where $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_m^d$ and probability is taken over the coins of the Enc and the KeyGen algorithm.

3.3.1 Security Definitions

Again, we consider both indistinguishability and simulation-based definitions.

INDISTINGUISHABILITY-BASED SECURITY. We give an indistinguishability-based security definition for Function-Hiding IPBFE (FH-IPBFE) in Figure 4. For an adversary $A = (A_1, A_2)$, define its IND-advantage for all $\lambda \in \mathbb{N}$ as:

$$\text{Adv}_{\text{FH-IPBFE}, A}^{\text{ind}}(\lambda) = \Pr[\mathbf{G}_{\text{FH-IPBFE}, A}^{\text{ind-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{FH-IPBFE}, A}^{\text{ind-0}}(\lambda) \Rightarrow 1].$$

We say that an FH-IPBFE scheme is *IND-secure* if $\text{Adv}_{\text{FH-IPBFE}, A}^{\text{ind}}(\cdot)$ is negligible for all PT adversaries A .

SIMULATION SECURITY. Next, we give a simulation-based security definition for FH-IPBFE in Figure 5. For an adversary $A = (A_1, A_2)$ and a simulator $S = (S_1, S_2, S_3, S_4)$, we define the SIM-advantage of A, S for all $\lambda \in \mathbb{N}$ as:

$$\text{Adv}_{\text{FH-IPBFE}, A, S}^{\text{sim}}(\lambda) = \Pr[\mathbf{G}_{\text{FH-IPBFE}, A}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{FH-IPBFE}, A, S}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

We say that an FH-IPBFE scheme is *SIM-secure* if for every PT adversary A there exists a PT simulator S such that $\text{Adv}_{\text{FH-IPBFE}, A, S}^{\text{sim}}(\cdot)$ is negligible.

4 Equivalence of FH-IPBFE and IPBFRE

We show in this Section the equivalence of FH-IPBFE and IPBFRE in detail. Therefore, we first show how to construct IPBFRE from FH-IPBFE. Then we use IPBFRE to construct non-zero inner product AC-IPBFRE (NZIP-AC-IPBFRE), an AC-IPBFRE where tags

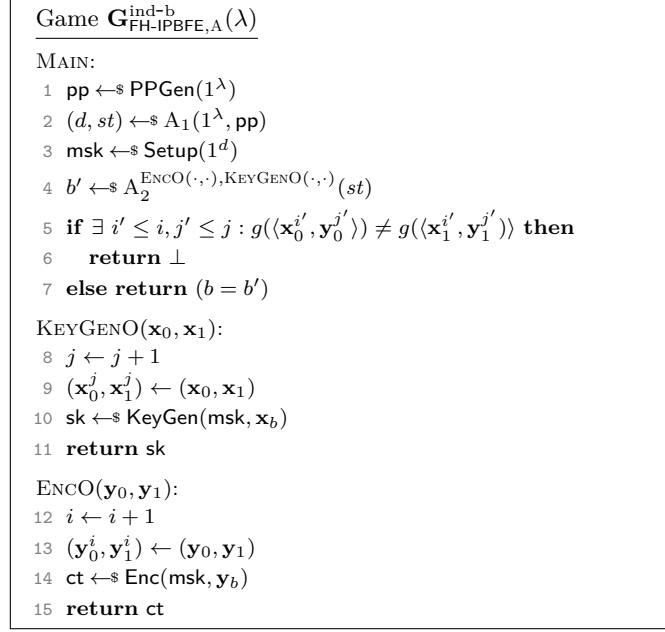


Figure 4: Games defining IND-security for FH-IPBFE.

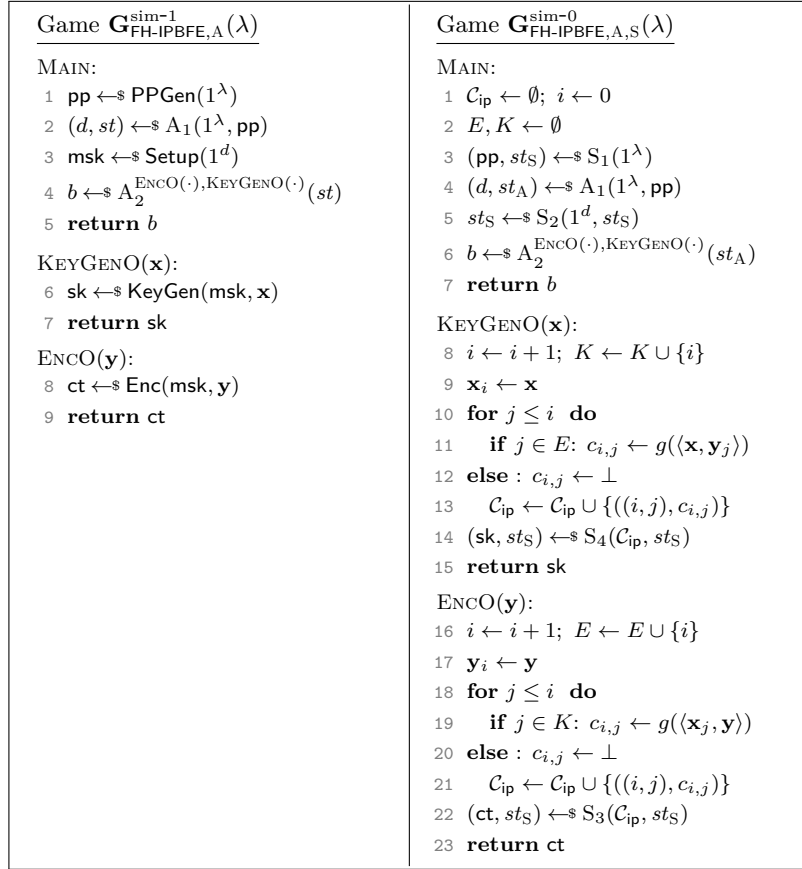


Figure 5: Games defining SIM-security for FH-IPBFE.

```

Enc'(msk, x):
1  ctx ←$ FH-IPBFE.Enc(msk, x)
2  skx ←$ FH-IPBFE.KeyGen(msk, x)
3  ct'x ← (ctx, skx)
4  return ct'x

Eval'(ct'x1, ct'x2):
5  (ctx1, skx1) ← ct'x1; (ctx2, skx2) ← ct'x2
6  y ←$ FH-IPBFE.Eval(skx1, ctx2)
7  return y

```

Figure 6: Transform from FH-IPBFE to IPBFE.

are vectors and decryption is possible if their inner product is non-zero. Finally, we use the NZIP-AC-IPBFRE to build AC-IPBFRE for small tag universe and arbitrary access policies can be generically constructed, which trivially implies FH-IPBFE. We also present an optimized version of this transformation specifically for FH-IPFE in Section A

4.1 From FH-IPBFE to IPBFE

First, we give a transformation that takes an FH-IPBFE and constructs an IPBFE using the FH-IPBFE scheme. This transformation uses ideas from [PR12]. This transformation does not use any specifics about inner-product functionalities. It can be generalized to turn FH-FE for any function class $\mathcal{F}$ into FRE for the function $g(x, y_f) := f(x)$, where y_f is a suitable encoding of f , if the function g is symmetric.

Let $\text{FH-IPBFE} = (\text{PPGen}, \text{Setup}, \text{Enc}, \text{KeyGen}, \text{Eval})$ be an FH-IPBFE scheme with message-space and key-space $\mathbb{Z}_p^d$ for a prime p and dimension $d \in \mathbb{N}$. Define the associated IPBFE scheme $\text{IPBFE}[\text{FH-IPBFE}] = (\text{PPGen}, \text{Setup}, \text{Enc}', \text{Eval}')$ with message-space $\mathbb{Z}_p^d$ where PPGen and Setup remain the same as FH-IPBFE and the rest is given in Figure 6.

Proposition 1. *If FH-IPBFE is decryption correct, then IPBFE[FH-IPBFE] is evaluation correct.*

Proof. Let $\text{pp} \in \text{Out}(\text{FH-IPBFE.PPGen}(1^\lambda))$, $\text{msk} \in \text{Out}(\text{FH-IPBFE.Setup}(\text{pp}, 1^d))$ for $\lambda, d \in \mathbb{N}$. Next, let $\text{sk}_{\mathbf{x}_1} \in \text{Out}(\text{FH-IPBFE.KeyGen}(\text{msk}, \mathbf{x}_1))$ and $\text{ct}_{\mathbf{x}_2} \in \text{Out}(\text{FH-IPBFE.Enc}(\text{msk}, \mathbf{x}_2))$. By decryption correctness of FH-IPBFE, we know that $\text{FH-IPBFE.Eval}(\text{pp}, \text{sk}_{\mathbf{x}_1}, \text{ct}_{\mathbf{x}_2}) = g(\langle \mathbf{x}_1, \mathbf{x}_2 \rangle)$. We have that

$$\text{IPBFE}[\text{FH-IPBFE}].\text{Eval}(\text{ct}'_{\mathbf{x}_1}, \text{ct}'_{\mathbf{x}_2}) = \text{FH-IPBFE.Eval}(\text{sk}_{\mathbf{x}_1}, \text{ct}_{\mathbf{x}_2}).$$

Thus, evaluation correctness of IPBFE[FH-IPBFE] is fulfilled. $\square$

Theorem 1 (IND-security). *If FH-IPBFE is IND-secure, then IPBFE[FH-IPBFE] is IND-secure. Concretely, for every adversary A against the IND-security of IPBFE[FH-IPBFE], there exists an adversary B against the IND-security of FH-IPBFE with roughly the same runtime as A such that for every $\lambda \in \mathbb{N}$*

$$\text{Adv}_{\text{FH-IPBFE}, \text{B}}^{\text{ind}}(\lambda) = \text{Adv}_{\text{IPBFE}[\text{FH-IPBFE}], \text{A}}^{\text{ind}}(\lambda).$$

Proof. Let A be an adversary against IND-security of IPBFE[FH-IPBFE]. We construct an adversary B against IND-security of FH-IPBFE in Figure 7.

It is easy to see that when A plays the $\mathbf{G}_{\text{IPBFE}[\text{FH-IPBFE}], \text{A}}^{\text{ind-b}}(\lambda)$ game, it perfectly simulates the game $\mathbf{G}_{\text{FH-IPBFE}, \text{B}}^{\text{ind-b}}(\lambda)$ for B and outputs the same bit as B. To show that these adversaries have the same advantage, what remains to show is that A causes the game to output $\perp$ exactly when B does.

Adversary $B = (B_1, B_2)$

$B_1(1^\lambda, \text{pp})$:

- 1 $(d, st) \leftarrow_s A_1(1^\lambda, \text{pp})$
- 2 **return** (d, st)

$B_2^{\text{ENCO}'(\cdot, \cdot), \text{KEYGENO}'(\cdot, \cdot)}(st)$:

- 3 $b' \leftarrow_s A_2^{\text{ENCO}'(\cdot, \cdot)}(st)$
- 4 **return** b'

$\text{ENCO}(\mathbf{x}_0, \mathbf{x}_1)$:

- 5 $\text{ct}_{\mathbf{x}} \leftarrow \text{ENCO}'(\mathbf{x}_0, \mathbf{x}_1)$
- 6 $\text{sk}_{\mathbf{x}} \leftarrow \text{KEYGENO}'(\mathbf{x}_0, \mathbf{x}_1)$
- 7 $\text{ct} \leftarrow (\text{ct}_{\mathbf{x}}, \text{sk}_{\mathbf{x}})$
- 8 **return** ct

Figure 7: Adversary against IND-security of IPBFRE[FH-IPBFE].

Assume A causes the game to output $\perp$. Then A made ENC queries with $(\mathbf{x}_0^j, \mathbf{x}_1^j)$ and $(\mathbf{x}_0^k, \mathbf{x}_1^k)$ such that $g(\langle \mathbf{x}_0^j, \mathbf{x}_0^k \rangle) \neq g(\langle \mathbf{x}_1^j, \mathbf{x}_1^k \rangle)$. In order to simulate these oracle queries, B made in particular an ENC' query for $(\mathbf{x}_0^j, \mathbf{x}_1^j)$ and an KEYGEN' query for $(\mathbf{x}_0^k, \mathbf{x}_1^k)$. Thus also B causes its game to abort.

Next, assume B causes the game to abort. Then B made an ENC' query for $(\mathbf{x}_0^i, \mathbf{x}_1^i)$ and an KEYGEN' query for $(\mathbf{y}_0^j, \mathbf{y}_1^j)$ such that $g(\langle \mathbf{x}_0^i, \mathbf{y}_0^j \rangle) \neq g(\langle \mathbf{x}_1^i, \mathbf{y}_1^j \rangle)$. These queries could only be cause by A making ENC queries for $(\mathbf{x}_0^i, \mathbf{x}_1^i)$ and $(\mathbf{y}_0^j, \mathbf{y}_1^j)$. Thus, A also made its game abort.

With this, we see

$$\Pr[\mathbf{G}_{\text{FH-IPBFE}, B}^{\text{ind-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{IPBFRE[FH-IPBFE]}, A}^{\text{ind-1}}(\lambda) \Rightarrow 1]$$

and

$$\Pr[\mathbf{G}_{\text{FH-IPBFE}, B}^{\text{ind-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{IPBFRE[FH-IPBFE]}, A}^{\text{ind-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

Theorem 2 (SIM-security). *If FH-IPBFE is SIM-secure, then IPBFRE[FH-IPBFE] is SIM-secure. Concretely, for every adversary A against IPBFRE[FH-IPBFE], there exists an adversary B against FH-IPBFE such that for every simulator S_B there exists a simulator S_A where for all $\lambda \in \mathbb{N}$*

$$\text{Adv}_{\text{IPBFRE[FH-IPBFE]}, A, S_A}^{\text{sim}}(\lambda) = \text{Adv}_{\text{FH-IPBFE}, B, S_B}^{\text{sim}}(\lambda).$$

Proof. Let A be an adversary against IPBFRE[FH-IPBFE]. We first construct an adversary B against SIM-security of FH-IPBFE in Figure 8.

Next, by SIM-security of FH-IPBFE, for an adversary B there exists a simulator $S_B = (S_{B,1}, S_{B,2}, S_{B,3}, S_{B,4})$. Then, consider the simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$ for IPBFRE[FH-IPBFE] for an adversary A also given in figure 8.

The simulator S_A gets the set $\mathcal{C}'_{\text{ip}}$ that contains all pairwise inner products of IPBFRE ciphertexts with each other using a global index to keep track of the queries. The simulator turns this into a set $\mathcal{C}_{\text{ip}}$ that is indexed by query-type (ENC or KEYGEN) specific indices, where the first index corresponds to the ciphertext and the second one corresponds to the key. When adding new tuples to $\mathcal{C}_{\text{ip}}$, the simulator must keep track of the indices and only add new values right before running $S_{B,3}$ or $S_{B,4}$ with an updated $\mathcal{C}_{\text{ip}}$ as given in simulator lines 9-13.

With this, we see

$$\Pr[\mathbf{G}_{\text{IPBFRE[FH-IPBFE]}, A}^{\text{sim-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{FH-IPBFE}, B}^{\text{sim-1}}(\lambda) \Rightarrow 1]$$

Adversary $B = (B_1, B_2)$	Simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$
$B_1(1^\lambda, \text{pp})$: 1 $(d, st) \leftarrow \$ A_1(1^\lambda, \text{pp})$ 2 return (d, st) $B_2^{\text{ENCO}'(\cdot), \text{KEYGENO}'(\cdot)}(st)$: 3 $b \leftarrow \$ A_2^{\text{ENCO}'(\cdot)}(st)$ 4 return b $\text{ENCO}(\mathbf{x})$: 5 $\text{ct}_{\mathbf{x}} \leftarrow \text{ENCO}'(\mathbf{x})$ 6 $\text{sk}_{\mathbf{x}} \leftarrow \text{KEYGENO}'(\mathbf{x})$ 7 $\text{ct} \leftarrow (\text{ct}_{\mathbf{x}}, \text{sk}_{\mathbf{x}})$ 8 return ct	$S_{A,1}(1^\lambda)$: 1 $(\text{pp}, st_B) \leftarrow \$ S_{B,1}(1^\lambda)$ 2 $st \leftarrow st_B$ 3 return (pp, st) $S_{A,2}(1^d, st)$: 4 $i \leftarrow 0$; $st_B \leftarrow st$ 5 $st_B \leftarrow \$ S_{B,2}(1^d, st_B)$ 6 $st \leftarrow (st_B, k)$ 7 return st $S_{A,3}(\mathcal{C}'_{\text{ip}}, st)$: 8 $(st_B, k) \leftarrow st$ 9 $i \leftarrow i + 1$ 10 $\mathcal{C}_{\text{ip}} \leftarrow \{(j, k), c_{j,k} \mid ((j, k), c_{j,k}) \in \mathcal{C}'_{\text{ip}} \wedge (j \leq i) \wedge (k < i)\} \cup \{(k, j), c_{j,k} \mid ((j, k), c_{j,k}) \in \mathcal{C}'_{\text{ip}} \wedge (j \leq i) \wedge (j < i)\}$ 11 $(\text{ct}_i, st_B) \leftarrow S_{B,3}(\mathcal{C}_{\text{ip}}, st_B)$ 12 $\mathcal{C}_{\text{ip}} \leftarrow \mathcal{C}_{\text{ip}} \cup \{(j, i), c_{j,i} \mid ((j, i), c_{j,i}) \in \mathcal{C}'_{\text{ip}} \wedge (i > j)\} \cup \{(i, j), c_{j,i} \mid ((j, i), c_{j,i}) \in \mathcal{C}'_{\text{ip}} \wedge (i > j)\}$ 13 $(\text{sk}_i, st_B) \leftarrow S_{B,4}(\mathcal{C}_{\text{ip}}, st_B)$ 14 $\text{ct} \leftarrow (\text{ct}_i, \text{sk}_i)$ 15 $st \leftarrow (st_B, k)$ 16 return (ct, st)

Figure 8: Adversary against SIM-security of FH-IPBFE and simulator for SIM security of IPBFRE[FH-IPBFE].

and

$$\Pr[\mathbf{G}_{\text{IPBFRE[FH-IPBFE]}, A, S_A}^{\text{sim-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{FH-IPBFE}, B, S_B}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

4.2 From IPBFRE to NZIP-AC-IPBFRE

Here, we present a transformation that converts any FRE for an inner-product based functionality $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$ into an access-controlled FRE for the same functionality where tags are vectors (possibly of a different dimension than the message vectors, but over the same field as the message vectors) and evaluation of two ciphertexts is possible if the inner-product of their tags is non-zero. A similar transformation has been considered in [DDM⁺23]. Formally, the message space is $\mathcal{M} = \mathbb{Z}_m^d$. The tag space $\mathcal{T}$ can be set to any subset of $\mathbb{Z}_p^\tau$ that satisfies⁴

$$\forall \mathbf{u}, \mathbf{v} \in \mathcal{T} : \langle \mathbf{u}, \mathbf{v} \rangle \in \{0, 1\}$$

The auxiliary information for the access policy is $\tau \in \mathbb{N}$ and a description of $\mathcal{T}$ with access policy

$$a_{\text{nzip}}(\text{pp}, \tau, \mathbf{u}, \mathbf{v}) := \begin{cases} 1 & \text{if } \mathbf{u}^\top \mathbf{v} \neq 0 \text{ and } \mathbf{u} \in \mathcal{T} \text{ and } \mathbf{v} \in \mathcal{T} \\ 0 & \text{otherwise.} \end{cases}$$

⁴For the inner-product, inner-product in the exponent and orthogonality functionality, this requirement can be relaxed to $\forall \mathbf{u}, \mathbf{v} \in \mathcal{T} : \langle \mathbf{u}, \mathbf{v} \rangle \in \{0\} \cup \mathbb{Z}_m^*$. For the inner-product and inner-product in the exponent functionality, this requires keeping track of the tags and dividing the result (in the exponent) by the inner product of the tags in **Eval**. Note that for all these functionalities g is multiplicatively homomorphic. Such a scheme however does not hide the tags any more. The tags can be encrypted with an IPFRE scheme to only leak the inner products between the tags. This variant has been considered in [DDM⁺23].

Let $\text{IPBFRE} = (\text{PPGen}, \text{Setup}, \text{Enc}, \text{Eval})$ be an FRE scheme with message-space $\mathbb{Z}_p^d$ for an inner-product based functionality $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$. Define the associated NZIP-AC-IPBFRE scheme $\text{NZIP-AC-IPBFRE}[\text{IPBFRE}] = (\text{PPGen}, \text{Setup}', \text{Enc}', \text{Eval}')$ with message-space $\mathbb{Z}_p^d$, tag-space $\mathbb{Z}_p^\tau$, and access policy a_{nzip} as in Figure 9.

Setup'(1^λ, 1^d, 1^τ):

1 **return** $\text{msk} \leftarrow \text{Setup}(1^\lambda, 1^{d \cdot \tau})$

Enc'(msk, $\mathbf{u} \in \mathbb{Z}_p^\tau, \mathbf{x} \in \mathbb{Z}_p^d$):

2 **return** $\text{Enc}(\text{msk}, \mathbf{u} \otimes \mathbf{x})$

Eval'(ct₁, ct₂):

3 **return** $\text{Eval}(\text{ct}_1, \text{ct}_2)$

Figure 9: Transform from FRE for an inner-product based functionality to NZIP-AC-FRE for the same functionality.

Proposition 2. *If IPBFRE is evaluation correct, then NZIP-AC-IPBFRE[IPBFRE] is evaluation correct.*

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_p^\tau$ with $\mathbf{u}^\top \mathbf{v} = 1$, $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{Z}_p^d$, $\text{ct}_1 \in \text{Out}(\text{Enc}(\text{msk}, \mathbf{u} \otimes \mathbf{x}_1))$, and $\text{ct}_2 \in \text{Out}(\text{Enc}(\text{msk}, \mathbf{v} \otimes \mathbf{x}_2))$. Then, by evaluation correctness of IPBFRE, $\text{Eval}(\text{ct}_1, \text{ct}_2)$ outputs $g((\mathbf{u} \otimes \mathbf{x}_1)^\top (\mathbf{v} \otimes \mathbf{x}_2)) = g(\mathbf{u}^\top \mathbf{v} \cdot \mathbf{x}_1^\top \mathbf{x}_2) = g(\mathbf{x}_1^\top \mathbf{x}_2)$. $\square$

Theorem 3 (IND-security). *If IPBFRE is IND-secure, then NZIP-AC-IPBFRE[IPBFRE] is IND-secure. Concretely, for every adversary A against IND-security of NZIP-AC-IPBFRE[IPBFRE], there exists an adversary B against IND-security of IPBFRE with roughly the same runtime such that*

$$\text{Adv}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], A}^{\text{ind}}(\lambda) \leq \text{Adv}_{\text{IPBFRE}, B}^{\text{ind}}(\lambda).$$

Proof. Let A be an adversary against IND-security of NZIP-AC-IPBFRE[IPBFRE]. We construct an adversary B against IND-security of IPBFRE in Figure 10.

It is easy to see that when A plays the $\mathbf{G}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], A}^{\text{ind-b}}(\lambda)$ game, it perfectly simulates the game $\mathbf{G}_{\text{IPBFRE}, B}^{\text{ind-b}}(\lambda)$ for B and outputs the same bit as B. This shows

$$\begin{aligned} & \Pr[\mathbf{G}_{\text{IPBFRE}, B}^{\text{ind-b}}(\lambda) \Rightarrow 1 \mid \mathbf{G}_{\text{IPBFRE}, B}^{\text{ind-b}}(\lambda) \not\Rightarrow \perp] \\ &= \Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], A}^{\text{ind-b}}(\lambda) \Rightarrow 1 \mid \mathbf{G}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], A}^{\text{ind-b}}(\lambda) \not\Rightarrow \perp]. \quad (1) \end{aligned}$$

Adversary B = (B₁, B₂)

B₁(1^λ, pp):

1 $(d, \tau, st) \leftarrow A_1(1^\lambda, \text{pp})$

2 **return** $(d \cdot \tau, st)$

B₂^{ENCO'($\cdot, \cdot$)}(st):

3 $b' \leftarrow A_2^{\text{ENCO}(\cdot, \cdot, \cdot)}(st)$

4 **return** b'

ENCO($\mathbf{x}_0, \mathbf{x}_1, \mathbf{u}_0, \mathbf{u}_1$):

5 $\text{ct} \leftarrow \text{ENCO}'(\mathbf{u}_0 \otimes \mathbf{x}_0, \mathbf{u}_1 \otimes \mathbf{x}_1)$

6 **return** $(\mathbf{u}, \text{ct})$

Figure 10: Adversary against IND-security of IPBFRE.

Adversary $B = (B_1, B_2)$	Simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$
$B_1(1^\lambda, \text{pp}):$ 1 $(d, \tau, st) \leftarrow \$ A_1(1^\lambda, \text{pp})$ 2 return $(d \cdot \tau, st)$	$S_{A,1}(1^\lambda):$ 1 $(\text{pp}, st) \leftarrow \$ S_{B,1}(1^\lambda)$ 2 return (pp, st)
$B_2^{\text{ENCO}'(\cdot)}(st):$ 3 $b \leftarrow \$ A_2^{\text{ENCO}'(\cdot)}(st)$ 4 return b	$S_{A,2}(1^d, 1^\tau, st):$ 3 $st \leftarrow \$ S_{B,2}(1^{d \cdot \tau}, st)$ 4 return st
$\text{ENCO}(\mathbf{x}, \mathbf{u}):$ 5 $\text{ct} \leftarrow \text{ENCO}'(\mathbf{x} \otimes \mathbf{u})$ 6 return ct	$S_{A,3}(\mathcal{C}_{\text{ip}}, st):$ 5 $\mathcal{C}'_{\text{ip}} \leftarrow \mathcal{C}_{\text{ip}} \cup \{((i, j), g(0)) \mid j \leq i \wedge \nexists c_{i,j} : ((i, j), c_{i,j}) \in \mathcal{C}_{\text{ip}}\}$ 6 $(\text{ct}, st) \leftarrow \$ S_{B,3}(\mathcal{C}'_{\text{ip}}, st)$ 7 return (ct, st)

Figure 11: Adversary against SIM-security of IPBFRE and simulator for SIM security of NZIP-AC-IPBFRE[IPBFRE].

To show that the advantage of B is at least as high as the advantage of A , what remains to show is that B causes the game to output $\perp$ only when A does so.

Therefore, let $\mathbf{z}_0^j, \mathbf{z}_1^j, \mathbf{u}_0^j, \mathbf{u}_1^j$ be the input of the j -th ENC oracle query of B_2 . The IND game for IPBFRE returns $\perp$ if there are indices j, k with $g(\langle \mathbf{z}_0^j, \mathbf{z}_0^k \rangle) \neq g(\langle \mathbf{z}_1^j, \mathbf{z}_1^k \rangle)$. Using that $\mathbf{z}_b^j = \mathbf{u}_b^j \otimes \mathbf{x}_b^j$, we get

$$\begin{aligned}
g(\langle \mathbf{u}_0^j, \mathbf{u}_0^k \rangle \langle \mathbf{x}_0^j, \mathbf{x}_0^k \rangle) &= g(\langle \mathbf{u}_0^j \otimes \mathbf{x}_0^j, \mathbf{u}_0^k \otimes \mathbf{x}_0^k \rangle) = g(\langle \mathbf{z}_0^j, \mathbf{z}_0^k \rangle) \\
&\neq g(\langle \mathbf{z}_1^j, \mathbf{z}_1^k \rangle) = g(\langle \mathbf{u}_1^j \otimes \mathbf{x}_1^j, \mathbf{u}_1^k \otimes \mathbf{x}_1^k \rangle) = g(\langle \mathbf{u}_1^j, \mathbf{u}_1^k \rangle \langle \mathbf{x}_1^j, \mathbf{x}_1^k \rangle) \quad (2)
\end{aligned}$$

which implies $\langle \mathbf{u}_0^j, \mathbf{u}_0^k \rangle \neq 0$ or $\langle \mathbf{u}_1^j, \mathbf{u}_1^k \rangle \neq 0$ and thus $\langle \mathbf{u}_0^j, \mathbf{u}_0^k \rangle \neq \langle \mathbf{u}_1^j, \mathbf{u}_1^k \rangle$ or $\langle \mathbf{u}_0^j, \mathbf{u}_0^k \rangle = \langle \mathbf{u}_1^j, \mathbf{u}_1^k \rangle = 1$. In the latter case, we must have $g(\langle \mathbf{x}_0^j, \mathbf{x}_0^k \rangle) \neq g(\langle \mathbf{x}_1^j, \mathbf{x}_1^k \rangle)$ to satisfy inequality (2). This shows, that also the IND game for NZIP-AC-IPBFRE[IPBFRE] outputs $\perp$ here.

With this, we see

$$\Pr[\mathbf{G}_{\text{IPBFRE}, B}^{\text{ind-1}}(\lambda) \Rightarrow \perp] \leq \Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE[IPBFRE]}, A}^{\text{ind-1}}(\lambda) \Rightarrow \perp]. \quad (3)$$

Combining equations (1) and (3) yields the result. $\square$

Theorem 4 (SIM-security). *If IPBFRE is SIM-secure, then NZIP-AC-IPBFRE[IPBFRE] is SIM-secure. Concretely, for every adversary A against SIM-security of NZIP-AC-IPBFRE[IPBFRE], there exists an adversary B against SIM-security of IPBFRE with roughly the same runtime as A , such that for every simulator S_B there exists a simulator S_A with*

$$\text{Adv}_{\text{NZIP-AC-IPBFRE[IPBFRE]}, A, S_A}^{\text{sim}}(\lambda) \leq \text{Adv}_{\text{IPBFRE}, B, S_B}^{\text{sim}}(\lambda).$$

Proof. Let $A = (A_1, A_2)$ be an adversary against the SIM-security of NZIP-AC-IPBFRE[IPBFRE]. We first construct an adversary $B = (B_1, B_2)$ against the SIM-security of IPBFRE in Figure 11. Next, by SIM-security of IPBFRE, for an adversary B there exists a simulator $S_B = (S_{B,1}, S_{B,2}, S_{B,3})$. Then, consider the simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$ for NZIP-AC-IPBFRE[IPBFRE] for an adversary A also given in Figure 11.

The simulator stage $S_{A,3}$ gets a set $\mathcal{C}_{\text{ip}}$ of all function evaluations where the corresponding tag vectors satisfy the access policy and adds them to the new set $\mathcal{C}'_{\text{ip}}$. Additionally, for every pair of vectors that has no inner-product value stored in $\mathcal{C}'_{\text{ip}}$ (i.e. pairs of vectors that did not satisfy the access policy), it stores them together with $g(0)$ as inner product

value in $\mathcal{C}'_{\text{ip}}$.⁵ Since for any vectors $\mathbf{x}, \mathbf{y}$ encrypted under tags $\mathbf{u}, \mathbf{v}$ that do not satisfy the access policy

$$g(\langle \mathbf{x} \otimes \mathbf{u}, \mathbf{y} \otimes \mathbf{v} \rangle) = g(\langle \mathbf{x}, \mathbf{y} \rangle \langle \mathbf{u}, \mathbf{v} \rangle) = g(\langle \mathbf{x}, \mathbf{y} \rangle \cdot 0) = g(0)$$

storing the value $g(0)$ in $\mathcal{C}'_{\text{ip}}$ provides the correct function evaluation for the underlying IPBFRE.

With this, we see

$$\Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], \text{A}}^{\text{sim-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{IPBFRE}, \text{B}}^{\text{sim-1}}(\lambda) \Rightarrow 1]$$

and

$$\Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}[\text{IPBFRE}], \text{A}, \text{S}_\text{A}}^{\text{sim-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{IPBFRE}, \text{B}, \text{S}_\text{B}}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

4.3 From NZIP-AC-IPBFRE to small universe AC-IPBFRE

We show here how an NZIP-AC-IPBFRE can be used to realize AC-IPFRE for general access policies in the small tag-universe case. The construction relies on the following effective version of Lagrange's four square theorem.

Theorem 5. *There exists a PT algorithm `LagrangeSoS` that inputs an arbitrary natural number $x \in \mathbb{N}$ and outputs (a_1, a_2, a_3, a_4) such that $a_1^2 + a_2^2 + a_3^2 + a_4^2 = x$.*

Proof. The existence of such a solution follows from Lagrange's four square theorem: Every positive integer is the sum of four squares. Several proofs of this classical theorem can be found for example in [HWHBS08, §20.5].

Now, Rabin and Shallit presented algorithms that can find such $a_1, \dots, a_4$ for m with expected time $\mathcal{O}((\log m)^2)$ if the extended Riemann hypothesis is true or $\mathcal{O}((\log m)^2 \log \log m)$ without relying on unproven conjectures [RS86]. Pollack and Treviño improved the expected runtime to $\mathcal{O}((\log m)^2 (\log \log m)^{-1})$ [PT18].⁶ $\square$

AC-IPFRE FOR GENERAL ACCESS POLICIES. We demonstrate how a NZIP-AC-IPFRE can be used to realize any access policy with an a priori fixed, polynomial number of different tags. Let $G = (V, E)$ be an undirected graph (with loops) expressing this access policy. That is, V is the set of tags and $\{u, v\} \in E$ iff it is possible to compute the inner product between ciphertexts with tag u and ciphertexts with tag v with `Eval`. In particular, the node u has a loop ($\{u\} \in E$) iff it is possible to compute the inner product between two ciphertexts both associated with tag u .

We now construct vectors to realize this access policy with a NZIP-AC-IPFRE. Therefore, let $L := \{e \in E \mid |e| = 1\}$ be the set of loops and fix an ordering $\{v_1, \dots, v_n\} = V$ on the set of nodes and $\{e_1, \dots, e_m\} = E \setminus L$ on the set of non-loop edges. The construction uses dimension $\tau := m + 4n$. We define the tag vector $\mathbf{u}_i$ as follows:

⁵For the relaxed version where the inner product of tags can be in $\{0\} \cup \mathbb{Z}_m^*$ and g is multiplicatively homomorphic the simulator has to compute the set $\mathcal{C}'_{\text{ip}}$ as

$$\begin{aligned} \mathcal{C}'_{\text{ip}} \leftarrow \{((i, j), c_{i,j} \cdot g((\mathbf{u}_i)^\top \mathbf{u}_j)) \mid ((i, j), c_{i,j}) \in \mathcal{C}_{\text{ip}} \wedge (i, \mathbf{u}_i) \in \mathcal{T} \wedge (j, \mathbf{u}_j) \in \mathcal{T}\} \\ \cup \{((i, j), g(0)) \mid \nexists c_{i,j} : ((i, j), c_{i,j}) \in \mathcal{C}_{\text{ip}}\}, \end{aligned}$$

where $\mathcal{T}$ contains all tuples of indices and tags, which is an additional input to the simulator in the non-tag hiding variant of the SIM-security game.

⁶Technically, we require worst-case polynomial runtime, which can be obtained by relaxing to a negligible correctness error.

$$\mathbf{u}_i := \left(\begin{array}{c} b_{i,1} \\ \vdots \\ b_{i,m} \\ 0 \\ \vdots \\ 0 \\ a_{i,1} \\ a_{i,2} \\ a_{i,3} \\ a_{i,4} \\ 0 \\ \vdots \\ 0 \end{array} \right) \left\{ \begin{array}{l} m \\ 4(i-1) \\ 4(n-i) \end{array} \right\} \quad \text{where} \quad \begin{array}{l} b_{i,k} := \begin{cases} 1 & \text{if } v_i \in e_k \\ 0 & \text{otherwise} \end{cases} \\ \mu_i := \sum_{k=1}^m b_{i,k} \\ b_i := \begin{cases} 1 & \text{if } \{v_i\} \in E \\ 0 & \text{otherwise} \end{cases} \\ (a_{i,j})_{j \in [4]} \leftarrow^s \text{LagrangeSoS}(mq - \mu_i + b_i) \end{array}$$

The summand mq in the argument of **LagrangeSoS** avoids a negative argument and can be replaced by any multiple of q that is sufficiently large.

The following theorem formalizes that these vectors, when used as tag vectors for an NZIP-AC-IPFRE, realize the access structured represented by the graph G .

Theorem 6. *Let $G = (V, E)$ be an undirected graph (with loops), $V =: \{v_1, \dots, v_n\}$, and $(\mathbf{u}_i)_{i \in [n]}$ constructed as described above. Then for all $i, j \in [n]$*

$$\begin{aligned} \{v_i, v_j\} \in E &\Rightarrow \langle \mathbf{u}_i, \mathbf{u}_j \rangle = 1 \\ \{v_i, v_j\} \notin E &\Rightarrow \langle \mathbf{u}_i, \mathbf{u}_j \rangle = 0 \end{aligned}$$

Proof. We first focus on the non-loop edges.

Case 1: Assume $\{v_i, v_j\} \in E$ and $i \neq j$. Then there exists an index $k \in [m]$ such that $e_k = \{v_i, v_j\}$ and thus $b_{i,k} = b_{j,k} = 1$. For all $k' \in [m]$ with $k \neq k'$ we have $b_{i,k'} = 0$ or $b_{j,k'} = 0$, because $e_{k'} \neq \{v_i, v_j\}$. This yields

$$\langle \mathbf{u}_i, \mathbf{u}_j \rangle = \left\langle \begin{pmatrix} b_{i,1} \\ \vdots \\ b_{i,m} \end{pmatrix}, \begin{pmatrix} b_{j,1} \\ \vdots \\ b_{j,m} \end{pmatrix} \right\rangle = 1.$$

Case 2: Assume $\{v_i, v_j\} \notin E$ and $i \neq j$. Then for all $k \in [m]$ we have $b_{i,k} = 0$ or $b_{j,k} = 0$, because $e_k \neq \{v_i, v_j\}$. This yields

$$\langle \mathbf{u}_i, \mathbf{u}_j \rangle = \left\langle \begin{pmatrix} b_{i,1} \\ \vdots \\ b_{i,m} \end{pmatrix}, \begin{pmatrix} b_{j,1} \\ \vdots \\ b_{j,m} \end{pmatrix} \right\rangle = 0.$$

Next, we consider the loops. For this, note that

$$\langle \mathbf{u}_i, \mathbf{u}_i \rangle = \left\langle \begin{pmatrix} b_{i,1} \\ \vdots \\ b_{i,m} \\ 0 \\ \vdots \\ 0 \\ a_{i,1} \\ a_{i,2} \\ a_{i,3} \\ a_{i,4} \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \begin{pmatrix} b_{i,1} \\ \vdots \\ b_{i,m} \\ 0 \\ \vdots \\ 0 \\ a_{i,1} \\ a_{i,2} \\ a_{i,3} \\ a_{i,4} \\ 0 \\ \vdots \\ 0 \end{pmatrix} \right\rangle = \mu_i + a_1^2 + a_2^2 + a_3^2 + a_4^2 = b_i .$$

Case 3: Assume $\{v_i\} \in E$. Then by definition $b_i = 1$ and thus $\langle \mathbf{u}_i, \mathbf{u}_i \rangle = 1$.

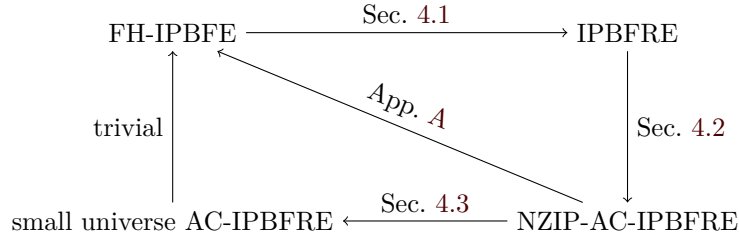
Case 4: Assume $\{v_i\} \notin E$. Then by definition $b_i = 0$ and thus $\langle \mathbf{u}_i, \mathbf{u}_i \rangle = 0$. $\square$

In Section D we present an alternative construction of these vectors that uses $\tau := 5n$ instead of $\tau := m + 4n$.

4.4 Equivalence of FH-IPBFE and IPBFRE

Section 4.1 shows how an FH-IPBFE can be generically transformed into an IPBFRE. Our transformation simply includes both a ciphertext and function key of the FH-IPBFE in an IPBFRE ciphertext. Our construction of general AC-IPBFRE from NZIP-AC-IPBFRE in Section 4.3 can be directly used as FH-IPBFE by using a universe with two tags. FH-IPBFE ciphertexts are then AC-IPBFRE ciphertexts with one tag and FH-IPBFE functional keys are AC-IPBFRE ciphertexts with the other tags. We given an optimized variant of this idea in Appendix A.

Thus, we have shown the following implications that give us the equivalence between FH-IPBFRE, IPBFRE, NZIP-AC-IPBFRE and AC-IBFRE with small tag universe:



5 The TinyFHIPFE Scheme

Here we give a construction of FH-IPFE that borrows some techniques of Wee [Wee20] in the context of (non-function-hiding) quadratic FE and is more efficient than prior constructions.

BILINEAR GROUPS. We consider asymmetric bilinear groups. An asymmetric *bilinear-group generator* $\mathcal{G}$ takes as input 1^λ and outputs a tuple $(p, g_1, g_2, \mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_T, \mathbf{e})$. Here, $\mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_T$

are groups of order p , where p is a prime of λ bits, and g_1, g_2 generate G_1, G_2 , respectively. The mapping $e: G_1 \times G_2 \rightarrow G_T$ is an efficiently computable, non-degenerate bilinear map. In particular, it satisfies $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$ for all $a, b \in \mathbb{Z}_p$, and $e(g_1, g_2)$ generates G_T .

For a group G of order p generated by g , for $x \in \mathbb{Z}_p$ we write g^x as $[x]_g$. We omit the subscript g when it is evident from the context. For brevity, we also write $\prod_{i \in [d]} e(\mathbf{a}[i], \mathbf{b}[i])$ as $e(\mathbf{a}, \mathbf{b})$, where $\mathbf{a} \in G_1^d$ and $\mathbf{b} \in G_2^d$.

BOUNDED IPFE. To capture ours and prior FH-IPFE constructions where inner-product is recovered in the exponent of the target group, we introduce the notion of *bounded* IPFE. In bounded IPFE, the **Eval** algorithm takes an additional input 1^B for a bound $B \in \mathbb{N}$ and uses an algorithm **Rec** that on input $(1^B, G_T, g_T, [z]_{g_T})$ outputs $z \in \mathbb{Z}_p$ such that $z = \text{DLog}_{G_T, g_T}([z]_{g_T})$ if $z \leq B$ or $\perp$. Evaluation correctness requires that

$$\Pr[\text{Eval}(1^B, \text{KeyGen}(\text{msk}, \mathbf{x}), \text{Enc}(\text{msk}, \mathbf{y})) = \langle \mathbf{x}, \mathbf{y} \rangle] = 1$$

for all $B, \lambda, d \in \mathbb{N}$, all $\text{pp} \in \text{Out}(\text{PPGen}(1^\lambda))$ and all $\text{msk} \in \text{Out}(\text{Setup}(\text{pp}, 1^d))$, where $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_m^d$ is such that $\langle \mathbf{x}, \mathbf{y} \rangle \leq B$. The probability is taken over the coins of the **Enc** algorithm.

5.1 Our Construction

Let $\mathcal{G}_{\text{asym}}$ be an asymmetric bilinear-group generator and $\text{SL}_4(\mathbb{Z}_p)$ be the special linear group of degree 4 over $\mathbb{Z}_p$. We define our bounded FH-IPFE scheme named **TinyFHIPFE** in Figure 12. (Recall pp is implicitly input to the remaining algorithms.) Note that, like previous FH-IPFE schemes, it recovers inner-product in the exponent. We find bounded inner-product is sufficient for the datasets considered in Section 7. **TinyFHIPFE** uses a short master secret key linear in the input vector dimension and a uniform random vector is sampled during each encryption. In order to reduce the size of the ciphertext, aside from the linear components encoding the input vector, other components of the ciphertext are carefully chosen to make the cross terms constant size.

PPGen(1^λ): 1 $(p, g_1, g_2, g_T, G_1, G_2, G_T, e) \leftarrow \mathcal{G}_{\text{asym}}(1^\lambda)$ 2 $\text{pp} \leftarrow (p, g_1, g_2, g_T, G_1, G_2, G_T, e)$ 3 return pp Setup($1^\lambda, 1^d$): 4 $\mathbf{B} \leftarrow \text{SL}_4(\mathbb{Z}_p)$ 5 $\mathbf{B}^* \leftarrow (\mathbf{B}^{-1})^T$ 6 $\mathbf{A} \leftarrow \mathbb{Z}_p^{2 \times d}$ 7 $\text{msk} \leftarrow (\mathbf{A}, \mathbf{B}, \mathbf{B}^*)$ 8 return msk KeyGen($\text{msk}, \mathbf{x}$): 9 $\mathbf{s} \leftarrow \mathbb{Z}_p^2$ 10 $(\mathbf{A}, \mathbf{B}, \mathbf{B}^*) \leftarrow \text{msk}$ 11 $\text{sk}_x \leftarrow [\mathbf{A}^T \mathbf{s} + \mathbf{x}]_{g_1}$ 12 $\text{sk}_k \leftarrow [\mathbf{B} \cdot (\mathbf{s} \parallel \mathbf{A} \mathbf{x} + \mathbf{A} \mathbf{A}^T \mathbf{s})]_{g_1}$ 13 $\text{sk} \leftarrow (\text{sk}_x, \text{sk}_k)$ 14 return sk	Enc($\text{msk}, \mathbf{y}$): 15 $\mathbf{s} \leftarrow \mathbb{Z}_p^2$ 16 $(\mathbf{A}, \mathbf{B}, \mathbf{B}^*) \leftarrow \text{msk}$ 17 $\text{ct}_y \leftarrow [\mathbf{A}^T \mathbf{s} + \mathbf{y}]_{g_2}$ 18 $\text{ct}_c \leftarrow [\mathbf{B}^* \cdot (\mathbf{A} \mathbf{y} \parallel \mathbf{s})]_{g_2}$ 19 $\text{ct} \leftarrow (\text{ct}_y, \text{ct}_c)$ 20 return ct Eval($1^B, \text{sk}, \text{ct}$): 21 $(\text{sk}_x, \text{sk}_k) \leftarrow \text{sk}$ 22 $(\text{ct}_y, \text{ct}_c) \leftarrow \text{ct}$ 23 $[z]_{g_T} \leftarrow e(\text{sk}_x, \text{ct}_y) / e(\text{sk}_k, \text{ct}_c)$ 24 $z \leftarrow \text{Rec}(1^B, G_T, g_T, [z]_{g_T})$ 25 return z
--	--

Figure 12: The **TinyFHIPFE** Construction.

Proposition 3. *TinyFHIPFE is evaluation correct.*

Proof. Let $\lambda, d \in \mathbb{N}$. Let $\text{pp} \leftarrow \text{TinyFHIPFE.PPGen}(1^\lambda)$, $\text{msk} \leftarrow \text{TinyFHIPFE.Setup}(1^\lambda, 1^d)$ $\text{ct} \leftarrow \text{TinyFHIPFE.Enc}(\text{msk}, \mathbf{y})$ and $\text{sk} \leftarrow \text{TinyFHIPFE.KeyGen}(\text{msk}, \mathbf{x})$ for $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_p^d$, we have $(\text{ct}_{\mathbf{y}}, \text{ct}_{\mathbf{c}}) \leftarrow \text{ct}$ and $(\text{sk}_{\mathbf{x}}, \text{sk}_{\mathbf{k}}) \leftarrow \text{sk}$, then

$$\begin{aligned} e(\text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{y}}) &= e(g_1, g_2)^{\langle \mathbf{A}^T \mathbf{s}_1 + \mathbf{x}, \mathbf{A}^T \mathbf{s}_2 + \mathbf{y} \rangle} \\ &= [\langle \mathbf{A}^T \mathbf{s}_1 + \mathbf{x}, \mathbf{A}^T \mathbf{s}_2 + \mathbf{y} \rangle]_{g_T} \\ &= [\langle \mathbf{A}^T \mathbf{s}_1, \mathbf{A}^T \mathbf{s}_2 \rangle + \langle \mathbf{A}^T \mathbf{s}_1, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{A}^T \mathbf{s}_2 \rangle + \langle \mathbf{x}, \mathbf{y} \rangle]_{g_T} \\ &= [\mathbf{s}_1^T \mathbf{A} \mathbf{A}^T \mathbf{s}_2 + \mathbf{s}_1^T \mathbf{A} \mathbf{y} + \mathbf{x}^T \mathbf{A}^T \mathbf{s}_2 + \langle \mathbf{x}, \mathbf{y} \rangle]_{g_T}. \end{aligned}$$

Similarly, we have that:

$$\begin{aligned} e(\text{sk}_{\mathbf{k}}, \text{ct}_{\mathbf{c}}) &= [\langle \mathbf{B} \cdot (\mathbf{s}_1 \| \mathbf{A} \mathbf{x} + \mathbf{A} \mathbf{A}^T \mathbf{s}_1), \mathbf{B}^* \cdot (\mathbf{A} \mathbf{y} \| \mathbf{s}_2) \rangle]_{g_T} \\ &= [(\mathbf{s}_1^T \mathbf{A} \mathbf{y} + \mathbf{x}^T \mathbf{A}^T \mathbf{s}_2 + \mathbf{s}_1^T \mathbf{A} \mathbf{A}^T \mathbf{s}_2)]_{g_T}. \end{aligned}$$

Thus, $[z]_{g_T} = e(\text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{y}}) / e(\text{sk}_{\mathbf{k}}, \text{ct}_{\mathbf{c}}) = g_T^{\langle \mathbf{x}, \mathbf{y} \rangle}$. Finally, by definition of Rec , $\text{Rec}(1^B, G_T, g_T, [z]_{g_T})$ outputs $\langle \mathbf{x}, \mathbf{y} \rangle$ when $\langle \mathbf{x}, \mathbf{y} \rangle \leq B$. $\square$

Remark 3. We can also construct a similarly structured IPFRE scheme by letting each ciphertext contain three components all in a symmetric group source: $\text{ct}_{\mathbf{x}}, \text{sk}_{\mathbf{k}}, \text{ct}_{\mathbf{c}}$. Correctness of the scheme would follow similar to the correctness of TinyFHIPFE. Here each ciphertext has $d + 8$ group elements.

SECURITY. We show SIM-security of TinyFHIPFE in the generic bilinear group model. Note that multi-message SIM-security, even for non-FH IPFRE, is impossible [ALMT20], so our reliance on idealized models to show SIM-security, which is the most meaningful security notion in practice, is essential here. Prior work on FH-IPFRE by Kim *et al.* [KLM⁺18] similarly analyze their FH-IPFRE scheme in the generic bilinear group model (but do not give an explicit advantage bound).

Theorem 7. *TinyFHIPFE is simulation-secure in the generic bilinear group model. In particular, for every adversary $\mathcal{A}$ that makes q_t total queries to its oracles, there exists a simulator $\mathcal{S}$ such that*

$$\text{Adv}_{\text{IPFRE}, \mathcal{A}, \mathcal{S}}^{\text{sim-ggm}}(\lambda) \leq \frac{(4q_t(d+4))^2}{p}.$$

The description of the bilinear generic group model and the details of the proof are provided in Appendix B.

6 Outsourced Database Protocols for ANN Search

In this Section, first we present a generic construction of an AC-IPFRE for the access policy a_{ann} . Then, we give formal definitions of an ANN data structure and an outsourced database protocol. Next, we present our ODB protocol for ANN that uses this AC-IPFRE. The security proofs of our AC-IPFRE[FH-IPFRE] and ANN-ODB[AC-IPFRE] are in the Appendix C.

6.1 The ACIPFRE[FHIPFE] scheme

We give a transform that lifts any IPFRE to an AC-IPFRE for the access policy a_{ann} . This transform is tailored to a_{ann} and is more efficient than going through the general transform from IPFRE to small tag universe AC-IPFRE in Section 4. We discuss the choice of the access policy function in the introduction, so here we directly present the function a_{ann} with tag-space $\{\text{data}, \text{query}, \text{update}\}$. (The first two arguments of the function namely pp

and aux are ignored.):

$$a_{\text{ann}}(\text{tag}_1, \text{tag}_2) = \begin{cases} 0 & \text{if } \text{tag}_1 = \text{tag}_2 = \text{data}, \\ 0 & \text{if } \text{tag}_1 = \text{tag}_2 = \text{query}, \\ 1 & \text{otherwise.} \end{cases}$$

OUR CONSTRUCTION. Our construction gives an AC-IPFRE for the access policy a_{ann} defined based on any FH-IPFE scheme. In particular, to allow ciphertexts whose tag is **update**, we include both the ciphertext and function key for the input vector. Define an AC-IPFRE scheme AC-IPFRE[FH-IPFE] as in Figure 13.

<u>PPGen(1^λ):</u> 1 $\text{pp} \leftarrow \text{FH-IPFE.PPGen}(1^\lambda)$ 2 return pp <u>Setup($1^d, \text{aux}$):</u> 3 $\text{msk} \leftarrow \text{FH-IPFE.Setup}(1^d)$ 4 return msk <u>Enc($\text{msk}, \text{tag}, \mathbf{x}$):</u> 5 if $\text{tag} = \text{data}$: 6 $\text{ct}_{\mathbf{x}} \leftarrow \text{FH-IPFE.Enc}(\text{msk}, \mathbf{x})$ 7 $\text{ct} \leftarrow (\perp, \text{ct}_{\mathbf{x}})$ 8 if $\text{tag} = \text{query}$: 9 $\text{sk}_{\mathbf{x}} \leftarrow \text{FH-IPFE.KeyGen}(\text{msk}, \mathbf{x})$ 10 $\text{ct} \leftarrow (\text{sk}_{\mathbf{x}}, \perp)$ 11 if $\text{tag} = \text{update}$: 12 $\text{sk}_{\mathbf{x}} \leftarrow \text{FH-IPFE.KeyGen}(\text{msk}, \mathbf{x})$ 13 $\text{ct}_{\mathbf{x}} \leftarrow \text{FH-IPFE.Enc}(\text{msk}, \mathbf{x})$ 14 $\text{ct} \leftarrow (\text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{x}})$ 15 return ct	<u>Eval($1^B, \text{ct}_1, \text{ct}_2$):</u> 16 $(\text{sk}_{\mathbf{x}_1}, \text{ct}_{\mathbf{x}_1}) \leftarrow \text{ct}_1$ 17 $(\text{sk}_{\mathbf{x}_2}, \text{ct}_{\mathbf{x}_2}) \leftarrow \text{ct}_2$ 18 if $\text{sk}_{\mathbf{x}_1} \neq \perp$ and $\text{ct}_{\mathbf{x}_2} \neq \perp$: 19 return $\text{FH-IPFE.Eval}(\text{sk}_{\mathbf{x}_1}, \text{ct}_{\mathbf{x}_2})$ 20 if $\text{sk}_{\mathbf{x}_2} \neq \perp$ and $\text{ct}_{\mathbf{x}_1} \neq \perp$: 21 return $\text{FH-IPFE.Eval}(\text{sk}_{\mathbf{x}_2}, \text{ct}_{\mathbf{x}_1})$ 22 else return $\perp$
--	--

Figure 13: The AC-IPFRE[FH-IPFE] construction.

CORRECTNESS. Evaluation of inner product requires evaluating a $\text{sk}_{\mathbf{x}}$ component from one ciphertext with a $\text{ct}_{\mathbf{x}}$ component from another ciphertext respectively. We can see by observing the cases in lines 18-21 that the inner product evaluation is possible for each case where the access function allows it. Correctness of lines 14 and 16 follow from evaluation correctness of the underlying FH-IPFE scheme.

Remark 4. Alternatively, one can construct an IPFRE scheme as discussed in Remark 3 and use it to construct an AC-IPFRE by omitting one of the components. More precisely, an IPFRE ciphertext contains three components: $\text{ct}_{\mathbf{x}}, \text{ct}_{\mathbf{c}}, \text{ct}_{\mathbf{k}}$. An AC-IPFRE ciphertext omits the components $\text{ct}_{\mathbf{c}}$ and $\text{ct}_{\mathbf{k}}$ for ciphertext with tags **data** and **query** respectively and keeps the ciphertext as is for tag **update**. While the ciphertexts have smaller size with respect to the number of group elements, we do not present this construction due to the difference in performance of symmetric and asymmetric groups for the same security parameter.

SECURITY. Security of AC-IPFRE[FH-IPFE] follows generically from the security of the underlying FH-IPFE. This security proof is given in the Appendix C.1.

6.2 ANN Data Structures

A database $\mathcal{D}$ is abstracted as a collection of n vectors $\mathbf{x}$ from $\mathbb{Z}_p^d$, each associated with a unique identifier $ID \in \mathbb{N}$. We say the database has a lookup operation that on input of an identifier ID , returns the associated vector $\mathbf{x}$. We denote this operation as $\mathbf{x} \leftarrow \mathcal{D}[ID]$. We require the ANN data structure to use a distance metric dist , which is black-box computable from inner-product (*e.g.* Euclidean).

A query is the defined as a vector $\mathbf{q}$ from $\mathbb{Z}_p^d$, which is associated with a unique identifier $ID_{\mathbf{q}} \in \mathbb{N}$. An ANN data structure $\mathcal{DS}_{\text{ANN}}$ consists of three algorithms, namely $\mathcal{DS}_{\text{ANN}} = (\text{Initialize}, \text{Insert}, \text{Search})$. In this work, we consider ANN algorithms which only require inner product comparison in order to insert and search the database. This can be formalized by providing the algorithms oracle access to an inner product comparison operation named $\text{COMPO}(ID_1, ID_2, ID_3)$ which on input three identifiers outputs whether $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle > \langle \mathbf{x}_1, \mathbf{x}_3 \rangle$ where $\mathbf{x}_i \leftarrow \mathcal{D}[ID_i]$. Thus, we rewrite the ANN data structure as follows:

Initialize(p, d, params) : On input a prime p , dimension d , and a set of parameters params , the Initialize algorithm initializes the data structure $\mathcal{DS}_{\text{ANN}}$.

Insert $^{\text{COMPO}(\cdot, \cdot, \cdot)}(ID)$: On input a record identifier ID , the insert algorithm with access to an inner-product comparison oracle COMPO , inserts the record identifier in the data structure $\mathcal{DS}_{\text{ANN}}$. This algorithm does not have a return value.

Search $^{\text{COMPO}(\cdot, \cdot, \cdot)}(ID_{\mathbf{q}})$: On input a query identifier $ID_{\mathbf{q}}$, the search algorithm with access to an inner-product comparison oracle COMPO , outputs an ID from the data structure $\mathcal{DS}_{\text{ANN}}$.

CORRECTNESS. We require search correctness of the ANN data structure. The ANN data structure is said to be $(1 - \delta)$ -correct, if for all databases $\mathcal{D}$ of size $n \in \mathbb{N}$ we have the following:

$$\Pr[ID \leftarrow \text{Search}^{\text{COMPO}(\cdot, \cdot, \cdot)}(ID_{\mathbf{q}}) : \text{dist}(\mathbf{q}, \mathbf{x}_i) \leq \text{dist}(\mathbf{q}, \mathbf{x}_j) \ \forall j \in [n]] \geq 1 - \delta$$

where $\mathcal{DS}_{\text{ANN}} \leftarrow \text{Initialize}(p, d, \text{params})$; $\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(ID_i) \forall i \in [n]$ is run before the search and the probability is over the coins of the insertion algorithm.

6.3 ANN Outsourced Database Protocols

We adopt the notion of outsourced database (ODB) system given by [BKK⁺21]; however, we modify the formalism to cover ANN and dynamic databases. An *ANN outsourced database protocol* ANN-ODB for a database $\mathcal{D} = \{(\mathbf{x}_i, ID_i)_{i \in [n]}\}$, where each $\mathbf{x}_i \in \mathbb{Z}_p^d$ and $ID_i \in \mathbb{N}$ consists of five algorithms $\text{ANN-ODB} = (\text{Setup}, \text{PreQuery}, \text{Query}, \text{PreUpdate}, \text{Update})$:

- **Setup**($1^\lambda, 1^n, \mathcal{D}$): On input a unary encoding of the security parameter λ , the size of the database n , and a database $\mathcal{D} = \{(\mathbf{x}_i, ID_i)_{i \in [n]}\}$, the setup algorithm outputs an ANN data structure $\mathcal{DS}_{\text{ANN}}$ and a database of ciphertexts $\mathcal{D}_S = \{(\text{ct}_i, ID_i)\}_{i \in [n]}$.
- **PreQuery**($\mathbf{q}, ID_{\mathbf{q}}$): On input a query vector $\mathbf{q}$ and its identifier $ID_{\mathbf{q}}$, the preprocess query algorithm outputs a ciphertext of the query vector $\text{ct}_{\mathbf{q}}$.
- **Query**($\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{q}}, ID_{\mathbf{q}})$): On input an ANN data structure $\mathcal{DS}_{\text{ANN}}$ and a tuple of query ciphertext and identifier $(\text{ct}_{\mathbf{q}}, ID_{\mathbf{q}})$, the query algorithm outputs an identifier ID representing the nearest neighbor of the query vector.
- **PreUpdate**($\mathbf{u}, ID_{\mathbf{u}}$): On input an update vector $\mathbf{u}$ and its identifier $ID_{\mathbf{u}}$, the preprocess update algorithm outputs a ciphertext of the update vector $\text{ct}_{\mathbf{u}}$.

- **Update**($\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{u}}, \text{ID}_{\mathbf{u}})$): On input an ANN data structure $\mathcal{DS}_{\text{ANN}}$ and a tuple of update ciphertext and identifier $(\text{ct}_{\mathbf{u}}, \text{ID}_{\mathbf{u}})$, the update algorithm has no output; the data structure is updated internally.

CORRECTNESS. We define **Query** correctness and **Update** correctness respectively.

Query correctness: For all $\lambda, n, \text{ID}_{\mathbf{q}} \in \mathbb{N}$, $\mathbf{q} \in \mathbb{Z}_p^d$, and for any database $\mathcal{D}$, we say that the ANN-ODB satisfies query correctness if the following two outputs are identical:

$$\text{Query}(\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{q}}, \text{ID}_{\mathbf{q}})), \mathcal{DS}_{\text{ANN}}.\text{Search}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_{\mathbf{q}})$$

where $(\mathcal{DS}_{\text{ANN}}, \mathcal{DS}_S) \leftarrow \text{Setup}(1^\lambda, 1^n, \mathcal{D})$ and $\text{ct}_{\mathbf{q}} \leftarrow \text{PreQuery}(\mathbf{q}, \text{ID}_{\mathbf{q}})$.

Update correctness: For all $\lambda, n, \text{ID}_{\mathbf{u}} \in \mathbb{N}$, $\mathbf{u} \in \mathbb{Z}_p^d$, and for any database $\mathcal{D}$, we say that the ANN-ODB satisfies query correctness if the $\mathcal{DS}_{\text{ANN}}$ is identical after running either of the following:

$$\text{Update}(\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{u}}, \text{ID}_{\mathbf{u}})), \mathcal{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_{\mathbf{u}})$$

where $(\mathcal{DS}_{\text{ANN}}, \mathcal{DS}_S) \leftarrow \text{Setup}(1^\lambda, 1^n, \mathcal{D})$ and $\text{ct}_{\mathbf{u}} \leftarrow \text{PreQuery}(\mathbf{u}, \text{ID}_{\mathbf{u}})$.

SECURITY. We give a simulation-based security definition for ANN-ODB. First, we define a leakage profile LP . Let $\text{ANN-ODB} = (\text{Setup}, \text{PreQuery}, \text{Query}, \text{PreUpdate}, \text{Update})$ be a dynamic ODB system for a database $\mathcal{D}$ with data and query vectors from $\mathbb{Z}_p^d$. A leakage profile $\text{LP} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Update}})$ is a tuple of algorithms such that:

Setup Leakage $\mathcal{L}_{\text{Setup}}$: On input a database of records $\mathcal{D} = \{(\mathbf{x}_i, \text{ID}_i)\}_{i \in [n]}$ and an insertion order insert-order , it outputs a string ℓ_{Setup} .

Query Leakage $\mathcal{L}_{\text{Query}}$: On input a database $\mathcal{D} = \{(\mathbf{x}_i, \text{ID}_i)\}_{i \in [n]}$ and a set of previous queries $\mathcal{Q} = \{(\mathbf{q}_i, \text{ID}_i)_{i \in [m]}\}$, it outputs a string ℓ_{Query} .

Update Leakage $\mathcal{L}_{\text{Update}}$: On input a database $\mathcal{D} = \{(\mathbf{x}_i, \text{ID}_i)\}_{i \in [n]}$, a set of previous queries $\mathcal{Q} = \{(\mathbf{q}_i, \text{ID}_i)_{i \in [m]}\}$, and a record $(\mathbf{x}, \text{ID})$, it outputs a string ℓ_{Update} .

The advantage of an adversary $A = (A_1, A_2)$ interacting with the games given in Figure 14 for $\lambda \in \mathbb{N}$ with respect to a simulator $S = (\text{SimSetup}, \text{SimQuery}, \text{SimUpdate})$ and a leakage profile $\text{LP} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Update}})$ is as follows:

$$\text{Adv}_{\text{ANN-ODB}, A, S, \text{LP}}^{\text{sim}}(\lambda) = \Pr[\mathbf{G}_{\text{ANN-ODB}, A}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{ANN-ODB}, A, S, \text{LP}}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

We say that an ANN-ODB scheme is *SIM-secure* under a leakage profile $\text{LP} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Update}})$ if for every PT adversary $A = (A_1, A_2)$ and for all $\lambda \in \mathbb{N}$, there exists a PT simulator $S = (\text{SimSetup}, \text{SimQuery}, \text{SimUpdate})$ such that $\text{Adv}_{\text{ANN-ODB}, A, S, \text{LP}}^{\text{sim}}(\lambda)$ is negligible.

6.4 Our ANN-ODB Protocol

Here, we present our ANN ODB protocol that uses an AC-IPFRE for the access policy a_{ann} . The communication flow of the protocol is given in Figure 15 and the algorithms of the protocol are given in Figure 16. Note that the user and the server use different version of the comparison oracles because of the difference in their databases.

SECURITY. First we give the leakage profile of the protocol $\text{ANN-ODB}[\text{AC-IPFRE}]$ in Figure 17.

Game $\mathbf{G}_{\text{ANN-ODB},A}^{\text{sim-1}}(\lambda)$	Game $\mathbf{G}_{\text{ANN-ODB},A,S}^{\text{sim-0}}(\lambda)$
MAIN: 1 $b \leftarrow \perp$ 2 $(\mathcal{D}, st) \leftarrow \mathcal{A}_1(1^\lambda, 1^n)$ 3 $(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S) \leftarrow \mathcal{S} \text{Setup}(1^\lambda, 1^n, \mathcal{D})$ 4 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \mathcal{A}_2(\mathcal{DS}, \mathcal{D}_S, st)$ 5 while $\text{op}_{\text{type}} \neq \text{terminate}$: 6 if $\text{op}_{\text{type}} = \text{Query}$: 7 $(\mathbf{q}, \text{ID}) \leftarrow \text{op}$ 8 $(\text{ct}_{\mathbf{q}}, \text{ID}) \leftarrow \text{PreQuery}(\mathbf{q}, \text{ID})$ 9 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \mathcal{A}_2(\text{ct}_{\mathbf{q}}, \text{ID}, st)$ 10 if $\text{op}_{\text{type}} = \text{Update}$: 11 $(\mathbf{u}, \text{ID}) \leftarrow \text{op}$ 12 $(\text{ct}_{\mathbf{u}}, \text{ID}) \leftarrow \text{PreUpdate}(\mathbf{u}, \text{ID})$ 13 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \mathcal{A}_2(\text{ct}_{\mathbf{u}}, \text{ID}, st)$ 14 if $\text{op}_{\text{type}} = \text{terminate}$: $b \leftarrow \text{op}$ 15 return b	MAIN: 1 $b \leftarrow \perp$ 2 $\mathcal{Q} \leftarrow \emptyset$ 3 $(\mathcal{D}, st_A) \leftarrow \mathcal{A}_1(1^\lambda, 1^n)$ 4 $\ell_{\text{Setup}} \leftarrow \mathcal{L}_{\text{Setup}}(\mathcal{D})$ 5 $(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S, st_S) \leftarrow \mathcal{S} \text{SimSetup}(\ell_{\text{Setup}})$ 6 $(\text{op}, \text{op}_{\text{type}}, st_A) \leftarrow \mathcal{A}_2(\mathcal{DS}, \mathcal{D}_S, st_A)$ 7 while $\text{op}_{\text{type}} \neq \text{terminate}$: 8 if $\text{op}_{\text{type}} = \text{Query}$: 9 $(\mathbf{q}, \text{ID}) \leftarrow \text{op}$ 10 $\ell_{\text{Query}} \leftarrow \mathcal{L}_{\text{Query}}(\mathcal{D}, \mathcal{Q}, (\mathbf{q}, \text{ID}))$ 11 $(\text{ct}_{\mathbf{q}}, st_S) \leftarrow \mathcal{S} \text{SimQuery}(\ell_{\text{Query}}, st_S)$ 12 $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(\mathbf{q}, \text{ID})\}$ 13 $(\text{op}, \text{op}_{\text{type}}, st_A) \leftarrow \mathcal{A}_2(\text{ct}_{\mathbf{q}}, \text{ID}, st_A)$ 14 if $\text{op}_{\text{type}} = \text{Update}$: 15 $(\mathbf{u}, \text{ID}) \leftarrow \text{op}$ 16 $\ell_{\text{Update}} \leftarrow \mathcal{L}_{\text{Update}}(\mathcal{D}, \mathcal{Q}, (\mathbf{u}, \text{ID}))$ 17 $(\text{ct}_{\mathbf{u}}, st_S) \leftarrow \mathcal{S} \text{SimUpdate}(\ell_{\text{Update}}, st_S)$ 18 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{u}, \text{ID})\}$ 19 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \mathcal{A}_2(\text{ct}_{\mathbf{u}}, \text{ID}, st_A)$ 20 if $\text{op}_{\text{type}} = \text{terminate}$: $b \leftarrow \text{op}$ 21 return b

Figure 14: Games defining SIM-security for ANN-ODB.

Algorithm 1

Client U	Server S
$(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S) \leftarrow \mathcal{S} \text{Setup}(1^\lambda, 1^n, \mathcal{D})$	$(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S)$
$\text{ct}_{\mathbf{q}} \leftarrow \mathcal{S} \text{PreQuery}(\mathbf{q}, \text{ID}_{\mathbf{q}})$	$\text{ct}_{\mathbf{q}}, \text{ID}_{\mathbf{q}}$
	$\text{ID} \leftarrow \text{Query}(\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{q}}, \text{ID}_{\mathbf{q}}))$
$\text{ct}_{\mathbf{u}} \leftarrow \mathcal{S} \text{PreUpdate}(\mathbf{u}, \text{ID}_{\mathbf{u}})$	$\text{ct}_{\mathbf{u}}, \text{ID}_{\mathbf{u}}$
	$\text{Update}(\mathcal{DS}_{\text{ANN}}, (\text{ct}_{\mathbf{u}}, \text{ID}_{\mathbf{u}}))$
..... Oracles	
$\text{COMPO}_U(\text{ID}_1, \text{ID}_2, \text{ID}_3)$	$\text{COMPO}_S(\text{ID}_1, \text{ID}_2, \text{ID}_3)$

Figure 15: Communication flow of ANN ODB protocol.

Theorem 8. *Our ANN-ODB protocol ANN-ODB[AC-IPFRE] is SIM-secure under the leakage profile $\text{LP} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Update}})$ given in Figure 17, given that AC-IPFRE is a SIM-secure for access policy a_{ann} .*

The proof follows easily from SIM-security of the underlying AC-IPFRE and has been deferred to the Appendix C.2.

ANN-ODB[AC-IPFRE] Protocol	
Setup ($1^\lambda, 1^d, 1^n, \mathcal{D}_{\text{init}}$): 1 $\text{pp} \leftarrow \text{AC-IPFRE.PPGen}(1^\lambda)$ 2 $\text{msk} \leftarrow \text{AC-IPFRE.Setup}(1^\lambda, 1^d)$ 3 $\text{DS}_{\text{ANN}}.\text{Initialize}(p, d, \text{params})$ 4 $\mathcal{D} \leftarrow \text{Shuffle}(\mathcal{D}_{\text{init}})$ 5 $\{(\mathbf{x}_i, \text{ID}_i)\}_{i \in [n]} \leftarrow \mathcal{D}$ 6 for $i \in [n]$: 7 $\text{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_i)$ 8 $\text{ct}_i \leftarrow \text{AC-IPFRE.Enc}(\text{msk}, \text{data}, \mathcal{D}[\text{ID}_i])$ 9 $\mathcal{D}_S[\text{ID}_i] \leftarrow \text{ct}_i$ 10 return $(\text{DS}_{\text{ANN}}, \mathcal{D}_S)$ PreQuery ($\mathbf{q}, \text{ID}_q$): 11 $\text{ct}_q \leftarrow \text{AC-IPFRE.Enc}(\text{msk}, \text{query}, \mathbf{q})$ 12 return $(\text{ct}_q, \text{ID}_q)$ Query ($\text{DS}_{\text{ANN}}, (\text{ct}_q, \text{ID}_q)$): 13 $\text{ID} \leftarrow \text{DS}_{\text{ANN}}.\text{Search}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_q)$ 14 return ID PreUpdate ($\mathbf{u}, \text{ID}_u$): 15 $\text{ct}_u \leftarrow \text{AC-IPFRE.Enc}(\text{msk}, \text{update}, \mathbf{u})$ 16 return $(\text{ct}_u, \text{ID}_u)$ Update ($\text{DS}_{\text{ANN}}, (\text{ct}_u, \text{ID}_u)$): 17 $\text{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_u)$ 18 return $\perp$	 COMPO_U ($\text{ID}_1, \text{ID}_2, \text{ID}_3$): 19 for $i \in [3]$: 20 $\mathbf{x}_i \leftarrow \mathcal{D}[\text{ID}_i]$ 21 if $\langle \mathbf{x}_1, \mathbf{x}_2 \rangle > \langle \mathbf{x}_2, \mathbf{x}_3 \rangle$: return 1 22 else : return 0 COMPO_S ($\text{ID}_1, \text{ID}_2, \text{ID}_3$): 23 for $i \in [3]$: 24 $\text{ct}_i \leftarrow \mathcal{D}_S[\text{ID}_i]$ 25 $\text{ip}_1 \leftarrow \text{AC-IPFRE.Eval}(\text{ct}_1, \text{ct}_2)$ 26 $\text{ip}_2 \leftarrow \text{AC-IPFRE.Eval}(\text{ct}_1, \text{ct}_3)$ 27 if $\text{ip}_1 > \text{ip}_2$: return 1 28 else : return 0 Shuffle ($\mathcal{D}_{\text{old}}$): 29 $\mathcal{D}_{\text{new}} \leftarrow \emptyset$ 30 $n \leftarrow \text{length}(\mathcal{D}_{\text{old}})$ 31 $\sigma \leftarrow \text{S}_n$ 32 for $i \in [n]$: 33 $\mathcal{D}_{\text{new}}[\text{ID}_i] \leftarrow \mathcal{D}_{\text{old}}[\text{ID}_{\sigma(i)}]$ 34 return $\mathcal{D}_{\text{new}}$

Figure 16: ANN-ODB[AC-IPFRE] protocol.

7 Experimental Evaluation

In this Section, we first evaluate the performance of the proposed TinyFHIPFE scheme and compare it with prior schemes [KLM⁺18, KKS19] to demonstrate its computational efficiency. We then implement the proposed ANN-ODB protocol, initiated by AC-IPFRE[TinyFHIPFE], to illustrate its practicality. For simplicity, we denote this protocol as Π in the following discussion.

IMPLEMENTATION SETUP. For the implementation of the underlying bilinear pairing group, we utilized the Relic library [AGM⁺], and configured the Curve25519 [Ber06]. Our experiments were conducted on a laptop equipped with an Intel I7-9750H CPU and 16 GB of memory, running Ubuntu 22.04.

7.1 FH-IPFE Benchmarks

We benchmark the proposed TinyFHIPFE against FH-IPFE schemes proposed in [KLM⁺18, KKS19], denoted as KLM⁺ and KKS, respectively. We choose KLM⁺ as a comparison target due to its compact ciphertext length and efficient Eval algorithm. However, its Setup algorithm is slow because it computes the inverse of a matrix of size $d \times d$, which takes time $\mathcal{O}(d^3)$. KKS attempt to address this issue by using $\mathcal{O}(d)$ master secret key but their ciphertexts and function keys are longer than both KLM⁺ and TinyFHIPFE. As illustrated in Figure 18, the proposed TinyFHIPFE exhibits even better performance. Our Setup time is almost always negligible, involving sampling of only $2d + 16$ random elements from $\mathbb{Z}_p$ and a constant-time operation of finding the inverse of a 4×4 matrix.

The proposed TinyIPFRE also outperforms in Enc and KeyGen operations. While KLM⁺'s Enc and KeyGen involve d^2 multiplications in $\mathbb{Z}_p$, which are cheaper than group

Leakage Profile LP

```

 $\mathcal{L}_{\text{Setup}}(\mathcal{D}, \text{insert-order}):$ 
1 for  $i, j, k \in [n]$  :
2    $\mathbf{x}_i \leftarrow \mathcal{D}[\text{ID}_i]; \mathbf{x}_j \leftarrow \mathcal{D}[\text{ID}_j]; \mathbf{x}_k \leftarrow \mathcal{D}[\text{ID}_k]$ 
3   if  $\langle \mathbf{x}_i, \mathbf{x}_j \rangle > \langle \mathbf{x}_i, \mathbf{x}_k \rangle$ :
4      $\text{ip-comp}[\text{ID}_i][\text{ID}_j][\text{ID}_k] \leftarrow 1$ 
5   else :
6      $\text{ip-comp}[\text{ID}_i][\text{ID}_j][\text{ID}_k] \leftarrow 0$ 
7   if  $\text{insert-order} = \perp$  :
8      $\text{insert-order} \leftarrow \{1, \dots, n\}$ 
9    $\ell_{\text{Setup}} \leftarrow \text{ip-comp} \cup \text{insert-order}$ 
10  return  $\ell_{\text{Setup}}$ 

 $\mathcal{L}_{\text{Query}}(\mathbf{q}, \mathcal{D}, \mathcal{Q}):$ 
11   $((\text{ID}_1, \mathbf{x}_1), \dots, (\text{ID}_n, \mathbf{x}_n)) \leftarrow \mathcal{D}$ 
12  for  $i \in [n]$  :
13     $\text{ip-val}[\text{ID}_i] \leftarrow \langle \mathbf{q}, \mathcal{D}[\text{ID}_i] \rangle$ 
14   $\ell_{\text{Query}} \leftarrow \text{ip-val}$ 
15  return  $\ell_{\text{Query}}$ 

 $\mathcal{L}_{\text{Update}}(\mathbf{u}, \mathcal{D}, \mathcal{Q}):$ 
16   $((\text{ID}_1, \mathbf{x}_1), \dots, (\text{ID}_n, \mathbf{x}_n)) \leftarrow \mathcal{D}$ 
17   $((\text{ID}_{n+1}, \mathbf{q}_1), \dots, (\text{ID}_{n+m}, \mathbf{q}_m)) \leftarrow \mathcal{Q}$ 
18  for  $i \in [n]$  :
19     $\text{ip-val}[\text{ID}_i] \leftarrow \langle \mathbf{u}, \mathbf{x}_i \rangle$ 
20  for  $i \in [m]$  :
21     $\text{ip-val}[\text{ID}_{n+i}] \leftarrow \langle \mathbf{u}, \mathbf{q}_i \rangle$ 
22   $\ell_{\text{Update}} \leftarrow \text{ip-val}$ 
23  return  $\ell_{\text{Update}}$ 

```

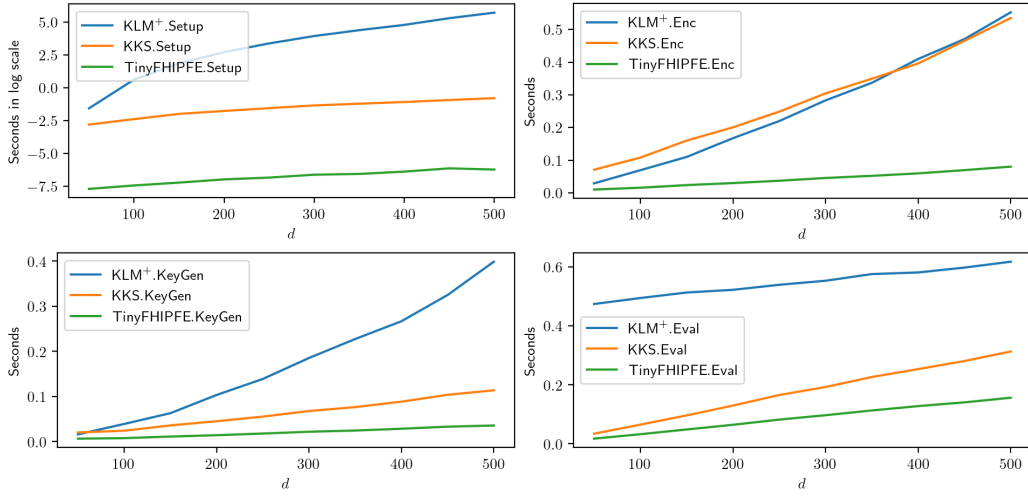
Figure 17: Leakage Profile of ANN-ODB[AC-IPFRE].

multiplication and exponentiation, their performance diminishes as plaintext length increases. For example, for plaintexts of length 500, $\text{KLM}^+.\text{Enc}$ runs roughly 7.5x slower than $\text{TinyFHIPFE}.\text{Enc}$. KKS performs a linear number of operations for both Enc and KeyGen , yet they have worse runtime because they perform more expensive group exponentiation. Namely, on average, $\text{KKS}.\text{Enc}$ is 6.5x slower than $\text{TinyFHIPFE}.\text{Enc}$, and $\text{KKS}.\text{KeyGen}$ is 2.6x slower than $\text{TinyFHIPFE}.\text{KeyGen}$.

To benchmark performance of the Eval operation, we bound the resulting inner-product by 10,000. The results presented here assume *preprocessing*, wherein a lookup table is constructed for TinyFHIPFE and KKS that allows solving the discrete logarithm to a fixed base in constant time. Notably, KLM^+ does not support such preprocessing because its evaluation algorithm does not solve discrete logarithm with respect to a *fixed base*. In this case, $\text{KKS}.\text{Eval}$ runs approximately 2x than $\text{TinyFHIPFE}.\text{Eval}$, with $\text{KLM}^+.\text{Eval}$ falling significantly further behind. Overall, we find that TinyFHIPFE exhibits significantly better performance than prior schemes, particularly for long plaintexts.

7.2 ANN-ODB Evaluation

In the following evaluation, we conduct experiments using the following datasets: SIFT [JDS10], MNIST [LCB10], NYTimes [New08], and GloVe [PSM14]. These datasets are commonly used for benchmarking ANN algorithms [ABF20] and their details are shown in Table 2. The “max inner product” is the maximum value of all pairwise inner products of vectors from the dataset. This value is useful later when we build lookup tables for solving discrete

**Figure 18:** Benchmarking FH-IPFE schemes.

log problem in constant time.

Table 2: Overview of datasets used in evaluation.

	SIFT	MNIST	NYTimes	GloVe
Dimension	128	784	256	100
Train Size	1,000,000	60,000	290,000	1,183,514
Distance Metric	Euclidean	Euclidean	Angular	Angular
Max Inner Product	24649	65025	74540	6326734

CHOICE OF ANN DATA STRUCTURE. Recall that we require the underlying ANN data structure to be inner product based. Fortunately, this is the case for the state-of-the-art Hierarchical Navigable Small Worlds (HNSW) algorithm [MY20, ABF20]. In the evaluation below we use the following parameters to setup the HNSW algorithm: $M = 32$, where M is the number of connections made for each new vertex during construction, $efConstruction = 30$ where $efConstruction$ is the number of candidate neighbors explored during construction, and $efSearch = 256$, where $efSearch$ is the number of candidate neighbors explored during search.

Table 3: Evaluation of our ANN-ODB protocol.

		SIFT	MNIST	NYTimes	GloVe
Setup	Index	315.2s	48.6s	144.7s	362.9s
	Encryption	55m	4m	17m	68m
	Preprocessing	8.1s	23.3s	25.7s	2111.2s
Query		8.5s	23.5s	11.4s	7.3s
Update		7.6s	22.2s	10.1s	6.6s

EVALUATION RESULTS. We present the evaluation results of Π in Table 3. We divide the setup oracle into three phases: the index phase, the encryption phase, and the preprocessing phase. In the index phase, the user indexes the plaintext data vectors using the HNSW algorithm, which is independent of the choice of the underlying encryption scheme. The preprocessing time is time needed to build the lookup table. The query time evaluation encompasses both the time taken by the client U to encrypt the query and the time it takes for the server to respond. Note that we omit network latency in this evaluation since there is only one round of communication involved. To determine the query time, we index

the entire training set and then select 50 vectors from the testing set to query, computing the average time of querying one vector. Similarly, the update protocol evaluates the time taken by the client U to encrypt an update query and the time it takes for the server to add it to the index. In this scenario, we index the first 90% of the training dataset and add 50 more vectors from the training set to the index, calculating the average time of updating one datapoint. Notably, during both the query and update phases, the client remains extremely lightweight, with execution times of only 20ms and 38ms, respectively. Additionally, the vast majority of the computational workload is handled on the server side.

COMPARISON TO OTHER SYSTEMS. Boldyreva and Tang [BT21] present a protocol that operates in the same setting as ours. They present a construction based on locality-sensitive hashing, symmetric encryption, and an oblivious map, which achieves strong security, in particular hiding access pattern. However, their scheme is significantly less practical than ours. In particular, running a search query on the MNIST dataset involves an interactive phase between the client and the server, where at least 11 calls to their most expensive component are made, lasting over 8 minutes in duration. Also note they achieve a 95% recall rate on SIFT, whereas we always achieve the baseline recall rate, here 98.3%.

References

- [AAB⁺15] Shashank Agrawal, Shweta Agrawal, Saikrishna Badrinarayanan, Abishek Kumarasubramanian, Manoj Prabhakaran, and Amit Sahai. On the practical security of inner product functional encryption. In Jonathan Katz, editor, *PKC 2015: 18th International Conference on Theory and Practice of Public Key Cryptography*, volume 9020 of *Lecture Notes in Computer Science*, pages 777–798, Gaithersburg, MD, USA, March 30 – April 1, 2015. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-46447-2_35.
- [ABDP15] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Simple functional encryption schemes for inner products. In Jonathan Katz, editor, *PKC 2015: 18th International Conference on Theory and Practice of Public Key Cryptography*, volume 9020 of *Lecture Notes in Computer Science*, pages 733–751, Gaithersburg, MD, USA, March 30 – April 1, 2015. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-46447-2_33.
- [ABF20] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020. doi:10.1016/j.is.2019.02.006.
- [ACGU20] Michel Abdalla, Dario Catalano, Romain Gay, and Bogdan Ursu. Inner-product functional encryption with fine-grained access control. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 467–497, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64840-4_16.
- [AGM⁺] D. F. Aranha, C. P. L. Gouvêa, T. Markmann, R. S. Wahby, and K. Liao. RELIC is an Efficient LIBrary for Cryptography. <https://github.com/relic-toolkit/relic>.
- [AGT21a] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-input quadratic functional encryption from pairings. In Tal Malkin and Chris Peikert, editors,

- Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 208–238, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. doi:[10.1007/978-3-030-84259-8_8](https://doi.org/10.1007/978-3-030-84259-8_8).
- [AGT21b] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-party functional encryption. In Kobbi Nissim and Brent Waters, editors, *TCC 2021: 19th Theory of Cryptography Conference, Part II*, volume 13043 of *Lecture Notes in Computer Science*, pages 224–255, Raleigh, NC, USA, November 8–11, 2021. Springer, Cham, Switzerland. doi:[10.1007/978-3-030-90453-1_8](https://doi.org/10.1007/978-3-030-90453-1_8).
- [Ala96] N. S. Aladow. On the ditribution of quadratic residues and nonresidues of a prime P in the series $1, 2, \dots, P - 1$. *Matematicheskii Sbornik*, 18(1):61–75, 1896. URL: <http://mi.mathnet.ru/eng/msb/v18/i1/p61>.
- [ALMT20] Shweta Agrawal, Benoît Libert, Monosij Maitra, and Radu Titiiu. Adaptive simulation security for inner product functional encryption. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 12110 of *Lecture Notes in Computer Science*, pages 34–64, Edinburgh, UK, May 4–7, 2020. Springer, Cham, Switzerland. doi:[10.1007/978-3-030-45374-9_2](https://doi.org/10.1007/978-3-030-45374-9_2).
- [BBO07] Mihir Bellare, Alexandra Boldyreva, and Adam O'Neill. Deterministic and efficiently searchable encryption. In Alfred Menezes, editor, *Advances in Cryptology – CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 535–552, Santa Barbara, CA, USA, August 19–23, 2007. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-540-74143-5_30](https://doi.org/10.1007/978-3-540-74143-5_30).
- [BCLO09] Alexandra Boldyreva, Nathan Chenette, Younho Lee, and Adam O'Neill. Order-preserving symmetric encryption. In Antoine Joux, editor, *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 224–241, Cologne, Germany, April 26–30, 2009. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-642-01001-9_13](https://doi.org/10.1007/978-3-642-01001-9_13).
- [Ber70] E. R. Berlekamp. Factoring polynomials over large finite fields. *Mathematics of Computation*, 24(111):713–735, 1970. doi:[10.2307/2004849](https://doi.org/10.2307/2004849).
- [Ber06] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006: 9th International Conference on Theory and Practice of Public Key Cryptography*, volume 3958 of *Lecture Notes in Computer Science*, pages 207–228, New York, NY, USA, April 24–26, 2006. Springer Berlin Heidelberg, Germany. doi:[10.1007/11745853_14](https://doi.org/10.1007/11745853_14).
- [BGW24] Alexandra Boldyreva, Zichen Gui, and Bogdan Warinschi. Understanding leakage in searchable encryption: a quantitative approach. *Proceedings on Privacy Enhancing Technologies*, 2024(4):503–524, October 2024. doi:[10.56553/popets-2024-0127](https://doi.org/10.56553/popets-2024-0127).
- [BJK15] Allison Bishop, Abhishek Jain, and Lucas Kowalczyk. Function-hiding inner product encryption. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology – ASIACRYPT 2015, Part I*, volume 9452 of *Lecture Notes in Computer Science*, pages 470–491, Auckland, New Zealand, November 30 – December 3, 2015. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-662-48797-6_20](https://doi.org/10.1007/978-3-662-48797-6_20).

- [BKK⁺21] Dmytro Bogatov, Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O’Neill. epsolute: Efficiently querying databases while providing differential privacy. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2262–2276, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press. doi:10.1145/3460120.3484786.
- [BR06] Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 409–426, St. Petersburg, Russia, May 28 – June 1, 2006. Springer Berlin Heidelberg, Germany. doi:10.1007/11761679_25.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In Yuval Ishai, editor, *TCC 2011: 8th Theory of Cryptography Conference*, volume 6597 of *Lecture Notes in Computer Science*, pages 253–273, Providence, RI, USA, March 28–30, 2011. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-642-19571-6_16.
- [BT21] Alexandra Boldyreva and Tianxin Tang. Privacy-preserving approximate k-nearest-neighbors search that hides access, query and volume patterns. *Proceedings on Privacy Enhancing Technologies*, 2021(4):549–574, October 2021. doi:10.2478/popets-2021-0084.
- [CLT18] Guilhem Castagnos, Fabien Laguillaumie, and Ida Tucker. Practical fully secure unrestricted inner product functional encryption modulo p . In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 733–764, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-030-03329-3_25.
- [DDM16] Pratish Datta, Ratna Dutta, and Sourav Mukhopadhyay. Functional encryption for inner product with full function privacy. In Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang, editors, *PKC 2016: 19th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 9614 of *Lecture Notes in Computer Science*, pages 164–195, Taipei, Taiwan, March 6–9, 2016. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-49384-7_7.
- [DDM⁺23] Uddipana Dowerah, Subhranil Dutta, Aikaterini Mitrokotsa, Sayantan Mukherjee, and Tapas Pal. Unbounded predicate inner product functional encryption from pairings. *Journal of Cryptology*, 36(3):29, July 2023. doi:10.1007/s00145-023-09458-2.
- [DP19] Edouard Dufour Sans and David Pointcheval. Unbounded inner-product functional encryption with succinct keys. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *ACNS 19: 17th International Conference on Applied Cryptography and Network Security*, volume 11464 of *Lecture Notes in Computer Science*, pages 426–441, Bogota, Colombia, June 5–7, 2019. Springer, Cham, Switzerland. doi:10.1007/978-3-030-21568-2_21.
- [FGHO22] Georg Fuchsbauer, Riddhi Ghosal, Nathan Hauke, and Adam O’Neill. Approximate distance-comparison-preserving symmetric encryption. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22: 13th International Conference*

- on Security in Communication Networks*, volume 13409 of *Lecture Notes in Computer Science*, pages 117–144, Amalfi, Italy, September 12–14, 2022. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-14791-3_6](https://doi.org/10.1007/978-3-031-14791-3_6).
- [GGG⁺14] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 578–602, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-642-55220-5_32](https://doi.org/10.1007/978-3-642-55220-5_32).
- [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th Annual Symposium on Foundations of Computer Science*, pages 40–49, Berkeley, CA, USA, October 26–29, 2013. IEEE Computer Society Press. doi:[10.1109/FOCS.2013.13](https://doi.org/10.1109/FOCS.2013.13).
- [HJL⁺17] Helene Haagh, Yue Ji, Chenxing Li, Claudio Orlandi, and Yifan Song. Revealing encryption for partial ordering. In Máire O'Neill, editor, *16th IMA International Conference on Cryptography and Coding*, volume 10655 of *Lecture Notes in Computer Science*, pages 3–22, Oxford, UK, December 12–14, 2017. Springer, Cham, Switzerland. doi:[10.1007/978-3-319-71045-7_1](https://doi.org/10.1007/978-3-319-71045-7_1).
- [HLOW24] Mohammad Hajiabadi, Roman Langrehr, Adam O'Neill, and Mingyuan Wang. On the black-box complexity of private-key inner-product functional encryption. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part III*, volume 15366 of *Lecture Notes in Computer Science*, pages 318–343, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland. doi:[10.1007/978-3-031-78020-2_11](https://doi.org/10.1007/978-3-031-78020-2_11).
- [HWHBS08] G.H. Hardy, E.M. Wright, D.R. Heath-Brown, and J.H. Silverman. *An Introduction to the Theory of Numbers*. Oxford mathematics. Oxford University Press, sixth edition, 2008.
- [JDJ19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi:[10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [JDS10] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128, 2010. doi:[10.1109/TPAMI.2010.57](https://doi.org/10.1109/TPAMI.2010.57).
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd Annual ACM Symposium on Theory of Computing*, pages 60–73, Virtual Event, Italy, June 21–25, 2021. ACM Press. doi:[10.1145/3406325.3451093](https://doi.org/10.1145/3406325.3451093).
- [JP18] Marc Joye and Alain Passelègue. Function-revealing encryption - definitions and constructions. In Dario Catalano and Roberto De Prisco, editors, *SCN 18: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 527–543, Amalfi, Italy, September 5–7, 2018. Springer, Cham, Switzerland. doi:[10.1007/978-3-319-98113-0_28](https://doi.org/10.1007/978-3-319-98113-0_28).

- [KKS19] Sungwook Kim, Jinsu Kim, and Jae Hong Seo. A new approach to practical function-private inner product encryption. *Theoretical Computer Science*, 783:22–40, 2019. doi:[10.1016/j.tcs.2019.03.016](https://doi.org/10.1016/j.tcs.2019.03.016).
- [KLM⁺18] Sam Kim, Kevin Lewi, Avradip Mandal, Hart Montgomery, Arnab Roy, and David J. Wu. Function-hiding inner product encryption is practical. In Dario Catalano and Roberto De Prisco, editors, *SCN 18: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 544–562, Amalfi, Italy, September 5–7, 2018. Springer, Cham, Switzerland. doi:[10.1007/978-3-319-98113-0_29](https://doi.org/10.1007/978-3-319-98113-0_29).
- [KZ20] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’20, page 39–48, New York, NY, USA, 2020. Association for Computing Machinery. doi:[10.1145/3397271.3401075](https://doi.org/10.1145/3397271.3401075).
- [LCB10] Yann LeCun, Corinna Cortes, and CJ Burges. MNIST handwritten digit database. *ATT Labs*, 2, 2010. URL: <http://yann.lecun.com/exdb/mnist>.
- [LLW21] Qiqi Lai, Feng-Hao Liu, and Zhedong Wang. New lattice two-stage sampling technique and its applications to functional encryption - stronger security and smaller ciphertexts. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 498–527, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. doi:[10.1007/978-3-030-77870-5_18](https://doi.org/10.1007/978-3-030-77870-5_18).
- [MY20] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(4):824–836, 2020.
- [New08] David Newman. Bag of Words. UCI Machine Learning Repository, 2008. doi:[10.24432/C5ZG6P](https://doi.org/10.24432/C5ZG6P).
- [PR12] Omkant Pandey and Yannis Rouselakis. Property preserving symmetric encryption. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 375–391, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. doi:[10.1007/978-3-642-29011-4_23](https://doi.org/10.1007/978-3-642-29011-4_23).
- [PSM14] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014. URL: <http://www.aclweb.org/anthology/D14-1162>.
- [PT18] P. Pollack and E. Treviño. Finding the four squares in lagrange’s theorem. *Integers: Electronic Journal of Combinatorial Number Theory*, 18A, 2018. doi:[10.5281/zenodo.10581444](https://doi.org/10.5281/zenodo.10581444).
- [RS86] Michael O. Rabin and Jeffery O. Shallit. Randomized algorithms in number theory. *Communications on Pure and Applied Mathematics*, 39(S1):S239–S256, 1986. doi:[10.1002/cpa.3160390713](https://doi.org/10.1002/cpa.3160390713).
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *Advances in Cryptology – EUROCRYPT’97*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266, Konstanz, Germany, May 11–15, 1997. Springer Berlin Heidelberg, Germany. doi:[10.1007/3-540-69053-0_18](https://doi.org/10.1007/3-540-69053-0_18).

- [TAO16] Junichi Tomida, Masayuki Abe, and Tatsuaki Okamoto. Efficient functional encryption for inner-product values with full-hiding security. In Matt Bishop and Anderson C. A. Nascimento, editors, *ISC 2016: 19th International Conference on Information Security*, volume 9866 of *Lecture Notes in Computer Science*, pages 408–425, Honolulu, HI, USA, September 3–6, 2016. Springer, Cham, Switzerland. doi:10.1007/978-3-319-45871-7_24.
- [TT18] Junichi Tomida and Katsuyuki Takashima. Unbounded inner product functional encryption from bilinear maps. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 609–639, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-030-03329-3_21.
- [Üna20] Akin Ünal. Impossibility results for lattice-based functional encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 169–199, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-45721-1_7.
- [Wee20] Hoeteck Wee. Functional encryption for quadratic functions from k -lin, revisited. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part I*, volume 12550 of *Lecture Notes in Computer Science*, pages 210–228, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64375-1_8.
- [XXL⁺21] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *International Conference on Learning Representations, ICLR '21*, 2021. URL: <https://openreview.net/forum?id=zeFrfgYZln>.
- [YNL21] Andrew Yates, Rodrigo Nogueira, and Jimmy Lin. Pretrained transformers for text ranking: BERT and beyond. In Greg Kondrak, Kalina Bontcheva, and Dan Gillick, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Tutorials*, pages 1–4, Online, June 2021. Association for Computational Linguistics. URL: <https://aclanthology.org/2021.naacl-tutorials.1/>, doi:10.18653/v1/2021.naacl-tutorials.1.
- [ZZV22] Hansi Zeng, Hamed Zamani, and Vishwa Vinay. Curriculum learning for dense retrieval distillation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '22*, page 1979–1983, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3477495.3531791.

A From NZIP-AC-IPBFRE to FH-IPBFE

In this Section we give an optimized version of the transformation from NZIP-AC-IPBFRE to FH-IPBFE. In many IPBFE/IPBFRE constructions the modulus m is a prime. In this case, the following theorem gives us a better solution than Lagrange's four square theorem.

Theorem 9. *There exists a PT algorithm PrimeSoS that inputs a prime m and outputs (a_1, a_2) such that $1 + a_1^2 + a_2^2 = 0 \pmod{m}$.*

Proof. We first proof the existence of a solution. The proof is taken from [HWHBS08, §6.7].

The statement is trivial for $m = 2$. Therefore, assume that m is odd in the following.

The polynomial $X^2 - c$, for $c \in \mathbb{Z}_m$, has at most two distinct solutions in $\mathbb{Z}_m$. Thus, by squaring every $a \in \mathbb{Z}_m$ we hit each quadratic residue at most twice and thus there must be at least $(m+1)/2$ quadratic residue in $\mathbb{Z}_m$.

Using the above argument with the polynomial $-X^2 - c - 1$, there must also be $(m+1)/2$ integers of the form $-a^2 - 1$ in $\mathbb{Z}_m$. By the pigeonhole principle, there must be one c with $-a^2 - 1 = c = b^2 \pmod{m}$ for some $a, b \in \mathbb{Z}_m$ which can be turned immediately into a solution of the above equation.

We next describe PrimeSoS, namely how to compute a_1 and a_2 efficiently. We show how to proceed for primes of the form $m = 4k + 3$. For primes of the form $m = 4k + 1$ one can proceed analogously (but we do not need that here, because we will show below that for those prime a solution with $\ell = 1$ can be found).

The first step is to find a number $c \in \mathbb{Z}_m$ that satisfies the properties required in the proof, i.e. such that $X^2 - c$ and $-X^2 - c - 1$ have both solutions in $\mathbb{Z}_m$. That is, c and $-(c+1)$ should be both quadratic residues. -1 is a quadratic non-residue for primes of the form $m = 4k + 3$, the second requirement is fulfilled iff $c+1$ is a quadratic non-residue (law of quadratic reciprocity and its first supplement). There are $(m+1)/4$ possible values for c satisfying this [Ala96], we can thus just compute one by trial and error. The values a_1 and a_2 can now be computed by computing the roots of the polynomials $X^2 - c$ and $-X^2 - c - 1$, which can be done efficiently [Ber70]. $\square$

We now focus on further optimizing efficiency. For primes of the form $m = 4k + 1$, we can do even better and find a solution with $\ell = 2$.

Theorem 10. *There exists a PT algorithm Prime1Mod4SoS that inputs a prime m of the form $m = 4k + 1$ for an $k \in \mathbb{Z}$ and outputs a_1 such that $1 + a_1^2 = 0 \pmod{m}$.*

Proof. The existence of such an a_1 follows from the first supplement to the law of quadratic reciprocity, $\left(\frac{-1}{m}\right) = 1$ for primes m of the form $m = 4k + 1$. This implies that there exists $a_1 \in \mathbb{Z}_m$ with $a_1^2 = -1 \pmod{m}$, which can be rearranged to the equation in the theorem.

We proceed to describe Prime1Mod4SoS. We can find a_1 from the above theorem in expected time $\mathcal{O}(\log m)$ as follows: By Euler's criterion, $\left(\frac{x}{m}\right) = x^{\frac{m-1}{2}} \pmod{m}$ and thus for every quadratic non-residue x we have $x^{\frac{m-1}{2}} = -1 \pmod{m}$ and by setting $a_1 = x^{\frac{m-1}{4}}$ we have a solution. Since every other (non-zero) number in $\mathbb{Z}_m$ is a quadratic non-residue, we can find one efficiently by trial and error (again, converting the algorithm to worst-case polynomial-time in standard ways). $\square$

```

Setup'(1λ, 1d):
1 Obtain modulus  $m$  from  $\text{pp}$ 
2 if  $m$  is prime then
3   if  $m \equiv 1 \pmod{4}$  then
4      $(a_i)_{1 \leq i \leq \ell} := (a_1) \leftarrow \$ \text{Prime1Mod4SoS}(m)$ 
5   else
6      $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2) \leftarrow \$ \text{PrimeSoS}(m)$ 
7   else
8      $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2, a_3, a_4) \leftarrow \$ \text{LagrangeSoS}(m - 1)$ 
9    $\text{msk} \leftarrow \text{Setup}(1^\lambda, 1^d, 1^{2\ell+1})$ 
10 return  $\text{msk}' := (\text{msk}, (a_i)_{1 \leq i \leq \ell})$ 
KeyGen( $\text{msk}' = (\text{msk}, (a_i)_{1 \leq i \leq \ell}), \mathbf{x}$ ):
11 return  $\text{Enc}(\text{msk}, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{x})$ 
Enc'( $\text{msk}' = (\text{msk}, (a_i)_{1 \leq i \leq \ell}), \mathbf{y}$ ):
12 return  $\text{Enc}(\text{msk}, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{y})$ 
Dec( $\text{pp}, \text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{y}}$ ):
13 return  $\text{Eval}(\text{pp}, \text{sk}_{\mathbf{x}}, \text{ct}_{\mathbf{y}})$ 

```

Figure 19: Transform from $\text{NZIP-AC-IPBFRE} = (\text{PPGen}, \text{Setup}, \text{Enc}, \text{Eval})$ to $\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}] = (\text{PPGen}, \text{Setup}', \text{KeyGen}, \text{Enc}', \text{Dec})$.

A.0.1 Our Encoding Functions and Their Properties

Given a sum of squares decomposition of the modulus $1 + a_1^2 + \dots + a_\ell^2 = 0 \pmod{m}$, the following two tag vectors will be used in our transformation:

$$\mathbf{u}_{(a_i)_{1 \leq i \leq \ell}} := \begin{pmatrix} 1 \\ a_1 \\ 0 \\ \vdots \\ a_\ell \\ 0 \end{pmatrix} \in \mathbb{Z}_m^{2\ell+1} \quad \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}} := \begin{pmatrix} 1 \\ 0 \\ a_1 \\ \vdots \\ 0 \\ a_\ell \end{pmatrix} \in \mathbb{Z}_m^{2\ell+1}.$$

The following gives the crucial properties of the tag vectors, which are easy to prove by inspection.

Lemma 1.

$$\langle \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}} \rangle = 0 \quad \text{and} \quad \langle \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}} \rangle = 0.$$

Lemma 2.

$$\langle \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}} \rangle = a_1^2 = 1.$$

A.0.2 The Transformation

Recall in Section 4.2 we showed how to construct NZIP-AC-IPBFRE generically from IPBFRE for the same inner-product based functionality $f(\mathbf{x}, \mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$. We now construct an FH-IPBFE generically from an NZIP-AC-IPBFRE . This transformation can easily be generalized to constructing FH-FE for any functionality from NZIP-AC-FRE for the same functionality. The transformation is given in Figure 19.

Proposition 4. *If NZIP-AC-IPBFRE is evaluation correct, then $\text{FH-IPBFE}[\text{IPBFRE}]$ is decryption correct.*


```

Adversary B = (B1, B2)

B1(1λ, pp):
1 Obtain modulus m from pp
2 if m is prime then
3   if m ≡ 1 mod 4 then
4     (ai)1≤i≤ℓ := (a1) ←$ Prime1Mod4SoS(m)
5   else
6     (ai)1≤i≤ℓ := (a1, a2) ←$ PrimeSoS(m)
7   else
8     (ai)1≤i≤ℓ := (a1, a2, a3, a4) ←$ LagrangeSoS(m - 1)
9   (d, st) ←$ A1(1λ, pp)
10  return (d, 2ℓ - 1, st)

B2ENC O' (·, ·)(st):
11 b' ←$ A2ENC O (·, ·), KEY GEN O (·, ·)(st)
12 return b'

ENC O(x0, x1):
13 ct ← ENC O'(x0, x1, v(ai)1≤i≤ℓ)
14 return ct

KEY GEN O(y0, y1):
15 sk ← ENC O'(y0, y1, u(ai)1≤i≤ℓ)
16 return sk

```

Figure 20: Adversary against IND-security of NZIP-AC-IPBFRE.

Proof. Let $\text{pp} \in \text{Out}(\text{PPGen}(1^\lambda))$, $\text{msk} \in \text{Out}(\text{Setup}(1^\lambda, 1^d))$, $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_m^d$, $\text{sk}_\mathbf{x} \in \text{Out}(\text{KeyGen}(\text{msk}, \mathbf{x}))$ and $\text{ct}_\mathbf{y} \in \text{Out}(\text{Enc}(\text{msk}, \mathbf{y}))$. Then by Lemma 2, the tags of the NZIP-AC-IPBFRE ciphertexts $\text{sk}_\mathbf{x}$ and $\text{ct}_\mathbf{y}$ satisfy the access policy and by evaluation correctness of NZIP-AC-IPBFRE $\text{Eval}(\text{pp}, \text{sk}_\mathbf{x}, \text{ct}_\mathbf{y}) = g(\langle \mathbf{x}, \mathbf{y} \rangle)$. Thus decryption correctness of FH-IPBFE[IPBFRE] follows. $\square$

Theorem 11 (IND-security). *If NZIP-AC-IPBFRE is IND-secure, then FH-IPBFE[NZIP-AC-IPBFRE] is IND-secure. Concretely, for every adversary A against IND-security of FH-IPBFE[NZIP-AC-IPBFRE], there exists an adversary B against IND-security of NZIP-AC-IPBFRE with roughly the same runtime, such that for all $\lambda \in \mathbb{N}$*

$$\text{Adv}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A}^{\text{ind}}(\lambda) \leq \text{Adv}_{\text{NZIP-AC-IPBFRE}, B}^{\text{ind}}(\lambda).$$

Proof. Assume there exists an adversary $A = (A_1, A_2)$ against the IND-security of FH-IPBFE[NZIP-AC-IPBFRE]. We then construct an adversary $B = (B_1, B_2)$ against the adaptive IND security of NZIP-AC-IPBFRE as follows:

It is easy to see that when A plays the $\mathbf{G}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A}^{\text{ind-b}}(\lambda)$ game, it perfectly simulates the game $\mathbf{G}_{\text{NZIP-AC-IPBFRE}, B}^{\text{ind-b}}(\lambda)$ for B and outputs the same bit as B. To show that these adversaries have the same advantage, what remains to show is that A causes the game to output $\perp$ exactly when B does.

Let $\mathbf{x}_0^j, \mathbf{x}_1^j$ be the input to the j -th ENC oracle query of A_2 and $\mathbf{y}_0^k, \mathbf{y}_1^k$ be the input to the k -th KEYGEN oracle query of A_2 . The IND-security game for FH-IPBFE[NZIP-AC-IPBFRE] returns $\perp$ if there exists indices j, k such that

$$g(\langle \mathbf{x}_0^j, \mathbf{y}_0^k \rangle) \neq g(\langle \mathbf{x}_1^j, \mathbf{y}_1^k \rangle). \quad (4)$$

To answer the j -th ENC oracle query of A_2 , B_2 makes a ENC query for $\mathbf{x}_0^j, \mathbf{x}_1^j, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}$ and to answer the k -th KEYGEN oracle query of A_2 , B_2 makes a ENC query for $\mathbf{y}_0^k, \mathbf{y}_1^k, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}$.

From $a(\text{pp}, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}) = 1$ (by Lemma 2) and Eq. (4) it follows that B causes the IND-security game for NZIP-AC-IPBFRE to return $\perp$, too.

On the other hand, let $\mathbf{z}_0^j, \mathbf{z}_1^j, \mathbf{w}^j$ be the input of the j -th ENC query of B_2 . The IND-security game for NZIP-AC-IPBFRE returns $\perp$ if there exists indices j, k such that $a(\text{pp}, \mathbf{w}^j, \mathbf{w}^k) = 1$ and

$$g(\langle \mathbf{z}_0^j, \mathbf{z}_0^k \rangle) \neq g(\langle \mathbf{z}_1^j, \mathbf{z}_1^k \rangle). \quad (5)$$

The way we defined B_2 , $\mathbf{w}^j, \mathbf{w}^k \in \{\mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}\}$ must hold. This shows that the first condition can happen by Lemma 1 only if exactly one of $\mathbf{w}^j, \mathbf{w}^k$ equals $\mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}$ (and the other vector equals $\mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}$). Without loss of generality let $\mathbf{w}^j = \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}$ and $\mathbf{w}^k = \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}$. Then the j -th ENC query of B_2 was caused by an ENC($\mathbf{z}_0^j, \mathbf{z}_1^j$) for A_2 and the k -th ENC query of B_2 was caused by an KEYGEN($\mathbf{z}_0^k, \mathbf{z}_1^k$) for A_2 . However, these two queries cause by Eq. (5) that the IND-security game for FH-IPBFE[NZIP-AC-IPBFRE] returns $\perp$, too.

With this, we see

$$\Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}, B}^{\text{ind-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A}^{\text{ind-1}}(\lambda) \Rightarrow 1]$$

and

$$\Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}, B}^{\text{ind-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A}^{\text{ind-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

Theorem 12 (SIM-security). *If NZIP-AC-IPBFRE is SIM-secure, then FH-IPBFE[NZIP-AC-IPBFRE] is SIM-secure. Concretely, for every adversary A against SIM-security of FH-IPBFE[NZIP-AC-IPBFRE], there exists an adversary B against SIM-security of NZIP-AC-IPBFRE with roughly the same runtime as A such that for every simulator S_A there exists a simulator S_B where for all $\lambda \in \mathbb{N}$*

$$\text{Adv}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A, S_A}^{\text{sim}}(\lambda) \leq \text{Adv}_{\text{NZIP-AC-IPBFRE}, B, S_B}^{\text{sim}}(\lambda).$$

Proof. First, consider an adversary A against simulation-based security of FH-IPBFE[NZIP-AC-IPBFRE]. If such an adversary exists, then we can use it to construct an adversary B against simulation-based security of NZIP-AC-IPBFRE as given in Figure 21.

Let $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$ be the simulator for the SIM security of NZIP-AC-IPBFRE. We first construct a simulator $S_B = (S_{B,1}, S_{B,2}, S_{B,3}, S_{B,4})$ for the SIM security of FH-IPBFE[NZIP-AC-IPBFRE] as described by Fig. 22.

The simulator stores in $\mathcal{I}$ for each query the type (ENC or KEYGEN query), a global query index and a type-specific query index. From this, the simulator can construct the set $\mathcal{T}$ of tag vectors for the underlying NZIP-AC-IPBFRE scheme, since all ENC queries are answered with an NZIP-AC-IPBFRE ciphertext with $\mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}$ as tag vector and all KEYGEN queries are answered with an NZIP-AC-IPBFRE ciphertext with $\mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}$ as tag vector. Furthermore, for all pairs of vectors $(\mathbf{x}, \mathbf{y})$ where $\mathbf{x}$ was queried with ENC and $\mathbf{y}$ was queried with KEYGEN, the simulator gets in $\mathcal{C}'_{\text{ip}}$ the function evaluation $g(\langle \mathbf{x}, \mathbf{y} \rangle)$. It stores these in $\mathcal{C}_{\text{ip}}$, but switching from the query-type specific indexing used in the SIM security game for FH-IPBFE to the global indexing used in the SIM security game for NZIP-AC-IPBFRE. For pairs of vectors where both vectors are queried in ENC or both are queried in KEYGEN, the access policy is not satisfied by Lemma 1 and therefore the simulator does not add anything corresponding to these pairs to $\mathcal{C}_{\text{ip}}$.

With this, we see

$$\Pr[\mathbf{G}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A}^{\text{sim-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}, B}^{\text{sim-1}}(\lambda) \Rightarrow 1]$$

and

$$\Pr[\mathbf{G}_{\text{FH-IPBFE}[\text{NZIP-AC-IPBFRE}], A, S_A}^{\text{sim-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{\text{NZIP-AC-IPBFRE}, B, S_B}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

```

Adversary  $B = (B_1, B_2)$ 

 $B_1(1^\lambda, \text{pp})$ :
1 Obtain modulus  $m$  from  $\text{pp}$ 
2 if  $m$  is prime then
3   if  $m \equiv 1 \pmod{4}$  then
4      $(a_i)_{1 \leq i \leq \ell} := (a_1) \leftarrow \$ \text{Prime1Mod4SoS}(m)$ 
5   else
6      $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2) \leftarrow \$ \text{PrimeSoS}(m)$ 
7   else
8      $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2, a_3, a_4) \leftarrow \$ \text{LagrangeSoS}(m - 1)$ 
9    $(d, st) \leftarrow \$ A_1(1^\lambda, \text{pp})$ 
10  return  $(d, 2\ell - 1, st)$ 

 $B_2^{\text{ENCO}'(\cdot)}(st)$ :
11  $b' \leftarrow \$ A_2^{\text{ENCO}(\cdot), \text{KEYGENO}(\cdot)}(st)$ 
12 return  $b'$ 

 $\text{ENCO}(\mathbf{x})$ :
13  $\text{ct} \leftarrow \text{ENCO}'(\mathbf{x}, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}})$ 
14 return  $\text{ct}$ 

 $\text{KEYGENO}(\mathbf{y})$ :
15  $\text{sk} \leftarrow \text{ENCO}'(\mathbf{y}, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}})$ 
16 return  $\text{sk}$ 

```

Figure 21: Adversary against SIM-security of NZIP-AC-IPBFRE.

```

Simulator  $S_B = (S_{B,1}, S_{B,2}, S_{B,3}, S_{B,4})$ 

 $S_{B,1}(1^\lambda)$ :
1   $(pp, st_{S_A}) \leftarrow S_{A,1}(1^\lambda)$ 
2  return  $(pp, st_{S_B} \leftarrow (pp, st_{S_A}))$ 

 $S_{B,2}(1^d, st_{S_B} = (pp, st_{S_A}))$ :
3  Obtain modulus  $m$  from  $pp$ 
4  if  $m$  is prime then
5    if  $m \equiv 1 \pmod{4}$  then
6       $(a_i)_{1 \leq i \leq \ell} := (a_1) \leftarrow \text{Prime1Mod4SoS}(m)$ 
7    else
8       $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2) \leftarrow \text{PrimeSoS}(m)$ 
9    else
10      $(a_i)_{1 \leq i \leq \ell} := (a_1, a_2, a_3, a_4) \leftarrow \text{LagrangeSoS}(m - 1)$ 
11   $st_{S_A} \leftarrow S_{A,2}(1^d, 1^{2\ell-1} st_{S_A})$ 
12   $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0; \mathcal{I} \leftarrow \emptyset$ 
13   $st_{S_B} \leftarrow (st_{S_A}, (a_i)_{1 \leq i \leq \ell}, i, j, k, \mathcal{I})$ 
14  return  $st_{S_B}$ 

 $S_{B,3}(C'_{ip}, st_{S_B} = (st_{S_A}, (a_i)_{1 \leq i \leq \ell}, i, j, k, \mathcal{I}))$ :
15   $i \leftarrow i + 1; k \leftarrow k + 1$ 
16   $\mathcal{I} \leftarrow \mathcal{I} \cup \{(i, k, \text{Enc})\}$ 
17   $\mathcal{T} \leftarrow \{(k, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}) \mid \exists i' \in \mathbb{N} : (i', k, \text{Enc}) \in \mathcal{I}\}$ 
       $\cup \{(k, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}) \mid \exists j' \in \mathbb{N} : (j', k, \text{KeyGen}) \in \mathcal{I}\}$ 
18   $C_{ip} \leftarrow \{((\min\{k', k''\}, \max\{k', k''\}), c_{i',j'}) \mid \exists i', j' \in \mathbb{N} : (i', k', \text{Enc}) \in \mathcal{I}$ 
       $\wedge (j', k'', \text{KeyGen}) \in \mathcal{I} \wedge ((i', j'), c_{i',j'}) \in C'_{ip}\}$ 
19   $(ct, st_{S_A}) \leftarrow S_{A,3}(C_{ip}, \mathcal{T}, st_{S_A})$ 
20  return  $(ct, st_{S_B} = (st_{S_A}, (a_i)_{1 \leq i \leq \ell}, i, j, k, \mathcal{I}))$ 

 $S_{B,4}(C'_{ip}, st_{S_B} = (st_{S_A}, (a_i)_{1 \leq i \leq \ell}, i, j, k, \mathcal{I}))$ :
21   $j \leftarrow j + 1; k \leftarrow k + 1$ 
22   $\mathcal{I} \leftarrow \mathcal{I} \cup \{(j, k, \text{KeyGen})\}$ 
23   $\mathcal{T} \leftarrow \{(k, \mathbf{v}_{(a_i)_{1 \leq i \leq \ell}}) \mid \exists i' \in \mathbb{N} : (i', k, \text{Enc}) \in \mathcal{I}\}$ 
       $\cup \{(k, \mathbf{u}_{(a_i)_{1 \leq i \leq \ell}}) \mid \exists j' \in \mathbb{N} : (j', k, \text{KeyGen}) \in \mathcal{I}\}$ 
24   $C_{ip} \leftarrow \{((\min\{k', k''\}, \max\{k', k''\}), c_{i',j'}) \mid \exists i', j' \in \mathbb{N} : (i', k', \text{Enc}) \in \mathcal{I}$ 
       $\wedge (j', k'', \text{KeyGen}) \in \mathcal{I} \wedge ((i', j'), c_{i',j'}) \in C'_{ip}\}$ 
25   $(sk, st_{S_A}) \leftarrow S_{A,3}(C_{ip}, \mathcal{T}, st_{S_A})$ 
26  return  $(sk, st_{S_B} = (st_{S_A}, (a_i)_{1 \leq i \leq \ell}, i, j, k, \mathcal{I}))$ 

```

Figure 22: Simulator for SIM-security of FH-IPBFE[NZIP-AC-IPBFRE].

B TinyFHIPFE Security

Unlike Shoup's original model [Sho97], we adopt an equivalent variant of the generic bilinear group model also previously used by Kim et al. [KLM⁺18]. In the generic group model, access to group elements of the ciphertexts and function keys is provided via “handles” representing them. Thus to facilitate group operations, the adversary has access to generic bilinear group model oracles OP, PAIR, ZT which provide group operation, pairing operation, and zero testing respectively. This is in addition to the encryption and key generation oracles ENCO and KEYGENO provided in the simulation security games.

We will prove security by first presenting the security games and then a simulator in the generic bilinear group model. Then, we will argue that an adversary cannot distinguish between the games given that our simulator is used in the **sim-0** game given in Fig. 24. To make this argument, we must analyze how the simulator answers zero-test queries and bound the difference in the outputs by using Schwartz-Zippel lemma.

In Figure 23 and Figure 24, we present the security games for simulation-based security of TinyFHIPFE in the generic bilinear group model. In the security games, L_1, L_2, L_t are lists of pairs, with operation **append**. For such a list L , we denote by $(x, y) \in L$ that (x, y) appears in L , and we denote by $y \in L$ that there exists x such that $(x, y) \in L$. Before describing the security games, we introduce the formal variables used in the security games and the simulator below.

Let q be the total number of encryption and keygen queries made by an adversary $\mathcal{A}$. Then, we define two sets of formal variables as follows.

$$\begin{aligned} \mathcal{T} &= \{x_k^i, y_k^i\}_{i \in [q], k \in [d]} \cup \{s_1^i, s_2^i\}_{i \in [q]} \cup \\ &\quad \{a_{1,k}, a_{2,k}\}_{k \in [d]} \cup \{b_{m,n}^i, b_{m,n}^{*i}\}_{m,n \in [4]} \cup \{v_1^i, v_2^i, w_1^i, w_2^i\}_{i \in [q]} \\ \mathcal{R} &= \{\phi_k^i\}_{i \in [q], k \in [d]} \cup \{\gamma_k^i\}_{i \in [q], k \in [d]} \cup \{\mu_m^i\}_{i \in [q], m \in [4]} \cup \{\psi_m^i\}_{i \in [q], m \in [4]} \end{aligned}$$

These variables model the following components of the scheme:

- x_k^i : k^{th} component of the i^{th} query input vector $\mathbf{x}_i$ when made to **KeyGen**.
- y_k^i : k^{th} component of the i^{th} query input vector $\mathbf{y}_i$ when made to **Enc**.
- s_1^i, s_2^i : components of the uniform random vector $\mathbf{s}_i$ sampled during **Enc**($\mathbf{y}^i$) or **KeyGen**($\mathbf{x}^i$).
- $a_{1,k}, a_{2,k}$: components of the k^{th} column of the secret key uniform random matrix $\mathbf{A}$ sampled during **Setup**.
- $b_{m,n}, b_{m,n}^*$: $(m, n)^{th}$ components of the secret key matrices $\mathbf{B}, \mathbf{B}^*$ sampled during **Setup**.
- v_1^i, v_2^i : components of the vector $\mathbf{A}\mathbf{y}^i$ calculated inside **Enc**($\mathbf{y}^i$).
- w_1^i, w_2^i : components of the vector $\mathbf{A}\mathbf{x}^i + \mathbf{A}\mathbf{A}^T\mathbf{s}_i$ calculated inside **KeyGen**($\mathbf{x}^i$).
- ϕ_k^i : k^{th} component of the vector $\mathbf{A}^T\mathbf{s}_i + \mathbf{x}^i$ calculated inside **KeyGen**($\mathbf{x}^i$). ϕ^i refers to the vector of ϕ_k^i for $k \in [d]$.
- γ_k^i : k^{th} component of the vector $\mathbf{A}^T\mathbf{s}_i + \mathbf{y}^i$ calculated inside **Enc**($\mathbf{y}^i$). γ^i refers to the vector of γ_k^i for $k \in [d]$.

- μ_m^i : m^{th} component of the vector $\mathbf{B} \cdot (\mathbf{s}_i \| \mathbf{A}\mathbf{x}^i + \mathbf{A}\mathbf{A}^T \mathbf{s}_i)$ calculated inside $\text{KeyGen}(\mathbf{x}_i)$.
- ψ_m^i : m^{th} component of the vector $\mathbf{B}^* \cdot (\mathbf{A}\mathbf{y}^i \| \mathbf{s}_i)$ calculated inside $\text{Enc}(\mathbf{y}_i)$.

Note: Given that $\mathbf{B}^* = (\mathbf{B}^{-1})^T$, $b_{m,n}^*$ can be specified as follows:

$$b_{m,n}^* = (-1)^{(m+n)} \det(\mathbf{B}_{m,n})$$

Here, $\mathbf{B}_{m,n}$ is the 3×3 matrix formed by removing the m^{th} row and n^{th} column of the matrix $\mathbf{B}$. Next, the determinant of a 3×3 matrix $\mathbf{C}$ is given by the following formula:

$$\det(\mathbf{C}) = \sum_{\sigma \in S_3} \epsilon(\sigma) \cdot c_{1,\sigma(1)} \cdot c_{2,\sigma(2)} \cdot c_{3,\sigma(3)}$$

Here, S_3 is all permutations of $\{1, 2, 3\}$, $\epsilon(\sigma)$ gives the sign of the corresponding term (1 if even permutation and -1 if odd permutation). Finally, $b_{m,n}^*$ can be written in terms of $\{b_{m,n}\}_{m,n \in [d]}$ as follows:

$$b_{m,n}^* = \sum_{\sigma \in S'_3} \text{sgn}(\sigma, m, n) \cdot b_{m_1, \sigma(m_1)} \cdot b_{m_2, \sigma(m_2)} \cdot b_{m_3, \sigma(m_3)}$$

Here, $m_1, m_2, m_3 \neq m$ and $\in [4]$ and S'_3 is all bijections from m_1, m_2, m_3 to $m_4, m_5, m_6 \neq n$ and $\in [4]$. We collapse the sign of the monomial into a function sgn for which the explicit specification will be not be necessary for the proof.

Theorem 13. *TinyFHIPFE is simulation-secure in the generic bilinear group model. In particular, for every adversary $\mathbf{A}$ that makes q_t total queries to its oracles, there exists a simulator $\mathbf{S}$ such that*

$$\text{Adv}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-ggm}}(\lambda) \leq \frac{(4q_t(d+4))^2}{p}.$$

Now, we analyze the advantage of the adversary which is given by the following equation:

$$\text{Adv}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-ggm}}(\lambda) = \Pr[\mathbf{G}_{\text{IPFE}, \mathbf{A}}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-0}}(\lambda) \Rightarrow 1] \quad (6)$$

First, in Figure 25, we provide a simulator for the game described above. Due to space constraints in the figure, we describe part of one of the simulator's algorithm $\mathbf{S}_6$ in text below.

The polynomial decomposition used by $\mathbf{S}_6$ is as follows:

$$p = \sum_{i,j \in [q]} \left(c_{i,j} \left(\left(\sum_{k \in [d]} \phi_k^i \gamma_k^j - \sum_{m \in [4]} \mu_m^i \psi_m^j \right) - z_{i,j} \right) + f^{i,j} \right) \quad (7)$$

where $z_{i,j}$ is the inner product value $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$, $c_{i,j}$ is the scalar coefficient of the first term of $\mu_m^i \psi_m^j$ which is $\mu_1^i \psi_1^j$ ($c_{i,j}$ could be 0), and $f^{i,j}$ consists of all remaining terms in terms of $\{\phi_k^i, \gamma_k^j, \mu_m^i, \psi_m^j\}$ for $k \in [d], m \in [4]$.

1. If $f^{i,j}$ consists of any terms, then return "false".
2. Else return "true".

Lemma 3. *If an adversary interacting with the two games $\mathbf{G}_{\text{IPFE}, \mathbf{A}}^{\text{sim-1}}(\lambda)$ and $\mathbf{G}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-0}}(\lambda)$ is only provided access to ENCO, KEYGENOOP, PAIR oracles (and not ZT), then it cannot distinguish between the two games. Formally, consider the modified games $\mathbf{G}_{\text{IPFE}, \mathbf{A}}^{\text{sim-1}}(\lambda)$ and $\mathbf{G}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-0}}(\lambda)$ where the oracle ZT is omitted. Then, for any PT adversary $\mathbf{A}$ and any PT simulator $\mathbf{S}$, we have that*

$$\text{Adv}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-ggm}}(\lambda) = \Pr[\mathbf{G}_{\text{IPFE}, \mathbf{A}}^{\text{sim-1}}(\lambda) \Rightarrow 1] - \Pr[\mathbf{G}_{\text{IPFE}, \mathbf{A}, \mathbf{S}}^{\text{sim-0}}(\lambda) \Rightarrow 1] = 0$$

Game $\mathbf{G}_{\text{IPFE}, \mathcal{A}}^{\text{sim-1}}(\lambda)$	
MAIN: 1 $L_1, L_2, L_t \leftarrow \emptyset$ // Initialize lists 2 $H \leftarrow \emptyset$ 3 $(p, g_1, g_2, g_t, \mathbf{G}_1, \mathbf{G}_2, \mathbf{G}_t, \mathbf{e}) \leftarrow \mathcal{G}_{\text{asym}}(1^\lambda)$ 4 $h_1, h_2, h_t \leftarrow \text{NEWHANDLE}(1^\lambda)$ 5 $L_1.\text{append}(g_1, h_1)$ 6 $L_2.\text{append}(g_2, h_2)$ 7 $L_t.\text{append}(g_t, h_t)$ 8 $\text{pp} \leftarrow (p, h_1, h_2, h_t)$ 9 $(d, st_A) \leftarrow \Lambda_1^{\text{OP}(\cdot, \cdot, \cdot), \text{PAIR}(\cdot, \cdot), \text{ZT}(\cdot, \cdot)}(\text{pp})$ 10 $\mathbf{B} \leftarrow \text{SL}_4(\mathbb{Z}_p)$ 11 $\mathbf{B}^* \leftarrow (\mathbf{B}^{-1})^T$ 12 $\mathbf{A} \leftarrow \mathbb{Z}_p^{2 \times d}$ 13 $\text{msk} \leftarrow (\mathbf{A}, \mathbf{B}, \mathbf{B}^*)$ 14 $b \leftarrow \Lambda_2^{\text{ENCO}(\cdot), \text{KEYGENO}(\cdot), \text{OP}(\cdot, \cdot, \cdot), \text{PAIR}(\cdot, \cdot), \text{ZT}(\cdot, \cdot)}(st_A)$ 15 return b KEYGENO(x): 16 $\mathbf{s} \leftarrow \mathbb{Z}_p^2$ 17 $\text{ct}_1 \leftarrow \mathbf{A}\mathbf{s} + \mathbf{x}$ 18 $\text{ct}_2 \leftarrow \mathbf{B} \cdot (\mathbf{s} \parallel \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{A}^T \mathbf{s})$ 19 $\text{ADDLIST}(L_1, \mathbf{s})$ 20 $\mathbf{h}_1 \leftarrow \text{ADDLIST}(L_1, \text{ct}_1)$ 21 $\mathbf{h}_2 \leftarrow \text{ADDLIST}(L_1, \text{ct}_2)$ 22 $\text{ct}_x \leftarrow (\mathbf{h}_1, \mathbf{h}_2)$ 23 return sk_x ENCO(y): 24 $\mathbf{s} \leftarrow \mathbb{Z}_p^2$ 25 $\text{sk}_1 \leftarrow \mathbf{A}\mathbf{s} + \mathbf{y}$ 26 $\text{sk}_2 \leftarrow \mathbf{B}^* \cdot (\mathbf{A}\mathbf{y} \parallel \mathbf{s})$ 27 $\text{ADDLIST}(L_2, \mathbf{s})$ 28 $\mathbf{h}_1 \leftarrow \text{ADDLIST}(L_2, \text{ct}_1)$ 29 $\mathbf{h}_2 \leftarrow \text{ADDLIST}(L_2, \text{ct}_2)$ 30 $\text{ct}_x \leftarrow (\mathbf{h}_1, \mathbf{h}_2)$ 31 return ct_x	OP(h_1, h_2, b): 32 if $b = 1$ then $L \leftarrow L_1$ 33 if $b = 2$ then $L \leftarrow L_2$ 34 else $L \leftarrow L_t$ 35 Find x_1 s.t. $(x_1, h_1) \in L$ 36 Find x_2 s.t. $(x_2, h_2) \in L$ 37 $x_3 \leftarrow x_1 + x_2$ 38 $h_3 \leftarrow \{0, 1\}^\lambda$ 39 $L.\text{append}(x_3, h_3)$ 40 return (h_3) PAIR(h_1, h_2): 41 Find x_1 s.t. $(x_1, h_1) \in L_1$ 42 Find x_2 s.t. $(x_2, h_2) \in L_2$ 43 $x_3 \leftarrow x_1 \cdot x_2$ 44 $h_3 \leftarrow \{0, 1\}^\lambda$ 45 $L_t.\text{append}(x_3, h_3)$ 46 return (h_3) ZT(h, b): 47 if $b = 1$ then $L \leftarrow L_1$ 48 if $b = 2$ then $L \leftarrow L_2$ 49 else $L \leftarrow L_t$ 50 Find x s.t. $(x, h) \in L$ 51 return $(x = 0)$ ADDLIST($L, \mathbf{x}$): 52 $k \leftarrow \mathbf{x} $ 53 for $i \in [k]$ do 54 $h_i \leftarrow \text{NEWHANDLE}(1^\lambda)$ 55 $L.\text{append}(\mathbf{x}[i], h_i)$ 56 return $(h_1, \dots, h_k)$ NEWHANDLE(1^λ): 57 $h \leftarrow \{0, 1\}^\lambda \setminus H$ 58 $H \leftarrow H \cup \{h\}$ 59 return h

Figure 23: Game defining simulation security of TinyFHIPFE in the generic group model.

Proof. By inspection of the oracles, we notice that the return value is a vector of appropriate size consisting of handles. Each of these handles is sampled independently and uniformly randomly in both games. Thus, given information-theoretically indistinguishable inputs, the adversary has exactly the same probability of returning 1 in each game. Thus, the probability of the game outputting 1 is also identical between the two games, and the advantage of the adversary is equal to 0. $\square$

Now, it suffices to consider the difference in The rest of the proof refers to the zero test oracle and analyzes return values in each game. In $\mathbf{G}_{\text{IPFE}, \mathcal{A}}^{\text{sim-1}}(\lambda)$, the zero test returns true when the value stored in zero. Thus, we analyze the polynomial decomposition in equation (7) that corresponds to $\mathbf{G}_{\text{IPFE}, \mathcal{A}}^{\text{sim-0}}(\lambda)$.

First, if the polynomial p is the identically zero polynomial, the simulator correctly responds with "zero". Next, the handles received by $\mathcal{A}$ correspond to ciphertexts which are

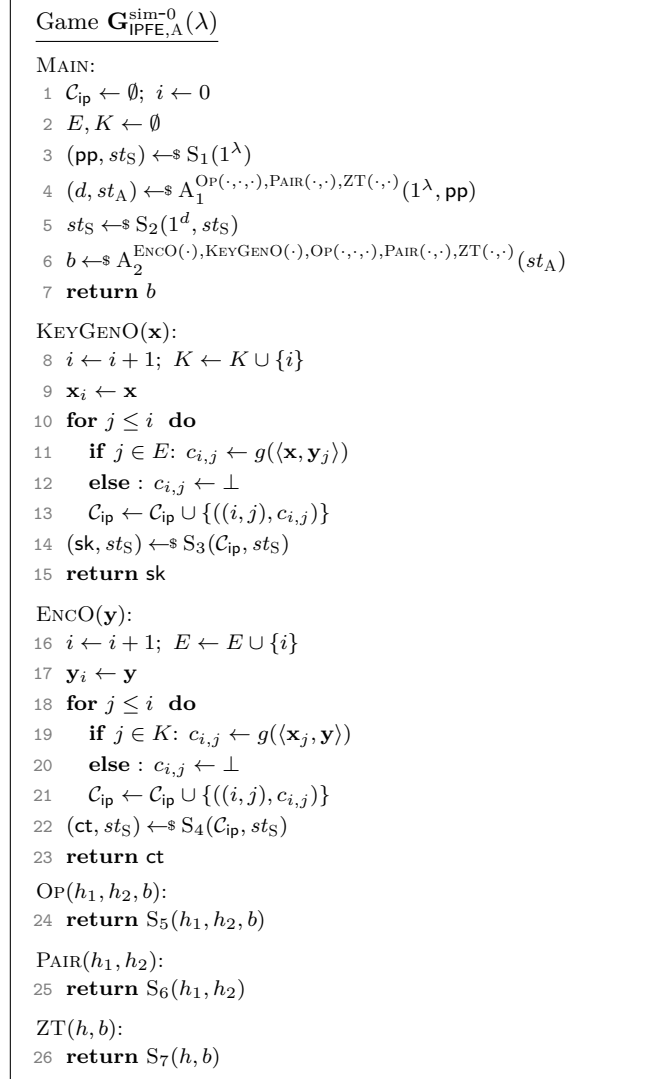


Figure 24: Game defining simulation security of TinyFHIPFE in the generic group model.

stored in the lists L_1, L_2 with the following components from the set $\mathcal{R}$ for each $i \in [q]$:

$$\phi_1^i, \dots, \phi_d^i, \gamma_1^i, \dots, \gamma_d^i, \mu_1^i, \mu_2^i, \mu_3^i, \mu_4^i, \psi_1^i, \psi_2^i, \psi_3^i, \psi_4^i$$

Lemma 4. *For all $i \in [q]$, each of $v_1^i, v_2^i, w_1^i, w_2^i$ is not the identically zero polynomial.*

Proof. First, consider v_1^i, v_2^i which represent components of the vector $\mathbf{A}\mathbf{y}_i$. In terms of formal variables, $v_1^i = \sum_{k \in [d]} a_{1,k} y_k^i$ and $v_2^i = \sum_{k \in [d]} a_{2,k} y_k^i$. Here, $a_{1,k}, a_{2,k}$ are formal variables for all $k \in [d]$. Thus, v_1^i, v_2^i are not identically zero.

Next, we look at w_1^i, w_2^i which represent components of the vector $\mathbf{A}\mathbf{x}_i + \mathbf{A}\mathbf{A}^T \mathbf{s}_i$. In terms of formal variables,

$$w_1^i = \sum_{k \in [d]} a_{1,k} x_k^i + \left(\sum_{k \in [d]} (a_{1,k})^2 \right) s_1^i + \left(\sum_{k \in [d]} (a_{1,k} a_{2,k}) \right) s_2^i$$

Simulator $S_{\text{IPFE}, \mathcal{A}}^{\text{sim-1}}(\lambda)$ S_1: 1 $\mathcal{C}_{\text{ip}} \leftarrow \emptyset$; $i \leftarrow 0$ 2 $L_1, L_2, L_t \leftarrow \emptyset$ 3 $(p, g_1, g_2, g_t, G_1, G_2, G_t, e) \leftarrow \mathcal{G}_{\text{asym}}(1^\lambda)$ 4 $g_1, g_2, g_t \leftarrow \text{NEWHANDLE}(1^\lambda)$ 5 $\text{pp} \leftarrow (g_1, g_2, g_t, p)$ 6 $st_S \leftarrow (L_1, L_2, L_t, \mathcal{C}_{\text{ip}})$ 7 Return (pp, st_S) $S_2(1^d, st_S)$: 8 Return st_S $S_3(\mathcal{C}_{\text{ip}}, st_S)$: // Key Generation 9 $i \leftarrow i + 1$ 10 $\text{ADDLIST}(L_1, \phi^i)$ 11 $\text{ADDLIST}(L_1, \psi^i)$ 12 $\text{sk}_x \leftarrow (\phi^i, \mu^i)$ 13 return sk_x $S_4(\mathcal{C}_{\text{ip}}, st_S)$: // Encryption 14 $i \leftarrow i + 1$ 15 $\text{ADDLIST}(L_2, \gamma^i)$ 16 $\text{ADDLIST}(L_2, \psi^i)$ 17 $\text{ct}_y \leftarrow (\gamma^i, \psi^i)$ 18 return ct_y $S_5(h_1, h_2, b)$: // Group Operation 19 if $b = 1$ then $L \leftarrow L_1$ 20 if $b = 2$ then $L \leftarrow L_2$ 21 else $L \leftarrow L_t$ 22 Find x_1 s.t. $(x_1, h_1) \in L$ 23 Find x_2 s.t. $(x_2, h_2) \in L$ 24 $x_3 \leftarrow x_1 + x_2$ 25 $h_3 \leftarrow \{0, 1\}^\lambda$ 26 $L.\text{append}(x_3, h_3)$ 27 return (h_3)	$S_6(h_1, h_2)$: // Pairing Operation 28 Find x_1 s.t. $(x_1, h_1) \in L_1$ 29 Find x_2 s.t. $(x_2, h_2) \in L_2$ 30 $x_3 \leftarrow x_1 \cdot x_2$ 31 $h_3 \leftarrow \{0, 1\}^\lambda$ 32 $L_t.\text{append}(x_3, h_3)$ 33 return (h_3) $S_7(h, b)$: // Zero Test 34 if $b = 1$ or $b = 2$ then 35 Find x s.t. $(x, h) \in L_b$ 36 if $x = 0$ then return true 37 else return false 38 if $b = 0$ then 39 Find x s.t. $(x, h) \in L_t$ 40 if $x = 0$ then return true 41 else return // according to equation 7 $\text{ADDLIST}(L, \mathbf{x})$: // $\mathbf{x}$ is a vector of polynomials 42 $k \leftarrow \mathbf{x}$ 43 for $i \in [k]$ do 44 $h_i \leftarrow \text{NEWHANDLE}(1^\lambda)$ 45 $L.\text{append}(\mathbf{x}[i], h_i)$ 46 return $(h_1, \dots, h_k)$ $\text{NEWHANDLE}(1^\lambda)$: 47 $h \leftarrow \{0, 1\}^\lambda \setminus H$ 48 $H \leftarrow H \cup \{h\}$ 49 return h
---	--

Figure 25: Simulator for simulation security of TinyFHIPFE in the generic group model.

Here all of $s_1^i, s_2^i, a_{1,k}, a_{2,k}$ are indeterminate formal variables, thus each of the summation terms are not identically zero. Hence, the polynomial is not identically zero. $\square$

Next, if the handle h belongs to one of the source groups, the polynomial p must be some linear combination of variables in $\mathcal{R}$. In particular, each of $\phi_k^i, \gamma_k^i, \mu_m^i, \psi_m^i$ for all $i \in [q], k \in [d], m \in [4]$ consist of monomials which are a product of either two indeterminates or an indeterminate and one of $v_1^i, v_2^i, w_1^i, w_2^i$ which are not identically zero as given by Lemma 1. Thus, any linear combination of these polynomials cannot be the identically zero polynomial. The simulator correctly answers the zero-test query when the input handle is in the source group.

On the other hand, analysis of an input handle in the target group is more involved. We consider all the possible pairing for the handles that $\mathcal{A}$ has access to.

In the target group, we know that $f^{i,j}$ consists of a linear combination of all possible pairings. First, we expand each of the pairing to show that they are not the identically zero polynomial. Next, we show that any leftover combination in $f^{i,j}$ can also not be

identically zero.

$$\begin{aligned}
\phi_k^i \cdot \gamma_\ell^j &= (a_{1,k} s_1^i + a_{2,k} s_2^i + x_k^i) \cdot (a_{1,\ell} s_1^j + a_{2,\ell} s_2^j + y_\ell^j) \\
\phi_k^i \cdot \psi_n^j &= (a_{1,k} s_1^i + a_{2,k} s_2^i + x_k^i) \cdot \\
&\quad (b_{n,1}^* v_2^j + b_{n,2}^* v_2^j + b_{n,3}^* s_1^j + b_{n,4}^* s_2^j) \\
\mu_m^i \cdot \gamma_\ell^j &= (b_{m,1} s_1^i + b_{m,2} s_2^i + b_{m,3} w_1^i + b_{m,4} w_2^i) \cdot \\
&\quad (a_{1,\ell} s_1^j + a_{2,\ell} s_2^j + y_\ell^j) \\
\mu_m^i \cdot \psi_n^j &= (b_{m,1} s_1^i + b_{m,2} s_2^i + b_{m,3} w_1^i + b_{m,4} w_2^i) \cdot \\
&\quad (b_{n,1}^* v_1^j + b_{n,2}^* v_2^j + b_{n,3}^* s_1^j + b_{n,4}^* s_2^j)
\end{aligned}$$

For further evaluation, we include the degree of all monomials in each handle and subsequent pairings with respect to the formal variables $\{s, a, b\}$ which can be derived from the polynomial expressions above.

- ϕ_k^i : degree 2
- μ_m^i : degree 2 and degree 4
- γ_k^i : degree 2
- ψ_m^i : degree 4
- $\phi_k^i \cdot \gamma_\ell^j$: degree 4
- $\phi_k^i \cdot \psi_n^j$: degree 6
- $\mu_m^i \cdot \gamma_\ell^j$: degree 4 and 6
- $\mu_m^i \cdot \psi_n^j$: degree 6 and 8

Case I: $f^{i,j}$ consists of terms from a pairing of the form $\mu_m^i \psi_n^j$ without loss of generality in the indices i and j .

First, notice that the degree of monomials coming from this pairing have degree either 6 or 8 in the formal variables, in particular all monomials have degree 4 in variables $\{b_{m,n}\}_{m,n \in [4]}$ since each monomial consists of the product $b_{m,n} \cdot b_{m',n'}^*$ for some indices $m, n, m', n' \in [4]$. However, in Case II we will see that other monomials of degree 6 or 8 do not have degree 4 in $\{b_{m,n}\}_{m,n \in [4]}$ and thus cannot be combined with monomials in this pairing to form the identically zero polynomial. Thus, it suffices to partition the cases into this one where there are pairings of the $\mu_m^i \psi_n^j$ (and potentially other pairings) in $f^{i,j}$ and case II with no pairings of the $\mu_m^i \psi_n^j$.

Next, we analyze the monomials within the pairing and argue that they cannot be combined to form the identically zero polynomial either. Crucial to this analysis is the fact that $f^{i,j}$ does not include the pairings $\psi_1^i \mu_1^j$ and $\psi_1^j \mu_1^i$ since the first part of the polynomial p includes all instances of them by factoring out its coefficient $c_{i,j}$. This fact leads to the argument that $f^{i,j}$ cannot include the entire sum $\gamma^{i,j}$ which would "cancel out" b and b^* variables. Below, we show a detailed proof of this argument.

Here, consider the expansion of $\mu_m^i \psi_n^j$. For simplicity let $y^i = (s_1^i, s_2^i, w_1^i, w_2^i)$ and $z^j = (v_1^j, v_2^j, s_1^j, s_2^j)$. We have:

$$\begin{aligned} \mu_m^i \psi_n^j &= \left(\sum_{k_1 \in [4]} b_{m,k_1} y_{k_1}^i \right) \left(\sum_{k_2 \in [4]} b_{n,k_2}^* z_{k_2}^j \right) \\ &= \sum_{k_1, k_2 \in [4]} \left(b_{n,k_2} b_{m,k_1}^* y_{k_1}^i z_{k_2}^j \right) \end{aligned}$$

As given before, b_{m,k_1}^* can be written as follows:

$$b_{m,k_1}^* = \sum_{\sigma \in S'_3} \text{sgn}(\sigma, m, k_1) \cdot b_{m_1, \sigma(m_1)} b_{m_2, \sigma(m_2)} b_{m_3, \sigma(m_3)}$$

Here, S'_3 consists of bijections from $m_1, m_2, m_3 \in [4]$ and $m_1, m_2, m_3 \neq m$ to $m_4, m_5, m_6 \in [4]$ and $m_4, m_5, m_6 \neq k_1$ and $\text{sgn}(\sigma)$ gives the sign of the corresponding term.

Sub-case (a): $f^{i,j}$ consists of terms from a pairing $\mu_m^i \psi_n^j$ where $m \neq n$.

In the expansion above, each monomial inside the sums consists of four b variables, one from μ_n^i and the other 3 from the product given by the expansion of b^* variable in ψ_n^j . Note that the monomial does not consist of $b_{m,k'}$ for any $k' \in [4]$. However, for every $m' \in [4], m' \neq m$, the analogous monomial must consist of $b_{m, \sigma(m)}$ in the inside product. Thus, these monomials cannot cancel out any monomials from $\psi_{m'}^i \mu_n^j$ where $m' \neq n$. Next, when $m \neq n$, $b_{n, \sigma(n)}$ is part of b_{m, k_1}^* and thus the monomial consists of two variables from the same row n , namely b_{n, k_2} and $b_{n, \sigma(n)}$. On the other hand, if $m = n$, each monomial consists of exactly one variable from each row of the matrix. Thus, monomials from $\mu_m^i \psi_n^j$ when $m \neq n$ cannot be canceled out by monomials in the case $m = n$. Thus, in this case $f^{i,j}$ cannot be identically zero.

Sub-case (b): $f^{i,j}$ does not consist of terms from a pairing $\mu_m^i \psi_n^j$ where $m \neq n$.

First, recall that $\mu_m^i \psi_n^j$ cannot be in $f^{i,j}$ by construction, thus we consider $\mu_m^i \psi_n^j$ where $m \in \{2, 3, 4\}$. Among all the monomials in the sum, there exists a monomial that contains both b_{m, k_2} and b_{1, k_2} . However, there is no monomial that contains both b_{m', k_2} and b_{1, k_2} for any $m' \in [4], m' \neq m$. Thus, each pairing contains a monomial with both b_{m, k_2} and b_{1, k_2} in the product which cannot be canceled by any other pairing. Thus, $f^{i,j}$ cannot be identically zero in this case either.

Notice that the above argument does not apply when $k_1 = k_2$ since $\sigma(r) \neq k_1$. In that case, each monomial consists of $b_{m, k}$ but cannot contain $b_{m', k}$ for any $m' \in [4], m' \neq m$. This applies to all $m \in [4]$ which means that monomials from pairings $m' \neq m$ contain $b_{m', k}$ but not $b_{m, k}$. Thus, these two types of monomials cannot cancel each other out and thus $f^{i,j}$ is not identically zero.

Case II: $f^{i,j}$ does not consist of terms from a pairing of the form $\mu_m^i \psi_n^j$.

Here, it suffices to analyze the monomials that have the same degree since monomials that have a different degree in the formal variables cannot cancel each other out in any linear combinations to form the identically zero polynomial.

First, consider all monomials of degree 4. There are two sources for these monomials: $\phi_k^i \cdot \gamma_\ell^j$ and $\mu_m^i \cdot \gamma_n^j$. Below in Table 4 for each pairing, we give an example of monomial of

Table 4: Degree 4 monomials.

monomial origin	$\deg(s)$	$\deg(a)$	$\deg(b)$	example
$\phi_k^i \cdot \gamma_\ell^j$	2	2	0	$a_{1,k} s_1^i a_{1,\ell} s_1^j$
$\mu_m^i \cdot \gamma_n^j$	2	1	1	$b_{m,1} s_1^i a_{1,\ell} s_1^j$
$\mu_m^i \cdot \gamma_n^j$	1	2	1	$b_{m,1} w_1^i a_{1,\ell} s_1^j$

degree 4 in the polynomial expansion. The other monomials differ only in indices and we provide the total number of such monomials.

Similarly, consider monomials of degree 6. There are two sources for these monomials: $\phi_k^i \cdot \psi_m^j$ and $\mu_m^i \cdot \gamma_n^j$. Below in Table 5 for each pairing, we give an example of monomial of degree 6 in the polynomial expansion. The other monomials differ only in indices.

Table 5: Degree 6 monomials.

monomial origin	$\deg(s)$	$\deg(a)$	$\deg(b)$	example
$\phi_k^i \cdot \psi_m^j$	1	2	3	$a_{1,k} s_1^i b_{m,1}^* v_1^j$
$\phi_k^i \cdot \psi_m^j$	2	1	3	$a_{1,k} s_1^i b_{m,1}^* s_1^j$
$\mu_m^i \cdot \gamma_\ell^j$	2	3	1	$b_{m,3} w_1^i a_{1,\ell} s_1^j$

In all the cases above, we can see that even though the monomials are of the same degree, the specific set of formal variables forming the polynomials are different for each type of monomial. Thus, there can be no relationship between these monomials and they cannot be combined to get the identically zero polynomial.

Finally, in Case II without pairings of the form $\psi_m^i \mu_n^j$, the polynomial $f^{i,j}$ cannot be the identically zero polynomial.

On the other hand, when $f^{i,j}$ is empty, we can simplify p as follows:

$$p = \sum_{i,j \in [q]} c_{i,j} \left(\left(\sum_{k \in [d]} \phi_k^i \gamma_k^j - \sum_{m \in [4]} \mu_m^i \psi_m^j \right) - z_{i,j} \right)$$

Then using the expansion of each polynomial above and the relationship between $\mathbf{B}$ and $\mathbf{B}^*$:

$$p = \sum_{i,j \in [q]} c_{i,j} \left(\left(\sum_{k \in [d]} x_k^i y_k^j \right) - z_{i,j} \right)$$

Now, the simulator knows that $z_{i,j}$ is the inner product of query vectors i and j , thus:

$$p = \sum_{i,j \in [q]} c_{i,j} (z_{i,j} - z_{i,j}) = 0$$

Thus, the simulator answers correctly for an honest evaluation.

Lemma 5. *Schwartz-Zippel Lemma: For a non-zero polynomial $P \in R[x_1, \dots, x_n]$ of total degree $d \geq 0$ over an integral domain R . Let S be a finite subset of R and let $r_1, \dots, r_n$ be selected at random independently and uniformly from S . Then*

$$\Pr[P(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|}.$$

Let q_t be the total number of queries made by the adversary $A = (A_1, A_2)$ to all of its oracles. We can split q into number of queries made to various oracles as follows:

1. Encryption Oracle $\text{ENCO}(\cdot)$: q_{enc}
2. Key Generation Oracle $\text{ENCO}(\cdot)$: q_{kg}
3. Group Operation Oracle $\text{OP}(\cdot, \cdot, \cdot)$: q_{op}
 - Group Operation Oracle in the first source group $\text{OP}(\cdot, \cdot, 1)$: $q_{op,1}$
 - Group Operation Oracle in the second source group $\text{OP}(\cdot, \cdot, 2)$: $q_{op,2}$
 - Group Operation Oracle in the target group $\text{OP}(\cdot, \cdot, 0)$: $q_{op,t}$
4. Bilinear Pairing Oracle $\text{PAIR}(\cdot, \cdot)$: q_{pair}
5. Zero Test Oracle $\text{OP}(\cdot, \cdot, \cdot)$: q_{zt}
 - Zero Test Oracle in the first source group $\text{ZT}(\cdot, 1)$: $q_{zt,1}$
 - Zero Test Oracle in the second source group $\text{ZT}(\cdot, 2)$: $q_{zt,2}$
 - Zero Test Oracle in the target group $\text{ZT}(\cdot, 0)$: $q_{zt,t}$

First, each Group Operation and Bilinear Pairing Oracle query returns one handle while an Encryption and Key Generation Oracle query returns $d + 4$ handles.

Now in each source group, we have the maximum possible degree of a polynomial is 4. Thus, by the Schwartz-Zippel Lemma, for each Zero Test Oracle query A makes, it has probability at most $\frac{4}{p}$ of the oracle returning **true** in sim-1 where it would not in the sim-0 game.

Similarly, in order to test linear combinations of polynomials, A takes each handle h and for every other handle h_i also in the same source group, runs the following queries: $\text{ZT}(\text{OP}(h, h_i, 1))$ or $\text{ZT}(\text{OP}(h, h_i, 2))$. Again, by Schwartz-Zippel Lemma, there is probability $\frac{4}{p}$ of the oracle returning **true** in sim-1 game and **false** in sim-0.

Then, we take the union bound on the probabilities above to bound the probability that at least one Zero Test Oracle query returns **true** as follows:

$$\frac{4}{p} \left(q_{zt,1} + q_{zt,2} + \binom{q_{kg}(d+4) + q_{op,1}}{2} + \binom{q_{enc}(d+4) + q_{op,2}}{2} \right).$$

Next, in the target group, we have the maximum possible degree of a polynomial is 8. First, for each Zero Test Oracle query A makes, it has probability at most $\frac{8}{p}$ of the oracle returning **true** in sim-1 where it would not in the sim-0 game.

In order to test linear combinations of polynomials, A again takes each handle h and for every other handle h_i also in the target group, runs the following queries: $\text{ZT}(\text{OP}(h, h_i, 0))$. Again, by Schwartz-Zippel Lemma, there is probability $\frac{8}{p}$ of the oracle returning **true** in sim-1 game and **false** in sim-0.

Then, we take union bound on these probabilities:

$$\frac{8}{p} \left(q_{zt,t} + \binom{q_{op,t} + q_{pair}}{2} \right).$$

Now, let's add the above probabilities together to calculate the probability of distinguishing games sim-1 and sim-0 as follows:

$$\begin{aligned}
& \frac{4}{p} \left(q_{zt,1} + q_{zt,2} + \binom{q_{kg}(d+4) + q_{op,1}}{2} + \binom{q_{enc}(d+4) + q_{op,2}}{2} \right) \\
& + \frac{8}{p} \left(q_{zt,t} + \binom{q_{op,t} + q_{pair}}{2} \right) \\
& = \frac{1}{p} \left(4q_{zt,1} + 4q_{zt,2} + 4 \binom{q_{kg}(d+4) + q_{op,1}}{2} + 4 \binom{q_{enc}(d+4) + q_{op,2}}{2} \right) \\
& + 8q_{zt,t} + 8 \binom{q_{op,t} + q_{pair}}{2} \\
& \leq \frac{1}{p} (8q_{zt} + 4((q_{op} + q_{pair} + (d+4)(q_{enc} + q_{kg}))^2)) \\
& \leq \frac{(4q_t(d+4))^2}{p} \quad \left(\leq \frac{\mathcal{O}(q_t^2)}{p} \right) .
\end{aligned}$$

This concludes the proof. $\blacksquare$

C Security Proofs of Outsourced Database Protocol for ANN Search

In this Section, we give the deferred security proofs of our AC-IPFRE[FH-IPFE] and ANN-ODB[AC-IPFRE] introduced in Section 6.

C.1 ACIPFRE[FHIPFE] Security

Theorem 14. *Our AC-IPFRE[FH-IPFE] scheme satisfies SIM-security, assuming that the underlying FH-IPFE satisfies SIM-security.*

Proof. Let A be an adversary against AC-IPFRE[FH-IPFE]. We first construct an adversary B against SIM-security of FH-IPFE in Figure 26.

Next, by SIM-security of FH-IPFE, for an adversary B there exists a simulator $S_B = (S_{B,1}, S_{B,2}, S_{B,3}, S_{B,4})$. Then, consider the simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$ for AC-IPFRE[FH-IPFE] for an adversary A also given in Figure 26.

The simulator $S_{A,3}$ gets the collection $\mathcal{C}'_{ip}$ that contains ‘accessible’ inner products of AC-IPFRE[FH-IPFE] ciphertexts with each other using a global index to keep track of the queries and $\mathcal{T}$ that contains the tags of each ciphertext. The simulator turns this into $\mathcal{M}_{ip}$ that still uses a global index but keeps track of (ENC or KEYGEN) queries. Since a ciphertext with tag C uses both a ciphertext and function key, the simulator uses an offset δ to keep track of the indices. When adding new tuples to $\mathcal{C}_{ip}$, the simulator must keep track of ciphertexts of tag C separately and only add new values right before running $S_{B,3}$ or $S_{B,4}$ with an updated $\mathcal{C}_{ip}$ as given in simulator lines 15-30.

With this, we see

$$\Pr[\mathbf{G}_{AC-IPFRE[FH-IPFE],A}^{\text{sim-1}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{FH-IPFE,B}^{\text{sim-1}}(\lambda) \Rightarrow 1]$$

and

$$\Pr[\mathbf{G}_{AC-IPFRE[FH-IPFE],A,S_A}^{\text{sim-0}}(\lambda) \Rightarrow 1] = \Pr[\mathbf{G}_{FH-IPFE,B,S_B}^{\text{sim-0}}(\lambda) \Rightarrow 1].$$

Subtracting yields the result. $\square$

C.2 ANN-ODB Security Proof

Theorem 15. *Our ANN ODB protocol ANN-ODB[AC-IPFRE] is SIM-secure under the leakage profile $\text{LP}_{\text{ANN-ODB}} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Update}})$ given in Figure 17, assuming that AC-IPFRE is a SIM-secure ACIPFRE for access policy a_{ann} .*

Consider the games given in Figure 27.

Now, we consider an adversary $B = (B_1, B_2)$ interacting with the games given in Figure 3 using an adversary $A = (A_1, A_2)$ against the games given in Figure 27. This adversary is given in Figure 28.

We also consider a simulator $S = (\text{SimPPGen}, \text{SimSetup}, \text{SimQuery}, \text{SimUpdate})$ interacting with the game given in Figure 27 (right) that uses an AC-IPFRE simulator $S_{AC-IPFRE} = (\text{SimPPGen}, \text{SimSetup}, \text{SimEnc})$ for the access policy a_{ann} . This simulator is given in Figure 29.

We can see that when the adversary B is the sim-1 AC-IPFRE game, it perfectly simulates the sim-1 ANN-ODB[AC-IPFRE] game for the adversary A . On the other hand, when the adversary B is in the sim-0 AC-IPFRE game with a simulator $S_{AC-IPFRE}$, it simulates the sim-0 ANN-ODB[AC-IPFRE] game for the adversary A with the simulator

Adversary $B = (B_1, B_2)$	Simulator $S_A = (S_{A,1}, S_{A,2}, S_{A,3})$
$B_1(1^\lambda, \text{pp}):$ 1 $(d, st) \leftarrow \$ A_1(1^\lambda, \text{pp})$ 2 return (d, st) $B_2^{\text{ENCO}'(\cdot), \text{KEYGENO}'(\cdot)}(st):$ 3 $b \leftarrow \$ A_2^{\text{ENCO}'(\cdot, \cdot)}(st)$ 4 return b $\text{ENCO}(\mathbf{x}, \text{tag}):$ 5 if $\text{tag} = \text{data} :$ 6 $\text{ct}_x \leftarrow \text{ENCO}'(\mathbf{x})$ 7 $\text{ct} \leftarrow (\perp, \text{ct}_x)$ 8 if $\text{tag} = \text{query} :$ 9 $\text{sk}_x \leftarrow \text{KEYGENO}'(\mathbf{x})$ 10 $\text{ct} \leftarrow (\text{sk}_x, \perp)$ 11 if $\text{tag} = \text{update} :$ 12 $\text{sk}_x \leftarrow \text{KEYGENO}'(\mathbf{x})$ 13 $\text{ct}_x \leftarrow \text{ENCO}'(\mathbf{x})$ 14 $\text{ct} \leftarrow (\text{sk}_x, \text{ct}_x)$ 15 return ct	$S_{A,1}(1^\lambda):$ 1 $(\text{pp}, st_B) \leftarrow \$ S_{B,1}(1^\lambda)$ 2 $st \leftarrow st_B$ 3 return (pp, st) $S_{A,2}(1^d, st):$ 4 $i \leftarrow 0; E, K \leftarrow \emptyset$ 5 $\delta \leftarrow 0$ 6 $st_B \leftarrow st$ 7 $st_B \leftarrow \$ S_{B,2}(1^d, st_B)$ 8 $st \leftarrow (st_B, k)$ 9 return st $S_{A,3}(C'_{ip}, \mathcal{T}, st):$ 10 $(st_B, k) \leftarrow st$ 11 $i \leftarrow i + 1; \text{sk}, \text{ct} \leftarrow \perp$ 12 $\text{tag} \leftarrow \mathcal{T}[i]$ 13 if $\text{tag} = \text{data}$ or $\text{tag} = \text{update} :$ 14 then $E \leftarrow E \cup \{i\}$ 15 if $\text{tag} = \text{query}$ or $\text{tag} = \text{update} :$ 16 then $K \leftarrow K \cup \{i\}$ 17 if $\text{tag} = \text{data} :$ 18 for $j \in K :$ 19 $C_{ip}[i + \delta][j], C_{ip}[j][i + \delta] \leftarrow C'_{ip}[i][j]$ 20 $(\text{ct}, st_B) \leftarrow S_{B,4}(C_{ip}, st_B)$ 21 if $\text{tag} = \text{query} :$ 22 for $j \in E :$ 23 $C_{ip}[i + \delta][j], C_{ip}[j][i + \delta] \leftarrow C'_{ip}[i][j]$ 24 $(\text{sk}, st_B) \leftarrow S_{B,3}(C_{ip}, st_B)$ 25 if $\text{tag} = \text{update} :$ 26 for $j \in E :$ 27 $C_{ip}[i + \delta][j], C_{ip}[j][i + \delta] \leftarrow C'_{ip}[i][j]$ 28 $(\text{sk}, st_B) \leftarrow S_{B,3}(C_{ip}, st_B)$ 29 $\delta \leftarrow \delta + 1$ 30 for $j \in K :$ 31 $C_{ip}[i + \delta][j], C_{ip}[j][i + \delta] \leftarrow C'_{ip}[i][j]$ 32 $(\text{ct}, st_B) \leftarrow S_{B,4}(C_{ip}, st_B)$ 33 $st \leftarrow (st_B, k)$ 34 return $((\text{sk}, \text{ct}), st)$

Figure 26: Adversary against SIM-security of FH-IPFE and simulator for SIM security of AC-IPFRE[FH-IPFE].

Game $\mathbf{G}_{\text{ANN-ODB[AC-IPFRE]},A}^{\text{sim-1}}(\lambda)$	Game $\mathbf{G}_{\text{ANN-ODB[AC-IPFRE]},A,S}^{\text{sim-0}}(\lambda)$
MAIN: 1 $\text{pp} \leftarrow \$ \text{AC-IPFRE.PPGen}(1^\lambda)$ 2 $(\mathcal{D}_{\text{init}}, st) \leftarrow \$ A_1(1^\lambda, 1^n, 1^d, \text{pp})$ 3 $\text{msk} \leftarrow \text{AC-IPFRE.Setup}(1^\lambda, 1^d)$ 4 $\mathcal{D} \leftarrow \text{Shuffle}(\mathcal{D}_{\text{init}})$ 5 for $i \in [n]$: 6 $\mathcal{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_i)$ 7 $\text{ct}_i \leftarrow \$ \text{AC-IPFRE.Enc}(\text{msk}, \text{data}, \mathcal{D}[\text{ID}_i])$ 8 $\mathcal{D}_S[\text{ID}_i] \leftarrow \text{ct}_i$ 9 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_2(\mathcal{DS}_{\text{ANN}}, (\text{ct}_i)_{i \in [n]}, st)$ 10 while $\text{op}_{\text{type}} \neq \text{terminate}$: 11 if $\text{op}_{\text{type}} = \text{Query}$: 12 $(\mathbf{q}, \text{ID}) \leftarrow \text{op}$ 13 $\text{ct}_{\mathbf{q}} \leftarrow \$ \text{AC-IPFRE.Enc}(\text{msk}, \text{query}, \mathbf{q})$ 14 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_3((\text{ct}_{\mathbf{q}}, \text{ID}), st)$ 15 if $\text{op}_{\text{type}} = \text{Update}$: 16 $(\mathbf{u}, \text{ID}) \leftarrow \text{op}$ 17 $\text{ct}_{\mathbf{u}} \leftarrow \$ \text{AC-IPFRE.Enc}(\text{msk}, \text{update}, \mathbf{u})$ 18 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_4((\text{ct}_{\mathbf{u}}, \text{ID}), st)$ 19 if $\text{op}_{\text{type}} = \text{terminate}$: $b \leftarrow \text{op}$ 20 return b Shuffle $(\mathcal{D}_{\text{old}})$: 21 $\mathcal{D}_{\text{new}} \leftarrow \emptyset$ 22 $n \leftarrow \text{length}(\mathcal{D}_{\text{old}})$ 23 $\sigma \leftarrow \$ S_n$ 24 for $i \in [n]$: 25 $\mathcal{D}_{\text{new}}[\text{ID}_i] \leftarrow \mathcal{D}_{\text{old}}[\text{ID}_{\sigma(i)}]$ 26 return $\mathcal{D}_{\text{new}}$ COMPO $(\text{ID}_i, \text{ID}_j, \text{ID}_k)$: 27 $\mathbf{x}_i \leftarrow \mathcal{D}[\text{ID}_i]; \mathbf{x}_j \leftarrow \mathcal{D}[\text{ID}_j]; \mathbf{x}_k \leftarrow \mathcal{D}[\text{ID}_k]$ 28 if $\langle \mathbf{x}_i, \mathbf{x}_j \rangle > \langle \mathbf{x}_i, \mathbf{x}_k \rangle$: 29 return 1 30 else : 31 return 0	MAIN: 1 $(\text{pp}, st_S) \leftarrow \$ S.\text{SimPPGen}(1^\lambda)$ 2 $(\mathcal{D}_{\text{init}}, st_A) \leftarrow \$ A_1(1^\lambda, 1^n, 1^d, \text{pp})$ 3 $\mathcal{D} \leftarrow \text{Shuffle}(\mathcal{D}_{\text{init}})$ 4 $\ell_{\text{Setup}} \leftarrow \mathcal{L}_{\text{Setup}}(\mathcal{D}, \perp)$ 5 $(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S) \leftarrow \$ S.\text{SimSetup}(\ell_{\text{Setup}}, st_S)$ 6 $(\text{op}, \text{op}_{\text{type}}, st_A) \leftarrow \$ A_2(\mathcal{DS}_{\text{ANN}}, \mathcal{D}_S, st_A)$ 7 while $\text{op}_{\text{type}} \neq \text{terminate}$: 8 if $\text{op}_{\text{type}} = \text{Query}$: 9 $(\mathbf{q}, \text{ID}) \leftarrow \text{op}; \mathcal{Q}[\text{ID}] \leftarrow \mathbf{q}$ 10 $\ell_{\text{Query}} \leftarrow \mathcal{L}_{\text{Query}}((\mathbf{q}, \text{ID}), \mathcal{D})$ 11 $(\text{ct}_{\mathbf{q}}, st_S) \leftarrow \$ S.\text{SimQuery}(\ell_{\text{Query}}, st_S)$ 12 $(\text{op}, \text{op}_{\text{type}}, st_A) \leftarrow \$ A_3((\text{ct}_{\mathbf{q}}, \text{ID}), st_A)$ 13 if $\text{op}_{\text{type}} = \text{Update}$: 14 $(\mathbf{u}, \text{ID}) \leftarrow \text{op}; \mathcal{D}[\text{ID}] \leftarrow \mathbf{u}$ 15 $\ell_{\text{Update}} \leftarrow \mathcal{L}_{\text{Update}}((\mathbf{u}, \text{ID}), \mathcal{D}, \mathcal{Q})$ 16 $(\text{ct}_{\mathbf{u}}, st_S) \leftarrow \$ S.\text{SimUpdate}(\ell_{\text{Update}}, st_S)$ 17 $(\text{op}, \text{op}_{\text{type}}, st_A) \leftarrow \$ A_4((\text{ct}_{\mathbf{u}}, \text{ID}), st_A)$ 18 if $\text{op}_{\text{type}} = \text{terminate}$: $b \leftarrow \text{op}$ 19 return b Shuffle $(\mathcal{D}_{\text{old}})$: 20 $\mathcal{D}_{\text{new}} \leftarrow \emptyset$ 21 $n \leftarrow \text{length}(\mathcal{D}_{\text{old}})$ 22 $\sigma \leftarrow \$ S_n$ 23 for $i \in [n]$: 24 $\mathcal{D}_{\text{new}}[\text{ID}_i] \leftarrow \mathcal{D}_{\text{old}}[\text{ID}_{\sigma(i)}]$ 25 return $\mathcal{D}_{\text{new}}$

Figure 27: ANN-ODB[FH-IPFE] Security.

S given in Figure 29 (right). Thus, ANN-ODB[AC-IPFRE] is persistent simulation secure under the given leakage profile.

Adversary B

$B_1(1^\lambda, \text{pp})$:

- 1 $(\mathcal{D}_{\text{init}}, st) \leftarrow \$ A_1(1^\lambda, 1^n, 1^d, \text{pp})$
- 2 $st \leftarrow \$ \mathcal{D}_{\text{init}}$
- 3 **return** (d, a_{ann}, st)

$B_2^{\text{ENCO}}(st)$:

- 4 $\mathcal{D}_{\text{init}} \leftarrow st$
- 5 $\mathcal{D} \leftarrow \text{Shuffle}(\mathcal{D}_{\text{init}})$
- 6 **for** $i \in [n]$:
- 7 $\mathcal{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_i)$
- 8 $\text{ct}_i \leftarrow \$ \text{ENCO}(\mathcal{D}[\text{ID}_i], \text{data})$
- 9 $\mathcal{D}_S[\text{ID}_i] \leftarrow \text{ct}_i$
- 10 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_2(\mathcal{DS}_{\text{ANN}}, (\text{ct}_i)_{i \in [n]}, st)$
- 11 **while** $\text{op}_{\text{type}} \neq \text{terminate}$:
- 12 **if** $\text{op}_{\text{type}} = \text{Query}$:
- 13 $(\mathbf{q}, \text{ID}) \leftarrow \text{op}$
- 14 $\text{ct}_{\mathbf{q}} \leftarrow \$ \text{ENCO}(\mathbf{q}, \text{query})$
- 15 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_3((\text{ct}_{\mathbf{q}}, \text{ID}), st)$
- 16 **if** $\text{op}_{\text{type}} = \text{Update}$:
- 17 $(\mathbf{u}, \text{ID}) \leftarrow \text{op}$
- 18 $\text{ct}_{\mathbf{u}} \leftarrow \$ \text{ENCO}(\mathbf{u}, \text{update})$
- 19 $(\text{op}, \text{op}_{\text{type}}, st) \leftarrow \$ A_4((\text{ct}_{\mathbf{u}}, \text{ID}), st)$
- 20 **if** $\text{op}_{\text{type}} = \text{terminate}$: $b \leftarrow \text{op}$
- 21 **return** b

$\text{Shuffle}(\mathcal{D}_{\text{old}})$:

- 22 $\mathcal{D}_{\text{new}} \leftarrow \emptyset$
- 23 $n \leftarrow \text{length}(\mathcal{D}_{\text{old}})$
- 24 $\sigma \leftarrow \$ S_n$
- 25 **for** $i \in [n]$:
- 26 $\mathcal{D}_{\text{new}}[\text{ID}_i] \leftarrow \mathcal{D}_{\text{old}}[\text{ID}_{\sigma(i)}]$
- 27 **return** $\mathcal{D}_{\text{new}}$

$\text{COMPO}(\text{ID}_i, \text{ID}_j, \text{ID}_k)$:

- 28 $\mathbf{x}_i \leftarrow \mathcal{D}[\text{ID}_i]; \mathbf{x}_j \leftarrow \mathcal{D}[\text{ID}_j]; \mathbf{x}_k \leftarrow \mathcal{D}[\text{ID}_k]$
- 29 **if** $\langle \mathbf{x}_i, \mathbf{x}_j \rangle > \langle \mathbf{x}_i, \mathbf{x}_k \rangle$:
- 30 **return** 1
- 31 **else** :
- 32 **return** 0

Figure 28: Adversary for ANN-ODB[AC-IPFRE] simulation security.

```

Simulator S
S.SimPPGen( $1^\lambda$ ):
1  ( $\text{pp}, st$ )  $\leftarrow$   $\text{SAC-IPFRE.SimPPGen}(1^\lambda)$ 
2  return ( $\text{pp}, st$ )
S.SimSetup( $\ell_{\text{Setup}}, st$ ):
3   $\text{DS}_{\text{ANN}}.\text{Initialize}(d, p, \text{params})$ 
4  ( $\text{ip-comp}, \text{insert-order}$ )  $\leftarrow \ell_{\text{Setup}}$ 
5   $C_{\text{ip}}, \mathcal{T} \leftarrow \emptyset; i \leftarrow 0$ 
6   $st \leftarrow \text{SAC-IPFRE.SimSetup}(1^d, \perp, st)$ 
7  for  $j \in \text{insert-order}$  :
8     $\text{DS}_{\text{ANN}}.\text{Insert}^{\text{COMPO}(\cdot, \cdot, \cdot)}(\text{ID}_j)$ 
9    for  $k \in \text{insert-order}$  :
10      $C_{\text{ip}}[j][k], C_{\text{ip}}[k][j] \leftarrow \perp$ 
11      $i \leftarrow i + 1; \mathcal{T}[i] \leftarrow \text{data}$ 
12     ( $\text{ct}_i, st$ )  $\leftarrow \text{SAC-IPFRE.SimEnc}(C_{\text{ip}}, \mathcal{T}, st)$ 
13    $\mathcal{D}_S \leftarrow \{(\text{ct}_j, \text{ID}_j)\}_{j \in [n]}; \mathcal{D}_Q \leftarrow \emptyset$ 
14    $st_S \leftarrow (st, C_{\text{ip}}, \mathcal{T})$ 
15   return ( $\text{DS}_{\text{ANN}}, \mathcal{D}_S, st_S$ )
S.SimQuery( $\ell_{\text{Query}}, st_S$ ):
16    $\delta \leftarrow 0$ 
17   ( $st, C_{\text{ip}}, \mathcal{T}$ )  $\leftarrow st_S$ 
18    $i \leftarrow i + 1; \mathcal{T}[i] \leftarrow \text{query}$ 
19    $\text{ip-val} \leftarrow \ell_{\text{Query}}$ 
20   for  $j \in [i]$  :
21     if  $\mathcal{T}[j] = \text{query}$ :
22        $C_{\text{ip}}[i][j], C_{\text{ip}}[j][i] \leftarrow \perp; \delta \leftarrow \delta + 1$ 
23     else :  $C_{\text{ip}}[i][j], C_{\text{ip}}[j][i] \leftarrow \text{ip-val}[j + \delta]$ 
24   ( $\text{ct}_q, st$ )  $\leftarrow \text{SAC-IPFRE.SimEnc}(C_{\text{ip}}, \mathcal{T}, st)$ 
25    $st_S \leftarrow (st, C_{\text{ip}}, \mathcal{T})$ 
26   return ( $\text{ct}_q, st_S$ )
S.SimUpdate( $\ell_{\text{Update}}, st_S$ ):
27   ( $st, C_{\text{ip}}, \mathcal{T}$ )  $\leftarrow st_S$ 
28    $i \leftarrow i + 1; \mathcal{T}[i] \leftarrow \text{update}$ 
29    $\text{ip-val} \leftarrow \ell_{\text{Update}}$ 
30   for  $j \in [i]$  :
31      $C_{\text{ip}}[i][j], C_{\text{ip}}[j][i] \leftarrow \text{ip-val}[j]$ 
32   ( $\text{ct}_u, st$ )  $\leftarrow \text{SAC-IPFRE.SimEnc}(C_{\text{ip}}, \mathcal{T}, st)$ 
33   return ( $\text{ct}_u, st_S$ )
COMPO( $\text{ID}_i, \text{ID}_j, \text{ID}_k$ ):
34   return  $\text{ip-comp}[\text{ID}_i][\text{ID}_j][\text{ID}_k]$ 

```

Figure 29: Simulator for ANN-ODB[AC-IPFRE] simulation security.

D Extension to dynamic access control policies with unbounded IPBFE

In this Section we sketch how to enhance our transformations to achieve a small universe AC-IPFRE where tags can be added dynamically. Towards this, we have to overcome two challenges. Firstly, we have to use an encoding such that the vector for the i -th tag can be computed given the access policy only for the first i tags. We present such an encoding at the end of this Section. This encoding has $\tau = 5n$ and is thus more efficient than the encoding used in Section 4.2 if $m > n$.

This change alone allows to define new tags dynamically, but the total number of tags still has to be fixed during setup. We can avoid this by adapting our transformation from FH-IPBFE to IPBFRE and then to NZIP-AC-IPFRE to an unbounded version.

Unbounded IPFE [TT18, DP19] is a variant of IPFE where the dimension of the vector space does not have to be fixed during setup. For the purpose of this work, we focus only on what is called in taxonomy of [TT18] the (E:con, K:con, D:ct-dom) variant. E:con, K:con means that the vectors for encryption and key generation have to be continuous, which means that for each vector there has to be a polynomial bound d such that only the first d entries of the vector can be non-zero. This is in contrast to the E:sep or K:sep variants, where these vectors can be any vectors from an exponential dimensional vector space with polynomial Hamming weight, described by an index-value map containing all non-zero entries.

D:ct-dom means that decryption is only possible if the encrypted vector is at least as long the vector used to generate the functional secret key. The secret key vector is then implicitly filled up with 0's for the missing entries.

The work [TT18] presents an unbounded (E:con, K:con, D:ct-dom) FH-IPFE that can be used to instantiate the transformations sketched here.

Plugging in such an unbounded FH-IPBFE in the FH-IPBFE to IPBFRE transform from Section A.1 yields an IPBFRE for unbounded, continuous ciphertexts. Note that, despite the D:ct-dom restriction for the underlying FH-IPBFE, we get an IPBFRE where the inner product between *every* pair of IPBFRE ciphertexts is revealed, because every IPBFRE ciphertext consists of a FH-IPBFE ciphertext and a functional FH-IPBFE secret key. The inner product then gets revealed because the FH-IPBFE ciphertext for the longer vector can be decrypted with the functional FH-IPBFE secret key for the shorter vector.

Next, if we plug in such an unbounded IPBFRE into our IPBFRE to NZIP-AC-IPBFRE transform from Section 4.1, we get an NZIP-AC-IPBFRE where *either* the dimension of the plaintext vectors *or* the dimension of the tag vectors can be unbounded. The version with unbounded plaintext vectors directly gives us an unbounded small universe AC-IPBFRE via the transformation in Section 4.2. The version with unbounded tag vectors gives us a small-universe AC-IPBFRE where we can dynamically add tags. The latter also requires some modifications to the encoding of the access policy graph as vectors, to ensure that adding a new tags only requires to append (implicitly) 0's to the existing vectors. When deploying this scheme, we have to ensure that all users agree on the *order* in which the new tags are added and how the access policy is defined on these new tags with respect to existing tags, because the encoding as vectors depends on this.

Next, we describe this modified encoding. We describe how to construct the vector $\mathbf{u}_i$ for the i -th tag, depending on the vectors $\mathbf{u}_1, \dots, \mathbf{u}_{n-1}$. We use $\mathbf{u}_{i,j}$ to denote the j -th entry of the vector $\mathbf{u}_i$ and $\mathbf{u}_{i,\leq j}$ to denote the vector $\mathbf{u}_i$ truncated to its first j entries.

The idea of the encoding is to maintain the following two invariants when adding a new vectors

1. For all $i \in [n]$ $\mathbf{u}_{i,5(i-1)+1} = 1$ and all $j \in [i-1]$ $\mathbf{u}_{j,5(i-1)+1} = 0$
2. For all $i \in [n]$ and all $j \in [n] \setminus \{i\}$ and all $k \in [4]$ $\mathbf{u}_{j,5(i-1)+k+1} = 0$.

In the i -th vector, we proceed from $j = 1$ to $j = i - 1$ to use the entry $\mathbf{u}_{i,5(j-1)+1}$ to ensure that the inner product $\langle \mathbf{u}_j, \mathbf{u}_i \rangle$ has the desired value. The first invariant ensures that when we do this for the j -th vector, this does not change the inner products between $\mathbf{u}_1, \dots, \mathbf{u}_{j-1}$ and $\mathbf{u}_i$. Finally, we use $\mathbf{u}_{i,5(i-1)+2}, \dots, \mathbf{u}_{i,5(i-1)+5}$ to ensure $\langle \mathbf{u}_i, \mathbf{u}_i \rangle$ has the desired value with the help of Lagrange's four square theorem.

Formally, the construction can be described as follows:

$$\begin{aligned}
& \forall i \in [n] : \mathbf{u}_{i,5(i-1)+1} := 1 \\
& \forall i \in [n] \forall j \in [i-1] : \mathbf{u}_{i,5(j-1)+1} := -\langle \mathbf{u}_{i,\leq 5(j-1)}, \mathbf{u}_{j,\leq 5(j-1)} \rangle + b_{i,j} \\
& \text{where } b_{i,j} := \begin{cases} 1 & \text{if } \{v_j, v_i\} \in E \\ 0 & \text{otherwise} \end{cases} \\
& \forall i \in [n] \forall j \in [i-1] \forall k \in [4] : \mathbf{u}_{i,5(j-1)+1+k} := 0 \\
& \forall i \in [n] \forall k \in [4] : \mathbf{u}_{i,5(i-1)+1+k} := a_{i,k} \text{ where} \\
& (a_{i,k})_{k \in [4]} \leftarrow^s \text{LagrangeSoS}(-\langle \mathbf{u}_{i,\leq 5(i-1)+1}, \mathbf{u}_{i,\leq 5(i-1)+1} \rangle + b_i) \\
& \text{and } b_i := \begin{cases} 1 & \text{if } \{v_i\} \in E \\ 0 & \text{otherwise} \end{cases} \\
& \forall i \in [n] \forall j \in [5i+1, 5n] : \mathbf{u}_{i,j} := 0
\end{aligned}$$

Since the definition of $\mathbf{u}_i$ depends only on the access policy for the first i tags, we can add tags dynamically. Moreover, since $\mathbf{u}_i$ is only non-zero on the first $5i$ entries, we can use it with unbounded NZIP-AC-IPBFRE to avoid having to fix the number of tags during setup.

Next, we show that the inner products between these vectors correctly encode the access policy. For $i, j \in [n]$ with $i < j$ we have

$$\begin{aligned}
\langle \mathbf{u}_i, \mathbf{u}_j \rangle &= \langle \mathbf{u}_{i,\leq 5i}, \mathbf{u}_{j,\leq 5i} \rangle \\
&= \langle \mathbf{u}_{i,\leq 5(i-1)}, \mathbf{u}_{j,\leq 5(i-1)} \rangle - \langle \mathbf{u}_{i,\leq 5(j-1)}, \mathbf{u}_{j,\leq 5(j-1)} \rangle + b_{i,j} = b_{i,j}.
\end{aligned}$$

For $i \in [n]$ we have

$$\begin{aligned}
\langle \mathbf{u}_i, \mathbf{u}_i \rangle &= \langle \mathbf{u}_{i,\leq 5i}, \mathbf{u}_{i,\leq 5i} \rangle = \\
&= \langle \mathbf{u}_{i,\leq 5(i-1)+1}, \mathbf{u}_{i,\leq 5(i-1)+1} \rangle + a_{i,1}^2 + a_{i,2}^2 + a_{i,3}^2 + a_{i,4}^2 = b_i.
\end{aligned}$$